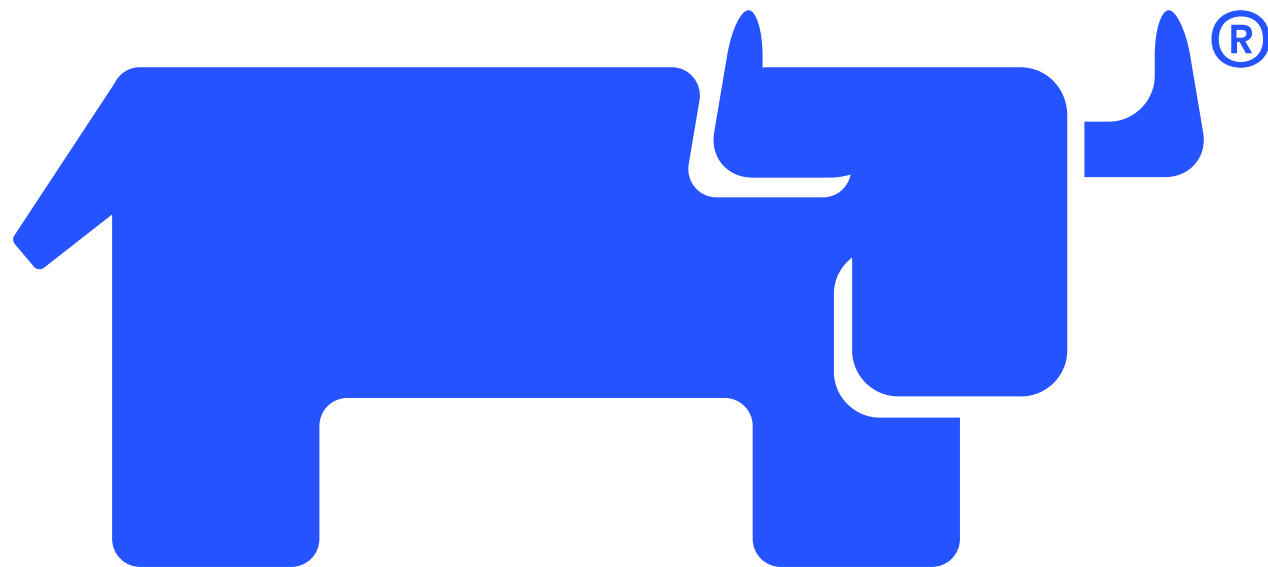




Rancher 101

소개

과정

담당자 및 강사

소개

01

다만 교육 진행 시, 다음과 같은 조건이 필요하다.

Docker 혹은 Podman과 같은 고수준 컨테이너 경험

OCI 표준도구 Buildah, Podman, Skopeo와 같은 도구 사용 경험이 있는 사용자

리눅스 커널 / 네트워크 / 스토리지 및 관리도구 사용 경험이 있는 사용자

02

기본적인 쿠버네티스 경험이 있는 사용자.

03

기업 수준의 오픈소스 컨테이너 시스템을 운영이 필요한 회사 혹은 엔지니어링 팀



강의일정

강의 일정은 아래와 같음.

DAY 1:

- RKE2 및 Rancher 설치

DAY 2:

- 랜처 자원 설명 및 활용

DAY 3:

- 랜처 및 하베스터 자원 설명 및 활용

DAY 4:

- 마지막 CICD 및 GPU활용을 위한 랜처/하베스터 활용



강사

강사

강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



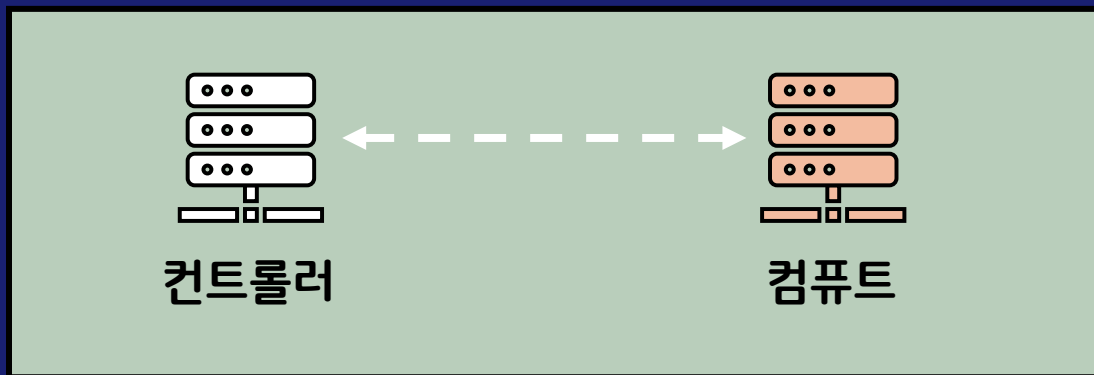
LAB

랩 및 교육대상

랩 대상

랩은 아래와 같이 구성이 되어 있습니다.

1. RKE2
2. 컨트롤러 x 1
3. 컴퓨트 x 2



master/worker node
eth1: vxlan, Canal Network

master/worker node
eth1: vxlan, Canal network

소개

Rancher

Rancher 소개

Rancher는 2014년도에 Rancher Labs 설립. 본래 목적은 쿠버네티스를 손쉽게 구축 및 설치가 주요 목적. 초기에는 Docker Swarm 및 Mesos와 같은 오케스트레이션 도구도 지원함.

2015년도, Docker기반으로 RKE1 베타를 출시하였고, 이때 자체 오케스트레이션 도구인 Cattle를 기본 제공.

2017년도, RKE2 개발을 시작 하였고, 이때 Docker에서 Kubernetes으로 전환 시작. 이때 Cattle를 포기 하였지만, 대시보드는 그대로 유지. RKE2 개발 시 주요 목적은 다중 클러스터 관리가 주요 목적.

2018년도, RKE2가 공식적으로 릴리즈 되었고, 이때는 K8S기반으로 설치 및 구성 지원.



Rancher 소개

2019년도에 RKE2전용으로 사용이 가능한 K3S릴리즈 하였으며, 주요 목적인 **IoT/Edge Computing** 목적으로 구성. 작은 크기로 빠르고 상대적으로 리소스를 적게 소모. 이러한 이유로 AI/ML에서 K8S보다 많이 사용함.

2020년도에 Rancher는 SUSE에 6억 달러로 인수. 이 시점으로 Rancher(RKE2)는 CNCF에 합류하면서 오픈소스 도구로 진입. 현재, 엔터프라이즈에서 레드햇 오픈시프트와 상대적인 대척점으로 경쟁 중.

이 이후, 수세는 Harvester, NeuVector, Fleet와 같은 여러 오픈소스에 투자 및 인수를 공격적으로 진행하고 있음. RKE2는 순수 업-스트림 버전으로 작성이 되기 때문에, 레드햇과 다르게 Vender Lock-In이 될 가능성이 현저히 낮음.



Rancher 소개

연도	내용
2014	Rancher Labs 설립
2015	Rancher 1.0 베타 출시 (Cattle 포함)
2016	Rancher 1.0 정식 출시, Kubernetes 지원 추가
2017	Rancher 2.0 개발 시작 (Kubernetes Only)
2018	Rancher 2.0 출시, RKE 도입
2019	K3s 발표 (경량 Kubernetes)
2020	SUSE, Rancher Labs 인수 (6억 달러)



HARVESTER

연도	내용
2021	Harvester 0.1 공개 (기술 프리뷰)
2022	Harvester 1.0 정식 출시
2023	Rancher 통합 개선, HCI 기능 강화
2024	네트워크 기능 향상, CSI 개선
2025	(예상) RKE2 및 Longhorn 통합 강화





Windows Hyper-V

오픈스택

오픈스택

오픈스택에서는 Heat Stack를 통해서 랩에서 사용하는 가상머신을 구성한다. RKE2라는 Stack를 실행하면, 랩에서 사용하는 자원을 생성한다.

1. 네트워크
2. 라우터
3. 인스턴스
4. 보안 그룹



설치

가상머신 구성

Rancher 설치 준비

네트워크 준비사항

Rancher 설치를 진행하기 위해서 다음과 같은 조건이 필요하다.

1. 한 대 이상의 가상머신 혹은 물리적 장치
2. VLAN으로 분리된 네트워크 혹은 물리적 스위치
3. 폐쇄망 구성 시, 내부에서 접근 가능한 이미지 레지스트리 혹은 RPM 저장소



지원 배포판

설치 시 사용이 가능한 운영체제는 다음과 같다.

1. SLES/SuSE Liberty, OpenSuSE
2. CentOS(EOL/EOS)
3. Rocky Linux
4. RHEL
5. Oracle Linux
6. Debian/Ubuntu Linux

강의 시, opensuse 혹은 rocky리눅스 기반으로 진행한다. 2~3회차 부터는 opensuse기반으로 진행 됩니다.



RKE1/2

설치 시, 클러스터 프로비저닝을 위한 조건은 다음과 같다.

1. RKE1, 현재는 레거시 및 커뮤니티에서는 K8S에 Docker를 더 이상 지원하지 않는다.(하위 containerd, cri-docker와 다른 부분)
2. RKE2, 쿠버네티스와 동일하게 바닐라 컴포넌트를 kubelet를 통해서 프로비저닝 및 관리한다.

위의 두 개의 차이점은 사이트에 다음과 같이 설명이 되어 있다.

"RKE1 uses Docker for deploying and managing control plane components, and it also uses Docker as the container runtime for Kubernetes. By contrast, RKE2 launches control plane components as static pods that are managed by the kubelet"



쿠버네티스

쿠버네티스를 구성하기 위해서 네트워크에 다음과 같은 구성이 필요하다.

이 구성은 최소 구성이며, 실제 프로덕션 상태에 따라서 다르다. 쿠버네티스 구성을 위해서 다음과 같은 조건이 필요하다.

1. kubelet이 API를 받기 위한 네트워크
2. POD가 사용 및 네트워크 구성을 위한 VXLAN/Geneve용도 네트워크 인터페이스



네트워크 프로토콜

쿠버네티스에서는 Geneve는 OVN를 통해서 사용이 가능하기 때문에, OVN구성이 필요하다. 그 이외에 쿠버네티스 CNI도구를 통해서 확장이 가능하다. RKE2에서 지원하는 네트워크는 다음과 같다.

이름	설명
Calico	VXLan기반으로 구성된 네트워크 데몬. 트래픽을 에이전트 기반으로 효율적으로 처리한다. 또한, SR-IOV/DPDK와 같은 기능을 소스코드 변경 없이 사용이 가능하다.
Flannel	쿠버네티스에서 기본적으로 제공하는 VXLan네트워크. 최대 호환성을 유지하며, 물리/가상 자원에서 전부 사용이 가능하다.
Cilium	VXLan기반으로 구성된 네트워크 데몬. 확장 기능을 제공하며, Calico와 거의 동일한 기능을 제공한다.
Canal	Calico와 함께 사용한다.



네트워크 프로토콜

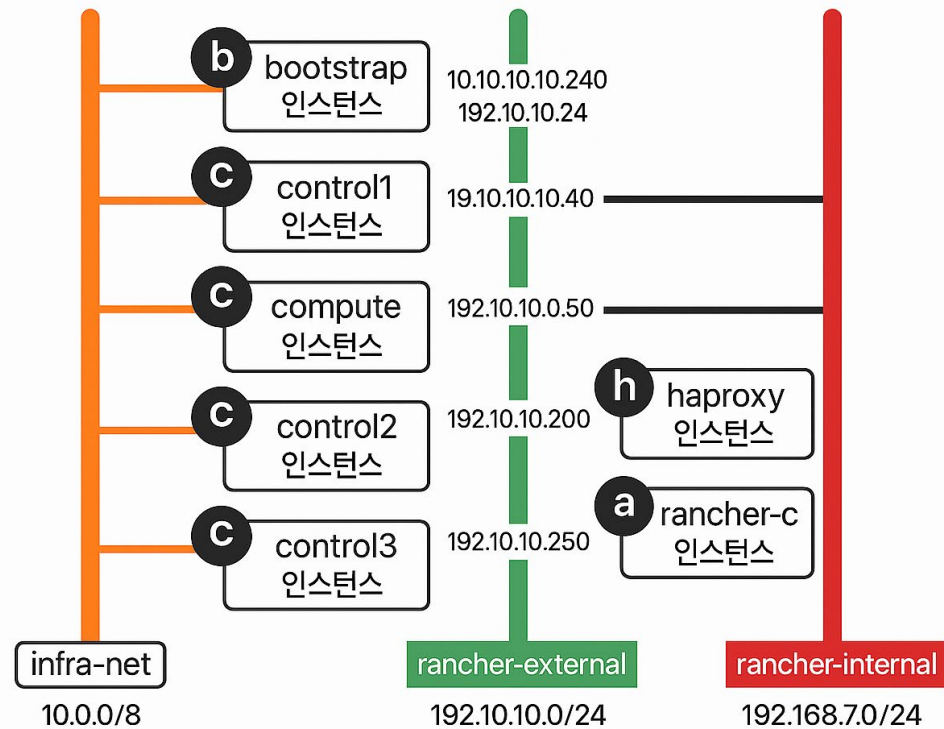
Rancher는 Canal기반으로 네트워크를 구성(기본값)한다. Canal으로 Flannel네트워크를 구성하여 클러스터에서 사용하도록 한다. 이외 네트워크는 기능 및 상황에 따라서 맞추어서 사용한다.

또한, 오픈스택에서 사용하는 경우, Flannel기반으로 구성을 추천한다. 그렇지 않는 경우, MTU 및 ToR기반으로 구성이 되어 있는 네트워크 시스템에서는 동작이 올바르게 되지 않는다.



랩 구성

이 부분은 강사 안내에 따라서 생성 및 구성한다. 자원 상황에 따라서 아래와 같이 구성한다.

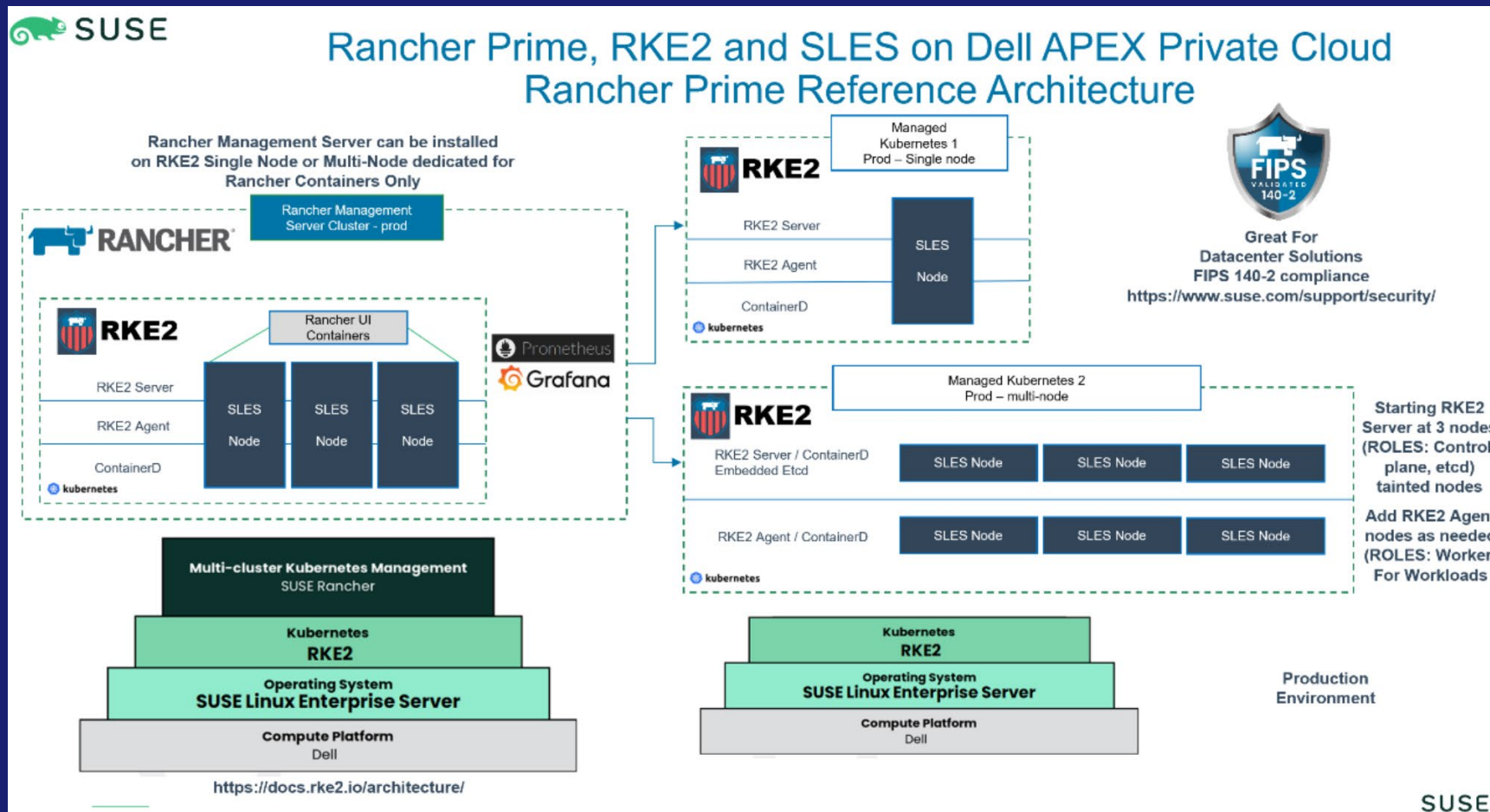


Rancher 아키텍처

RKE2/Kubernetes/OpenShift

K3S/K8S

Rancher 소프트웨어 아키텍처



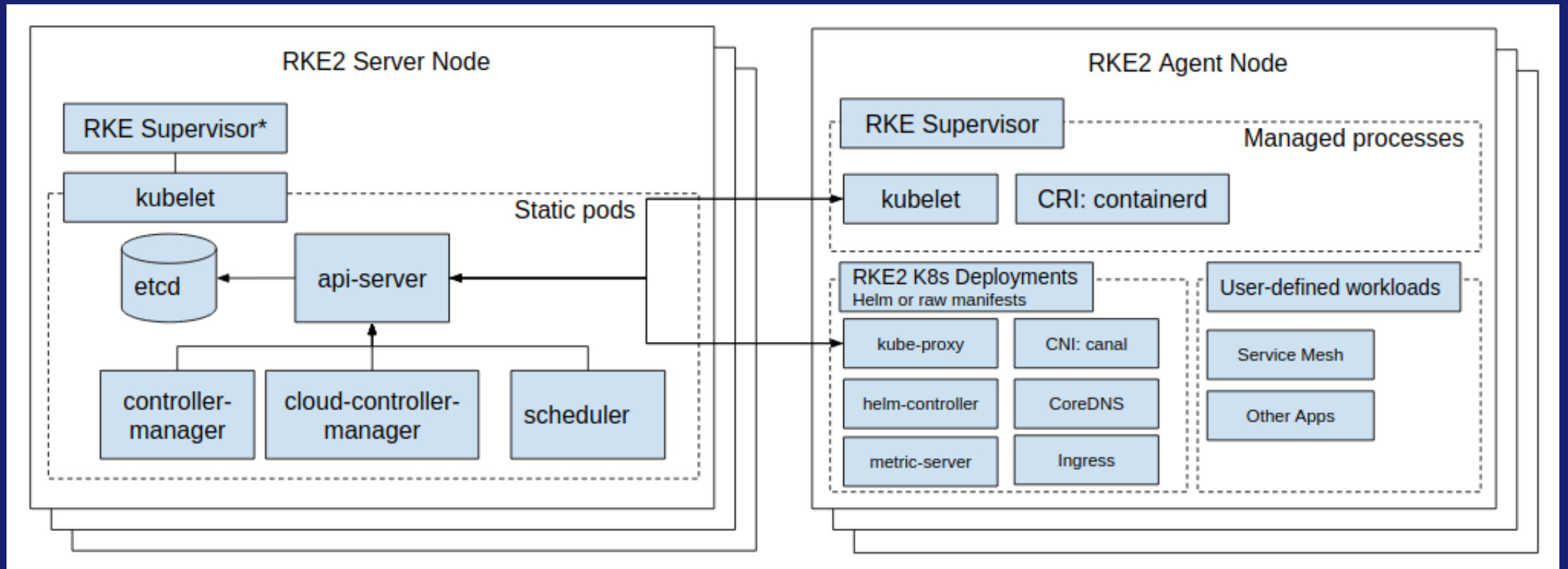
Rancher 소프트웨어 아키텍처

Rancher는 다음과 같은 소프트웨어 아키텍처를 가지고 있다.

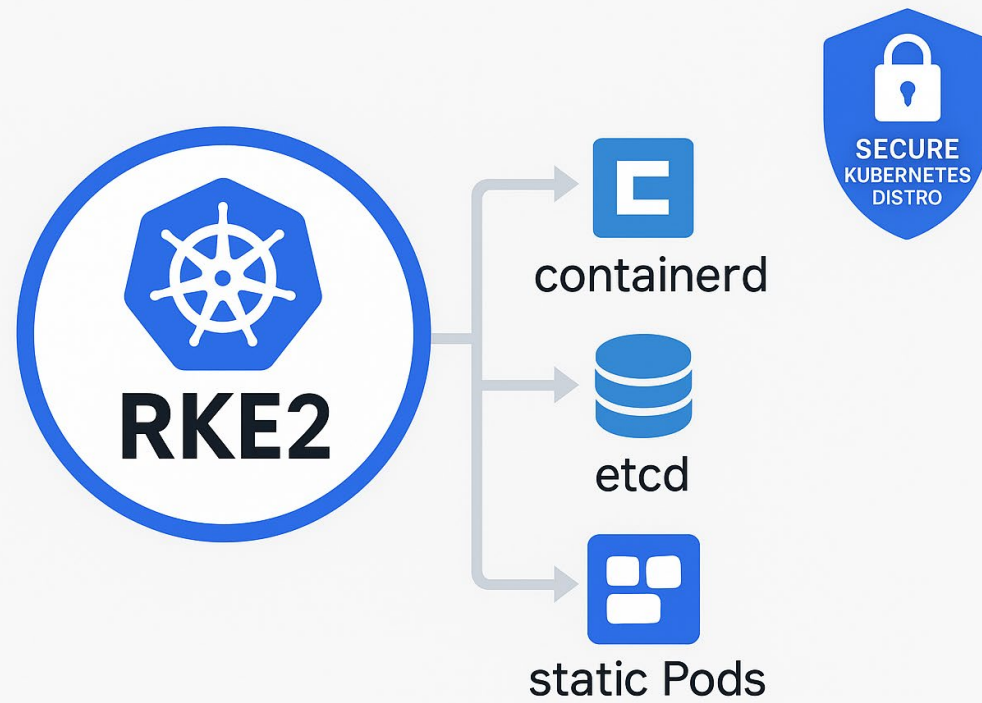
분류	이름	비고
OS	OpenSuSE/SLES	타 배포판 사용도 가능함
쿠버네티스	RKE2/K3S/K8S	K8S도 사용이 가능함 타 쿠버네티스 플랫폼에서 사용 가능
라이선스	오픈소스	업 스트림/다운 스트림 차이가 없음 마이그레이션 필요 없이 전환 가능
패키지 관리자	HelmChart	Operator 사용 가능 Helm Museum지원
다중 클러스터	지원	Kubernetes API와 호환 지원 가능
L4/L7	지원	하드웨어 혹은 소프트웨어로 구성 가능



RKE2



RKE2



RKE2

1. RKE2(Rancher Kubernetes Engine 2)는 보안 강화된 쿠버네티스 배포판
2. CIS Benchmarks, FIPS-140-2, SELinux 지원
3. 컨테이너 런타임: containerd
4. etcd 내장, HA 구성 가능
5. 기본 구성 요소는 static Pod로 실행



RKE2 서버/에이전트 구성

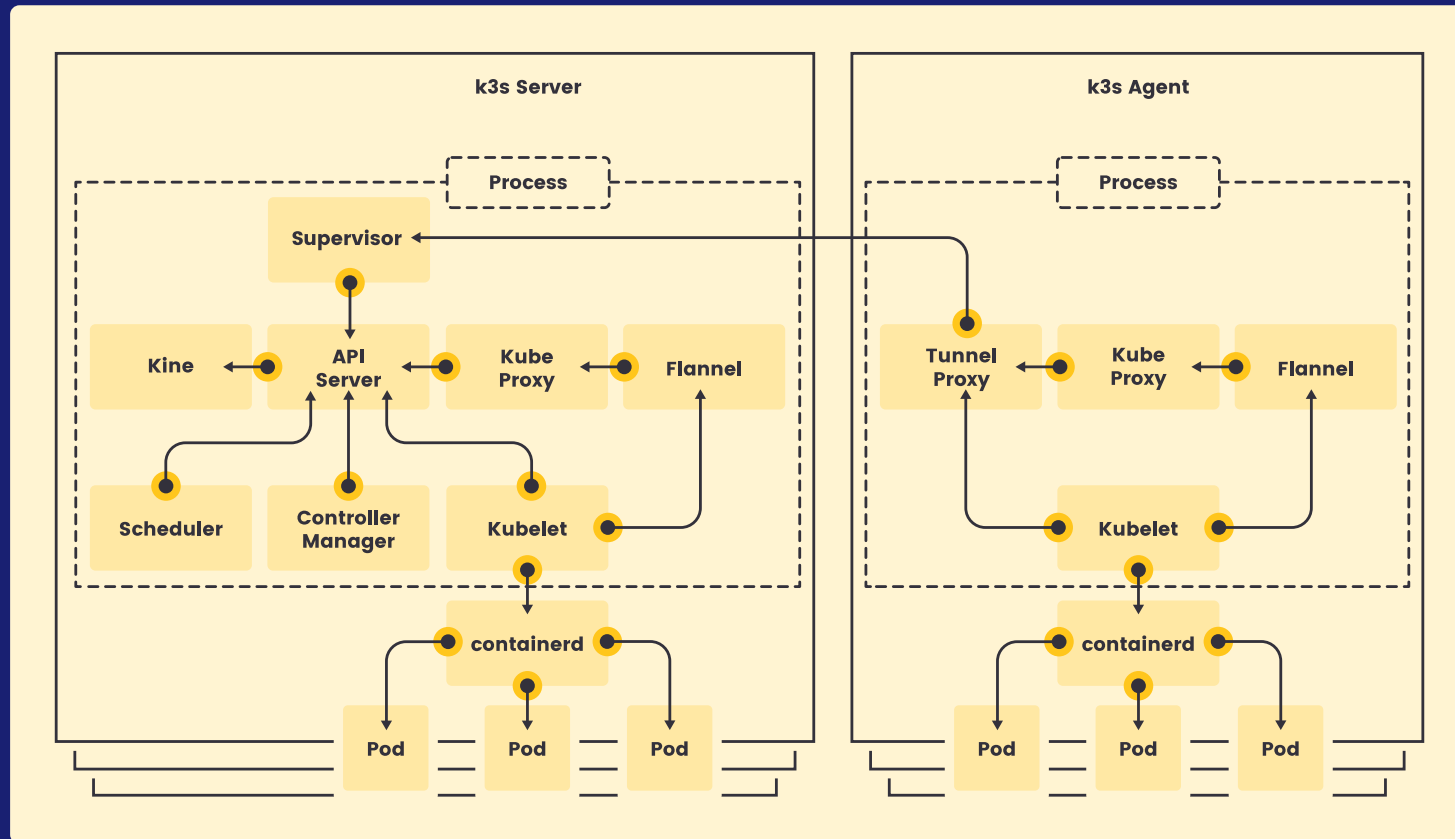
특이사항

1. server는 agent 역할도 포함함 (서버 = 컨트롤 + 워커 가능)
2. HA 클러스터 구성 시 서버 다중 구성 필요

구성 요소	역할
RKE2 Server	etcd 실행, Kubernetes control plane 역할 수행
RKE2 Agent	kubelet + containerd + CNI 실행. 워커 노드 역할 수행



K3S



비교

많이 사용하는 쿠버네티스 제품에 대해서 비교하면 보통 다음과 같이 평가가 된다.

플랫폼	설명
Rancher Kubernetes Engine 2	쿠버네티스 기반으로 구성된 보안 중심 컨테이너 오케스트레이션 도구
OpenShift	쿠버네티스 기반으로 구성된 레드햇 컨테이너 오케스트레이션 도구
Kubernetes	표준 오픈소스 컨테이너 오케스트레이션 도구



비교

일반적인 기능은 다음과 같은 차이가 있다.

기능	RKE2	OpenShift	Kubernetes
설치 편의성	쉬움 (스크립트 기반)	어려움 (앤서블/테라폼 혼용)	중간 (kubeadm/YAML)
보안	기본적으로 CIS 하드닝 적용	SELinux, SCC(보안 컨텍스트) 적용	운영체제에 따라서 다름
네트워크	CNI 플러그인 선택 가능 (Flannel, Calico 등)	OpenShift SDN (OVN, Multus)	표준 CNI플러그인 전부 지원
서비스 메쉬	Istio, Linkerd 사용 가능	기본 Istio 내장 (Service Mesh Operator)	표준 CNCF사용 가능

계속



비교

일반적인 기능은 다음과 같은 차이가 있다.

기능	RKE2	OpenShift	Kubernetes
스토리지	CSI 지원		
운영 관리	Rancher UI와 통합	OpenShift 콘솔 제공	kubectl 및 대시보드
업데이트 관리	Rancher UI를 통한 자동화	레드햇 가이드 기반으로 업데이트	수동 업데이트 필요
멀티클러스터	Rancher 기반 멀티클러스터 지원	OpenShift Cluster Manager 제공	Kubernetes Federation V1/V2지원
오버헤드	낮음 (경량화)	높음	보통
컨테이너 실행 속도	빠름 (Containerd 사용)	빠름 (CRI-O 사용)	빠름 (Containerd, CRI)
네트워크 성능	CNI에 따라 다름	OpenShift SDN, 느림	CNI에 따라 다름
업데이트 성능	빠른 롤링 업데이트	안정적이지만 복잡함	노드 단위 업데이트

계속



비교

RKE2의 최대 장점은 아래와 같다.

사용 목적	추천 플랫폼
보안 중심 Kubernetes 운영	RKE2 (기본 CIS 하드닝)
엔터프라이즈 환경	RKE2/OpenShift
순수 Kubernetes 운영 (유연성, 확장성 우선)	Kubernetes (Upstream)
멀티클러스터 및 하이브리드 클라우드	RKE2 / OpenShift
빠른 배포 및 경량 운영	RKE2
강력한 CI/CD 및 개발자 환경 제공	RKE2/OpenShift



설치 전 준비

설명

설명

Rancher를 설치하기 위해서 다음과 같은 **Configuration Options**를 참조 및 사용한다. 이 설정을 통해서 **kubelet**에서 클러스터 프로비저닝을 수행한다.

<https://docs.rke2.io/install/configuration>

이 설정은 `/etc/rancher/rke2/config.yaml` 참조한다. 여러 파일이 있는 경우, `config.yaml.d/` 디렉터리 밑에 알파벳 순서대로 불러오게 된다. 모든 설정은 YAML형식으로 작성 및 구성한다. 아래는 참고용.

```
# config.yaml
token: boop
node-label:
  - foo=bar
  - bar=baz
```



설명

노드에 스케줄링을 설정하기 위해서 아래와 같이 선언이 가능하다. 컨트롤러는 기본적으로 NoSchedule로 되어있다. write-kubeconfig-mode는 kubelet 파일 생성 시 적용되는 퍼미션.

```
# config.yaml.d/test1.yaml
write-kubeconfig-mode: 600
node-taint:
  - alice=bob:NoExecute
  - charlie=delta:NoSchedule
```



준비 및 적용(공통, 모든 노드)

커널 파라미터 조정이 필요하다. 아래와 같이 구성한다. 이 부분은 쿠버네티스에서 요구하는 사항이다. **rke2**는 **rke2-server**에서 **kernel parameter** 및 **kernel module**를 설정 및 구성한다. 하지만, 가급적이면 영구적으로 적용 하도록 한다.

```
# cat <<EOF | tee /etc/sysctl.d/99-k3s-net.conf
net.ipv4.ip_forward=1
net.ipv4.conf.all.send_redirects=0
net.ipv4.conf.default.send_redirects=0
net.ipv4.conf.default.accept_source_route=0
net.ipv4.conf.all.accept_redirects=0
net.ipv4.conf.default.accept_redirects=0
net.ipv4.conf.all.log_martians=1
```



준비 및 적용(공통, 모든 노드)

앞에 내용 이어서...

```
net.ipv6.conf.all.accept_ra=0
net.ipv6.conf.default.accept_ra=0
net.ipv6.conf.all.accept_redirects=0
net.ipv6.conf.default.accept_redirects=0
kernel.keys.root_maxbytes=25000000
kernel.keys.root_maxkeys=1000000
kernel.panic=10
kernel.panic_on_oops=1
```



준비 및 적용(공통, 모든 노드)

커널 파라미터 저장 및 적용.

```
vm.overcommit_memory=1
vm.panic_on_oom=0
net.ipv4.ip_local_reserved_ports=30000-32767
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-arptables=1
net.bridge.bridge-nf-call-ip6tables=1
EOF
# sysctl -q --system
```



준비 및 적용(공통, 모든 노드)

방화벽 및 SELinux와 같은 부분은 필요 없으니, 활성화가 되어 있으면 중지한다.

스왑이 동작중이면, 스왑을 끄고 설치를 진행 하도록 한다. 대다수 요구사항은 바닐라 쿠버네티스와 동일하다.

```
systemctl disable --now firewalld
setenforce 0
systemctl disable --now apparmor
swapon -s
swapoff -a
vi /etc/fstab
systemctl daemon-reload
```



준비 및 적용(공통, 모든 노드)

커널 모듈에 다음과 같은 모듈을 등록한다.

```
modprobe br_netfilter
modprobe overlay
cat <<EOF | tee /etc/modules-load.d/k8s.conf
br_netfilter
overlay
EOF

lsmod
systemctl daemon-reload
```



설치 준비 1

RKE2 설치하기 위해서 최소 한 개의 컨트롤러 혹은 부트스트랩 노드를 생성 및 구성한다. Rancher는 두 가지 설치 방법을 제공한다.

1. 기존 쿠버네티스 클러스터에 Rancher 설치 및 구성
2. RKE2를 통해서 쿠버네티스 설치 및 Rancher 구성



설치 준비 2

우리는 수세에서 제공하는 RKE2의 K3S기반으로 설치 및 구성한다. 간단하게 아래와 같이 RKE2도구 구성 및 설치 후 Rancher 설치를 수행한다.

```
curl -sfL https://get.rke2.io | INSTALL_RKE2_TYPE="server" sh -  
systemctl enable rke2-server.service
```

```
mkdir -p /etc/rancher/rke2  
cat <<EOF> /etc/rancher/rke2/config.yaml  
cni: "flannel"  
tls-san: example.com  
EOF
```



설치 준비 3

Rancher에서 사용하는 설정 파일은 `/etc/rancher/rke2`에 생성 및 구성한다. 클러스터 구성 시 사용하는 사용자 설정 파일은 `config.yaml`이다. 아래와 같이 작성 및 설치를 진행한다.

```
mkdir -p /etc/rancher/rke2
cat <<EOF /etc/rancher/rke2/config.yaml
cni: "flannel"
tls-san: example.com
EOF
```

CNI를 명시하지 않는 경우, 기본적으로 Canal로 구성이 된다. 오픈스택 및 SDN 네트워크에서 사용하는 경우, flannel으로 생성 및 구성한다.



설치 준비 4

설정이 완료가 되면, `rke2-server.service`를 시작하면, 쿠버네티스 클러스터를 프로비저닝을 수행한다. 내용이 변경이 되면, `rke2-server`를 재시작 한다. `rke2-server.service`는 `containerd`기반으로 동작한다.

```
systemctl start rke2-server.service  
systemctl restart rke2-server.service
```

컴퓨터 노드는 다음과 같이 추가 및 실행한다. 서비스 이름은 `rke2-agent.service`이다.

```
systemctl start rke2-agent.service  
systemctl restart rke2-agent.service
```



바이너리 배포 준비

위에 작업이 완료가 되면, 클러스터에 접근 하기 위해서 rke2에서 생성한 **kubeconfig** 및 **binary**도구를 셀 변수에 명시한다. 명시는 아래처럼 작업을 수행한다.

RKE2는 기본적인 도구를 미리 특정 위치에 배포하기 때문에, RPM기반으로 설치하는 방법과 많이 다르다. 사용자가 원하는 경우 RPM기반으로 설치가 가능하다.

일반적으로는 tar기반으로 구성 및 설치를 많이 사용하기 때문에, tar기반으로 설치 과정을 진행한다. RPM를 원하는 경우 Rancher2문서를 참고한다.



바이너리 도구 1

부트스트랩 및 컨트롤러 노드에는 아래와 같이 설정 및 생성하면, kubectl, rke2, rke2-install, rke2-uninstall 명령어 사용이 가능하다.

```
mkdir ~/.kube/  
export PATH=$PATH:/opt/rke2/bin  
cp /etc/rancher/rke2/rke2.yaml ~/.kube/config  
export PATH=$PATH:/var/lib/rancher/rke2/bin/  
echo 'export PATH=/usr/local/bin:/var/lib/rancher/rke2/bin:$PATH' >>  
~/.bashrc  
zypper install -y bash-completion
```



바이너리 도구 2

쿠버네티스에서 제공하는 `kubectl` 명령어 및 컨테이너를 생성 및 관리하는 CRI-O 관리 명령어를 사용하기 위해
서 아래와 같이 설정한다. 참고로 Rancher는 containerd 기반으로 부스 스트랩 구성이 된다.

자동완성 기능이 필요한 경우, 아래와 같이 작업을 수행한다.

```
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
echo 'export #
CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml' >> ~/.bashrc
```

계
속



ctr 바이너리

RKE2의 자원을 확인하기 위해서 containerd의 **ctr**명령어를 사용해야 한다. 아래와 같이 설정하면 containerd 자원 조회가 가능하다.

```
export CONTAINERD_ADDRESS=/run/k3s/containerd/containerd.sock
```

```
ctr container ls
```

```
systemctl status rke2-server.service
```

```
namespace k8s.io
```

```
ctr -n k8s.io container ls
```

```
ctr -n k8s.io task ls
```



추가설명

Ingress 서비스 구성이 필요하다면 다음과 같이 작업을 수행한다. Nginx를 사용하는 경우 header 포워딩을 위해서 아래와 같이 설정한다. 아래는 **메니페스트** 파일을 직접 수정하는 방식이다.

추가적인 설명들은 **메니페스트** 파일 수정을 권장하나, 가급적이면 유지보수를 위해서 기본 값 기반으로 설치 후, 변경을 권장한다.



인그레스 설정

메니페스트 파일은 다음과 같이 구성이 되어있다. 쿠버네티스 기본 패키지인 Helm으로 구성이 되어있다.

```
vi /var/lib/rancher/rke2/server/manifests/rke2-ingress-nginx-config.yaml
---
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-ingress-nginx
  namespace: kube-system
```



인그레스 설정

메니페스트 파일은 다음과 같이 구성이 되어있다. 쿠버네티스 기본 패키지인 Helm으로 구성이 되어있다.

```
valuesContent: |-  
  controller:  
    config:  
      use-forwarded-headers: "true"  
systemctl restart rke2-server
```



설치 시 설정 디렉터리 1

위의 내용 기반으로 아래와 같이 클러스터 설정 파일 작성이 가능하다.

```
mkdir -p /etc/rancher/rke2/  
cat <<EOF> /etc/rancher/rke2/config.yaml  
---  
write-kubeconfig-mode: "0644"  
advertise-address: ""  
server: ""  
token: ""  
node-name: bootstrap  
tls-san:  
  - 192.168.10.250  
  - example.com
```

tls-san에는 다음과 같이 권고한다.
1. 각 노드에서 사용하는 아이피
2. 외부 접근 시 사용하는 도메인 주소

계
속



설치 시 설정 디렉터리 2

위의 내용 기반으로 아래와 같이 클러스터 설정 파일 작성이 가능하다. 아래는 부트스트랩이 생성할 쿠버네티스 클러스터 네트워크 및 서비스 네트워크이다.

```
cluster-cidr: 192.168.0.0/16
service-cidr: 11.88.0.0/16
cluster-dns: 11.88.0.10
cluster-domain: rke2-cluster.example.com
etcd-arg: "--quota-backend-bytes 2048000000"
etcd-snapshot-schedule-cron: "0 3 * * *"
etcd-snapshot-retention: 10
disable: {}
```



설치 시 설정 디렉터리 3

추가적으로 kubernetes kubelet서비스에 옵션 변경 혹은 추가를 원하는 경우 아래와 같이 추가가 가능하다.

```
disable-kube-proxy: true
```

```
kube-apiserver-arg:
```

- '--default-not-ready-toleration-seconds=30'
- '--default-unreachable-toleration-seconds=30'

```
kube-controller-manager-arg:
```

- '--node-monitor-period=4s'

```
kubelet-arg:
```

- '--node-status-update-frequency=4s'
- '--max-pods=100'



설치 시 설정 디렉터리 4

특정 기능은 아래와 같이 명시가 가능하다. 여기서는 네트워크 및 커널 파라미터 그리고, egress에 대한 설정이다.

```
egress-selector-mode: disabled  
protect-kernel-defaults: true  
cni: "flannel"
```



ETCD 백업 주기(WAL)

etcd 백업 주기에 대해서 다음과 같이 설정이 가능하다.

```
services:
  etcd:
    backup_config:
      enabled: true
      interval_hours: 12
      retention: 6
      safe_timestamp: false
      timeout: 300
    creation: 12h
```



스케줄러 및 포트 번호

kube-proxy 및 schedule에서 자원 구성 시, 동작 방법에 대해서 아래와 같이 명시가 가능하다.

```
kube-api:
  always_pull_images: false
  pod_security_policy: false
  secrets_encryption_config:
    enabled: false
  service_node_port_range: 30000-32767
kube-controller: {}
kubelet:
  fail_swap_on: false
  generate_serving_certificate: false
kubeproxy: {}
scheduler: {}
```



기본적인 설정 1

자주 사용하는 내용을 정리하면 다음과 같다.

```
write-kubeconfig-mode: "0644"      ## kubelet파일 생성 시 퍼미션
advertise-address: 192.168.10.250   ## VIP이나 혹은 노드의 아이피
node-name: bootstrap                ## 생성 시 사용하는 노드의 이름
tls-san:                             ## 클러스터에서 사용할 도메인 및 아이피 주소
  - 192.168.10.250
  - example.com
cni: none                           ## 쿠버네티스에서 사용하는 네트워크 인터페이스
cluster-cidr: 192.168.0.0/16        ## 클러스터에서 사용하는 POD 네트워크
service-cidr: 10.88.0.0/16          ## 클러스터에서 사용하는 SVC 네트워크
cluster-dns: 10.88.0.10             ## CoreDNS가 사용하는 DNS 네트워크
cluster-domain: rke2-cluster.example.com ## 클러스터가 사용하는 클러스터 도메인
```



기본적인 설정 2

앞에서 사용한 `config.yaml`은 다음과 같이 구성이 되어있다.

`disable:`

- `rke2-ingress-nginx`

ingress 서비스를 사용하지 않는다

`egress-selector-mode: disabled`
서 접근이 안되는 경우가 있다

egress가 활성화가 되어 있으면, 보안 문제로 외부에

`protect-kernel-defaults: true`

커널 파라미터 수정 불가



기본적인 설정 3

앞에서 이야기 잠깐 설명하였던, taint를 통해서 클러스터에 구성된 노드 중, 컨트롤러에 실행 권한을 주지 않기 위해서 아래와 같이 `config.yaml`에 추가가 가능하다.

```
node-taint:  
  - "CriticalAddonsOnly=true:NoExecute"
```

Ingress서비스가 필요하지 않으면, 다음과 같이 명시한다.

```
disable:  
  - rke2-ingress-nginx      ## 이 부분을 주석처리 혹은 제거  
disable: {}                 ## disable할 서비스가 없으면 중괄호로 표시한다.
```



네트워크 설정

네트워크 플러그인(Container Network Interface)를 선언이 가능하다. 선언하지 않는 경우, RKE2에서 설치하지 않으며, 사용자가 수동으로 설치 및 구성이 가능하다. 여기서는 flannel으로 사용한다.

```
cni: none  
cni: flannel  
cni: calico  
cni: canal
```



오프라인 설치 구성 1

오프라인 설치 혹은 빠르게 프로비저닝을 원하는 경우, 아래에서 이미지 깃헙에서 이미지 내려받기가 가능하다. 명령어로 받는 경우, 아래처럼 실행한다.

 rke2-images-core.linux-amd64.tar.zst	638 MB	2 weeks ago
 rke2-images-core.linux-amd64.txt	899 Bytes	2 weeks ago
 rke2-images-core.linux-arm64.tar.gz	768 MB	2 weeks ago
 rke2-images-core.linux-arm64.tar.zst	597 MB	2 weeks ago
 rke2-images-core.linux-arm64.txt	899 Bytes	2 weeks ago



오프라인 설치 구성 2

이미지 내려받기가 완료가 되면, 아래 위치에 이미지를 이동한다. docker.io에서 이미지를 내려받기 때문에, 상황에 따라서 빠르게 내려받기가 되거나 혹은, 느린 경우가 있다.

이러한 상황이 자주 발생하면, 오프라인 이미지 기반으로 설치를 권장한다. 이미지 내려받기 시, docker.io사이트 상황에 따라서 최대 10+분 정도 소모가 된다.

```
mkdir -p /var/lib/rancher/rke2/agent/images/  
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz  
ls -l /var/lib/rancher/rke2/agent/images/
```



오프라인 설치 구성 3

Air-gapped에서 이야기 하지만 core image는 RKE2 설치에 필수 이미지만 가지고 있다. 전체 이미지 부분은 뒤에서 다루도록 한다.



참고

<https://docs.rke2.io/install/methods>

<https://github.com/rancher/rke2/discussions/4292>

<https://docs.rke2.io/import-images?import-images=Offline+image+importing>

HAPROXY(250) DNSMASQ

설치

도메인 서비스 구성(DNSMASQ)

Rancher A 레코드 설정.

```
zypper install -y dnsmasq or dnf install -y dnsmasq
cat <<EOF> /etc/dnsmasq.d/rancher.conf
domain=example.com
address=/dashboard.example.com/10.10.10.250
address=/rancher.example.com/10.10.10.250
address=/rancher-node1.example.com/10.10.10.X
address=/rancher-node2.example.com/10.10.10.X
address=/rancher-node3.example.com/10.10.10.X
cache-size=1000
listen-address=127.0.0.1,10.10.10.250
EOF
```



계속

도메인 서비스 구성(DNSMASQ)

Harvester A 레코드 설정.

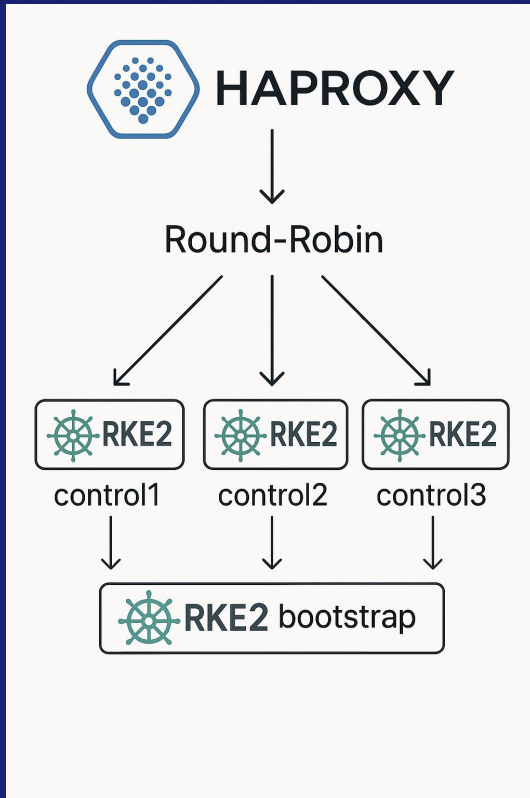
```
cat <<EOF> /etc/dnsmasq.d/harvester.conf
domain=example.com
address=/suse-virt-01.example.com/10.0.1.8
address=/suse-virt-02.example.com/10.0.3.66
address=/suse-virt-03.example.com/10.0.2.73
address=/harvester.example.com/10.10.10.240
listen-address=127.0.0.1,10.10.10.250
EOF
```

```
systemctl restart dnsmasq.service
```



HAPROXY 설치

L7를 위해서 간단하게 HAPROXY기반으로 구성한다.



HAPROXY 설치

HAProxy기반으로 L4를 구성한다. 컨트롤러로 구성할 컴퓨터의 아이피를 명시한다. 자세한 HAPROXY 설정은 공식 문서는 아래 링크를 참고한다.

<https://www.suse.com/ko-kr/support/kb/doc/?id=000020175>

```
zypper install -y haproxy  
dnf install -y haproxy  
vi /etc/haproxy/haproxy.cfg
```



HAPROXY 설정 1

```
frontend http_front
```

```
    bind *:80
```

```
    mode http
```

```
    option httplog
```

```
    acl rancher_host hdr(host) -i rancher.example.com dashboard.example.com
```

```
    acl harvester_host hdr(host) -i harvester.example.com
```

```
    use_backend rancher_cluster if rancher_host
```

```
    use_backend harvester_backend if harvester_host
```



HAPROXY 설정 2

```
frontend https_front
    bind *:443
    mode tcp
    option tcplog
    tcp-request inspect-delay 1s
    tcp-request content accept if { req_ssl_hello_type 1 }
    acl rancher_sni req_ssl_sni -i rancher.example.com dashboard.example.com
    acl harvester_sni req_ssl_sni -i harvester.example.com
    use_backend rancher_cluster_ssl if rancher_sni
    use_backend harvester_backend_ssl if harvester_sni
```



HAPROXY 설정 3

```
backend rancher_cluster
    mode http
    balance roundrobin
    server rancher1 10.0.2.209:80 check
    server rancher2 10.0.4.174:80 check
    server rancher3 10.0.4.129:80 check
```



HAPROXY 설정 4

```
backend harvester_backend
```

```
mode http
```

```
balance roundrobin
```

```
server harvester1 10.0.3.79:80 check
```

```
server harvester2 10.0.1.134:80 check
```

```
server harvester3 10.0.4.215:80 check
```



HAPROXY 설정 5

```
backend rancher_cluster_ssl
    mode tcp
    balance roundrobin
    option tcp-check
    server rancher1 10.0.2.209:443 check
    server rancher2 10.0.4.174:443 check
    server rancher3 10.0.4.129:443 check
```



HAPROXY 설정 6

```
frontend rke2_cluster_comm
    bind *:9345
    mode tcp
    option tcplog
    default_backend rke2_backend

backend rke2_backend
    mode tcp
    balance roundrobin
    option tcp-check
    server rancher1 10.0.3.218:9345 check
    server rancher2 10.0.4.246:9345 check
    server rancher3 10.0.1.61:9345 check
```



HAPROXY 설정 7

```
backend harvester_backend_ssl  
    mode tcp  
    balance roundrobin  
    option tcp-check  
    server harvester1 10.0.3.79:443 check  
    server harvester2 10.0.1.134:443 check  
    server harvester3 10.0.4.215:443 check
```



HAPROXY 설정 8

오류가 발생하면 기본 설정 중 global를 제외하고 나머지 제거 후 서비스 시작한다.

```
sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config
setenforce 0
systemctl disable --now firewalld
systemctl status haproxy.service
systemctl enable --now haproxy.service
```



NTP 클라이언트 구성

폐쇄망 경우 NTP서버를 반드시 구축해야 한다. chronyd 기반으로 서버 구축, systemd-timesyncd 기반으로 클라이언트 구축을 권장한다.

```
dnf install systemd-timesyncd -y
dnf install chronyd -y
vi /etc/chrony.conf
pool 2.rocky.pool.ntp.org iburst
allow 10.0.0.0/8
local stratum 10

systemctl restart chronyd.service
```



진행 전, 설치 팁

쉬운 설치

간단한 설치

bootstrap 구성이 복잡/필요 없다고 판단이 되면, Token를 직접 구성하여 구성하여도 상관 없다. 다만, RKE2 매뉴얼에 따라서 Token 및 API URL은 비워두고 bootstrap 구성을 권장한다.

1. bootstrap 노드 구성 권장; 하지만 필수는 아님
2. Token은 안전을 위해서 bootstrap 노드가 생성한걸 사용
 - 하지만 필수는 아님
3. API URL는 첫 번째 노드는 무조건 비워 있음
 - 역할상 bootstrap 노드로 인식
4. bootstrap노드는 컨트롤러 노드로 사용은 가능함
 - 하지만 권장하지는 않음
5. 간단한 설치에서는 CANAL 기반으로 CNI구성



첫 번째 노드

첫 번째 노드는 일반적으로 bootstrap노드로 구성이 된다.

```
cat <<EOF> ~/node-config.yaml
---
write-kubeconfig-mode: "0644"
token: "rancher"
server: ""
tls-san:
  - 10.10.10.250
node-ip: "10.0.1.130"
node-name: "rancher-node1.example.com"
disable-kube-proxy: false
```



첫 번째 노드

위의 내용 계속 이어서...

```
cluster-domain: "rancher.example.com"
```

```
cluster-cidr: "11.44.0.0/16"
```

```
service-cidr: "12.88.0.0/16"
```

```
cluster-dns: "12.88.0.10"
```

```
cluster-name: "rke2-lab-cluster"
```

```
EOF
```



첫 번째 노드

아래와 같이 빠른 구성하여도 상관 없다. 첫 번째 노드에 아래와 같이 구성한다.

```
curl -sL https://get.rke2.io | sh -  
mkdir -p /etc/rancher/rke2/  
cp node-config.yaml /etc/rancher/rke2/config.yaml  
sed -i 's/SELINUX=enforcing/SELINUX=permissive'/g /etc/selinux/config  
setenforce 0  
# hostnamectl set-hostname rancher-node1.example.com  
swapoff -a  
systemctl disable --now firewalld
```

계
속



첫 번째 노드

아래와 같이 빠른 구성하여도 상관 없다. 첫 번째 노드에 아래와 같이 구성한다.

```
mkdir -p /var/lib/rancher/rke2/agent/images/
```

```
dnf install wget -y
```

```
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz
```

```
systemctl enable --now rke2-server
```



나머지 노드(nodeX)

나머지 노드는 다음과 같이 구성한다.

```
cat <<EOF>> node-config.yaml
---
write-kubeconfig-mode: "0644"
token: "rancher"
server: "https://demo-cluster-1.example.com:9345"
tls-san:
  - 10.10.10.250
node-ip: "10.0.3.251"
node-name: "rancher-node2.example.com"
disable-kube-proxy: false
```



나머지 노드

위의 내용 계속 이어서...

```
cluster-domain: "rancher.example.com"
```

```
cluster-cidr: "11.44.0.0/16"
```

```
service-cidr: "12.88.0.0/16"
```

```
cluster-dns: "12.88.0.10"
```

```
EOF
```



나머지 노드

나머지 노드는 다음과 같이 실행한다.

```
curl -sfL https://get.rke2.io | sh -  
mkdir -p /etc/rancher/rke2/  
cp node-config.yaml /etc/rancher/rke2/config.yaml  
# hostnamectl set-hostname rancher-node2/3.example.com  
setenforce 0  
swapoff -a  
systemctl disable --now firewalld  
dnf install wget -y
```

계
속



나머지 노드

나머지 노드는 다음과 같이 실행한다.

```
sed -i 's/SELINUX=enforcing/SELINUX=permissive'/g /etc/selinux/config  
mkdir -p /var/lib/rancher/rke2/agent/images/  
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz  
systemctl enable --now rke2-server
```



참고

<https://docs.rke2.io/architecture#::~text=Architecture%20Overview%E2%80%8B%20With%20RKE2%20we%20take%20lessons,while%20sourcing%20needed%20content%20from%20the%20network.>

BOOTSTRAP(10)

설치

설치준비

설치를 진행 하기 위해서 컨트롤러 구성을 시작한다. 앞서 실행한 `install_rke2.sh`를 실행하면, 총 두 개의 서비스가 구성이 된다.

1. `rke2-agent`

2. `rke2-server`

1번은 컴퓨트 노드(흔히 워커노드 혹은 미니언)에서 사용하는 서비스. 2번은 컨트롤러 노드(흔히 마스터 노드)에서 사용하는 서비스.

처음 클러스터 구성 시, **부트스트랩 → 컨트롤러 → 컴퓨트** 같은 방식으로 구성해야 한다.

```
systemctl disable rke2-agent
systemctl mask rke2-agent
systemctl enable --now rke2-server.service
```



설치준비

DNS가 올바르게 조회가 되지 않는 경우(오픈스택 랩 한정), 아래와 같이 `/etc/resolv.conf`에 작성한다.

```
vi /etc/resolv.conf
```

```
nameserver 10.10.10.250      ## dnsmasq가 구성된 서버  
nameserver 8.8.8.8          ## 외부 DNS 접근이 필요한 경우
```

```
setenforce 0
```

```
sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config  
systemctl disable --now firewalld.service
```



systemd-resolved

systemd-resolved를 사용하는 경우, local만 인식이 되기 때문에 다음과 같이 설정한다.
기존 레거시 /etc/resolv.conf를 사용하는 경우 해당 사항이 없다.

```
vi config.yaml
resolved-conf: "/run/systemd/resolve/resolv.conf"
```



설치준비

설치를 좀 더 쉽게 하기 위해서 다음과 같은 부분은 옵션으로 사용한다.

컨트롤러에 일반 애플리케이션은 배포가 되지 않도록 하는 옵션. bootstrap노드에서 POD확인 히, 혼동이 되면 위의 옵션은 주석처리 후 설치 진행.

```
vi config.yaml
```

```
node-taint:
```

- "node-role.kubernetes.io/control-plane=true:Schedule"



설치준비

부트스트랩 서버에서 아래와 같이 실행한다. 스크립트 파일을 내려받기 후 사용하여도 상관없다.

실행이 완료가 되면, 부트스트랩 서버 구성을 위한 **config.yaml** 파일을 작성한다. 구별하기 위해서 홈 디렉터리에 **bootstrap.yaml** 파일 이름으로 작성 및 생성한다. 부트스트랩 노드는 설치 후, 제거가 되기 때문에 별도로 네트워크 설정은 하지 않는다.

최신 버전 기준으로 설치하는 경우, 별도로 버전 변수 명시가 필요 없다.

```
curl -sfL https://get.rke2.io | sh -
```



설치시작(BOOTSTRAP)

```
cat <<EOF bootstrap.yaml
```

```
---
```

```
write-kubeconfig-mode: "0644"
```

```
token: ""
```

```
server: ""
```

tls-san에 작성되는 아이피는 아키텍처에 따라서 달라집니다.

```
tls-san:
```

- rancher.example.com
- dashboard.example.com
- <RANCHER_NODES>



설치시작(BOOTSTRAP)

node-ip: "192.168.10.240"

node-name: "bootstrap-node"

node-external-ip: "192.10.10.240"

disable-kube-proxy: false

cni: "flannel"

cluster-domain: "rancher.example.com"

cni의 기본값은 canal 입니다.

계
속



설치시작(BOOTSTRAP)

```
#node-taint:
```

```
# - "node-role.kubernetes.io/control-plane=true:Schedule"
```

```
cluster-cidr: "11.44.0.0/16"
```

```
service-cidr: "12.88.0.0/16"
```

```
cluster-dns: "12.88.0.10"
```

```
cluster-name: "rke2-lab-cluster"
```

```
flannel-iface: "ens3"
```

아키텍처에 따라서 control-plane에
Schedule 제외가 됩니다.
bootstrap은 control, compute 둘 다 필요.

계
속



설치시작(BOOTSTRAP)

생성된 내용으로 아래와 같이 작업을 수행한다.

```
hostnamectl set-hostname bootstrap.example.com
export PATH=$PATH:/var/lib/rancher/rke2/bin/
export INSTALL_RKE2_VERSION="v1.30.0+rke2r1"
export INSTALL_RKE2_CHANNEL="stable"
export INSTALL_RKE2_TYPE="server"
curl -sfL https://get.rke2.io > install_rke2.sh
chmod ug+x install_rke2.sh
./install_rke2.sh
```



설치시작(BOOTSTRAP)

위의 내용 계속 이어서...

```
sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config  
systemctl disable --now firewalld  
setenforce 0  
mkdir -p /etc/rancher/rke2/  
cp bootstrap.yaml /etc/rancher/rke2/config.yaml
```



설치시작(BOOTSTRAP)

생성된 내용으로 아래와 같이 작업을 수행한다.

```
mkdir -p /var/lib/rancher/rke2/agent/images/  
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz  
systemctl enable --now rke2-server.service  
mkdir -p ~/.kube/  
cp /etc/rancher/rke2/rke2.yaml ~/.kube/config  
kubectl get node
```

air-gapped 설치를 위해서 이미지를 복사한다.



kubectl 자동완성

쿠버네티스에서 제공하는 `kubectl` 명령어 및 컨테이너를 생성 및 관리하는 `CRI-O` 관리 명령어를 사용하기 위해
서 아래와 같이 설정한다. 참고로 Rancher는 `containerd` 기반으로 부스 스트랩 구성이 된다.

자동완성 기능이 필요한 경우, 아래와 같이 작업을 수행한다.

```
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
echo 'export #
CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml' >> ~/.bashrc
```

계
속



Helm 명령어

Helm 명령어는 아래와 같이 설치한다. 보통 첫 번째 노드에서 kubectl, helm과 같은 명령어를 많이 사용한다.

```
dnf install git tar
```

```
zypper install git tar
```

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash
```



CoreDNS(조정)

Bootstrap, Controller

설명

CoreDNS는 쿠버네티스에서 사용하는 내부 DNS 서비스이다. 기존에 많이 사용하는 Bind9과 동일하다. 다만, 쿠버네티스에서는 많은 기능이 필요 없기 때문에 **CoreDNS**기반으로 A/AAAA/PTR과 같은 레코드를 생성 및 구성 관리한다.

현 클러스터는 레지스트리 서버가 없기 때문에 외부에서 이미지를 가져와야 한다. 이러한 이유로, 외부 DNS서버에 접근이 가능하도록 설정을 변경한다.



COREDNS 설정 방식

CoreDNS의 설정 변경이 필요한 경우 아래와 같이 수정이 가능하다. 다만, 어떠한 방식으로 수정 및 변경이 가능한지 정리한다.

수세 리눅스는 Static Pod Manifest를 통해서 접근이 가능하다.

구성 방식	적용 위치	사용 시점	특징
config.yaml	CoreDNS 직접 설정 불가	설치 시	제한적
Static Pod Manifest	/var/lib/rancher/rke2/server/manifests/	설치 전/후 모두	직접 적용 가능
HelmChartConfig	helm.cattle.io/v1 CRD	설치 후	공식 권장 방식
kubectrl edit	kubectrl -n kube-system edit configmap coredns	설치 후	수동/임시 변경



설정 변경

/var/lib/rancher/rke2/server/manifests/ 디렉터리에 아래와 같이 작성 및 배포한다.

```
cat <<EOF> 99-custom-coredns.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health
      ready
```



설정 변경

앞에 내용 계속 이어서...

```
hosts {  
    10.10.10.10 myapp.local  
    fallthrough  
}  
kubernetes cluster.local in-addr.arpa ip6.arpa {  
    pods insecure  
    fallthrough in-addr.arpa ip6.arpa  
}  
prometheus :9153  
forward . 8.8.8.8 1.1.1.1  
cache 30  
loop
```

계속



설정 변경

앞에 내용 계속 이어서...

```
        reload
        loadbalance
    }
EOF
# systemctl restart rke2-server
```



HelmChart

Chart로 수정하는 경우 아래와 같이 작성한다.

```
cat <<EOF> coredns-helmchartconfig.yaml
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-coredns
  namespace: kube-system
spec:
  valuesContent: |
    servers:
      - zones:
          - "."
        port: 53
```



HelmChart

Chart로 수정하는 경우 아래와 같이 작성한다.

```
plugins:  
  - name: errors  
  - name: health  
  - name: hosts  
    parameters: /etc/coredns/custom_hosts  
    fallthrough: true  
  - name: kubernetes  
    parameters: cluster.local in-addr.arpa ip6.arpa  
  - name: forward  
    parameters: . 8.8.8.8 1.1.1.1
```



HelmChart

Chart로 수정하는 경우 아래와 같이 작성한다.

- name: cache
parameters: 30
- name: loop
- name: reload
- name: loadbalance

이미 구성이 되어 있는 경우, 제거 후 재구성 권장

EOF

```
kubectl apply -f coredns-helmchartconfig.yaml
```

```
kubectl -n kube-system rollout restart deployment coredns
```

```
kubectl delete helmchart rke2-coredns -n kube-system
```



HelmChart

혹은 위와 동일하게 `/var/lib/rancher/rke2/server/manifests/` 파일 넣어서 배포.

```
cp coredns-helmchartconfig.yaml /var/lib/rancher/rke2/server/manifests/  
systemctl restart rke2-server  
kubectl get pod -n kube-system -w
```



컨트롤러(10)

설치

설치시작(컨트롤러 1)

컨트롤러 구성을 위해서 부트스트랩에서 토큰 정보를 얻는다. 토큰 정보를 `bootstrap.yaml`파일에 붙여넣기 한다.

```
control1# mkdir -p /etc/rancher/rke2/
bootstrap# cat /var/lib/rancher/rke2/server/agent-token
K1029119e89b48970b5b5902b6c3d79349ae7ca4a21255169a09dfa0538b6818274::server
:a82a0812afb645d037a7da8a5c198b9d
bootstrap# scp /var/lib/rancher/rke2/server/tls/server-ca.crt
control1:/etc/pki/trust/anchors
control1# update-ca-certificates
bootstrap# scp /var/lib/rancher/rke2/server/tls/server-ca.crt
control1:/etc/pki/ca-trust/source/anchors
control1# update-ca-trust
```

계
속



설치시작(컨트롤러 1)

설치를 위한 `control1.yaml`파일을 아래와 같이 생성한다.

```
control1# cat <<EOF> ~/control1.yaml
---
server: "https://192.168.10.250:9345"
write-kubeconfig-mode: "0644"
tls-san:
  - rancher.example.com
  - dashboard.example.com
  - <RANCHER_NODES>
```



설치시작(컨트롤러 1)

앞에 내용 계속 이어서...

```
token: $(cat /etc/rancher/rke2/token)
node-name: "control1.example.com"
node-ip: "192.168.10.10"
node-external-ip: "192.10.10.10"
disable-kube-proxy: false
cni: "flannel"
cluster-domain: "rancher.example.com"
```

external-ip는 외부에서 접근하는 인터페이스



설치시작(컨트롤러 1)

앞에 내용 계속 이어서...

```
#node-taint:  
# - "node-role.kubernetes.io/control-plane=true:NoSchedule"  
cluster-cidr: "11.44.0.0/16"  
service-cidr: "12.88.0.0/16"  
cluster-dns: "12.88.0.10"  
cluster-name: "rke2-lab-cluster"  
flannel-iface: "ens3"
```

flannel-iface는 canal에서 사용 가능하나,
HelmChart에서 수정 권장



설치시작(컨트롤러 1)

```
hostnamectl set-hostname control1.example.com
```

```
export PATH=$PATH:/var/lib/rancher/rke2/bin/
```

```
export INSTALL_RKE2_VERSION="v1.30.0+rke2r1"
```

```
export INSTALL_RKE2_CHANNEL="stable"
```

```
export INSTALL_RKE2_TYPE="server"
```

명시하지 않으면 server, 최신 버전으로 설치

```
curl -sfL https://get.rke2.io > install_rke2.sh
```

```
chmod ug+x install_rke2.sh
```



설치시작(컨트롤러 1)

```
./install_rke2.sh  
mkdir -p /var/lib/rancher/rke2/agent/images/  
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz  
mkdir -p /etc/rancher/rke2  
cp ~/control1.yaml /etc/rancher/rke2/config.yaml  
sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config  
setenforce 0  
systemctl enable --now rke2-server.service
```



kubectl 명령어

쿠버네티스에서 제공하는 `kubectl` 명령어 및 컨테이너를 생성 및 관리하는 CRI-O 관리 명령어를 사용하기 위해
서 아래와 같이 설정한다. 참고로 Rancher는 containerd 기반으로 부스 스트랩 구성이 된다.

자동완성 기능이 필요한 경우, 아래와 같이 작업을 수행한다.

```
dnf install bash-completion -y
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
echo 'export #
CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml' >>
```

계
속

bashrc



디버깅(컨트롤러 1)

컨트롤러 노드가 한 대 이상인 경우, 노드 대수에 맞게 위의 작업을 반복적으로 하면 된다. 다만, `config.yaml`과 일에서 노드 및 아이피 주소 변경이 필요하다. 올바르게 구성이 되지 않으면, 아래와 같이 `systemd-journald`를 통해서 로그 추적이 가능하다.

```
journalctl -u RKE2-server -f
```

위의 작업이 완료가 되면, 추가적으로 컨트롤러 2/3번도 추가한다. 구성이 완료가 되면, `kubectl`명령어 사용을 위해서 "kubeconfig"파일을 생성한다.

```
cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
```

```
kubectl get node
```

```
kubectl get pod
```



설치확인(부트스트랩)

문제없이 구성이 되면, 다음과 같이 결과가 출력이 된다.

```
bootstrap# kubectl get nodes
```

bootstrap.example.com	Ready	control-plane,etcd,master	38m
control1.example.com	Ready	control-plane,etcd,master	5m
control2.example.com	Ready	control-plane,etcd,master	5m
control3.example.com	Ready	control-plane,etcd,master	5m



참고자료

https://docs.rke2.io/reference/server_config

https://docs.rke2.io/networking/networking_services

https://docs.rke2.io/networking/multus_sriov

<https://docs.rke2.io/architecture>

컴퓨트(40)

설치

설치시작(컴퓨터 1)

나머지 compute, bootstrap노드에서 토큰 정보를 확인 후, 기존 controller와 동일하게 수행한다.

```
bootstrap# cat /var/lib/rancher/rke2/server/agent-token  
K1080761c54cd6ef58545b03ce9773f900c0273266561ac84a6fbcecf5537241b30::server  
:c44f1fa30674a70e4627d22e65bc716d
```

토큰 및 호스트 이름, 그리고 서버 이름을 적절하게 변경한다. 처음 부트스트래핑 하기전에 CA키가 없기 때문에 올바르게 L4서버와 통신이 안될 수 있다. server:필드에 컨트롤러1번으로 수정 후 진행한다. 추후에 다시 L4로 수정한다.

```
config.yaml  
server: https://192.168.10.250:9345  
token: $(cat /etc/rancher/rke2/token)  
node-name: compute1.example.com
```



설치시작(컴퓨터 1)

컴퓨터 구성을 위해서 부트스트랩에서 토큰 정보를 얻는다. 토큰 정보를 `bootstrap.yaml` 파일에 붙여넣기 한다.

```
bootstrap# scp /var/lib/rancher/rke2/server/tls/server-ca.crt
compute1:/etc/pki/trust/anchors
compute1# update-ca-certificates
bootstrap# scp /var/lib/rancher/rke2/server/tls/server-ca.crt
compute1:/etc/pki/ca-trust/source/
compute1# update-ca-trust
```



설치시작(컴퓨터 1)

토큰 정보 확인 후, 설정파일에 다음과 같이 추가한다.

```
cat <<EOF> compute1.yaml
---
server: "https://192.168.10.250:9345"
write-kubeconfig-mode: "0644"
token: $(cat /etc/rancher/rke2/token)
node-name: "compute1.example.com"
node-ip: "192.168.10.40"
node-external-ip: "192.10.10.40"
EOF
```



설치시작(컴퓨터 1)

토큰 정보 확인 후, 설정파일에 다음과 같이 추가한다.

```
cluster-domain: "rancher.example.com"  
cluster-cidr: "11.44.0.0/16"  
service-cidr: "12.88.0.0/16"  
cluster-dns: "12.88.0.10"  
cluster-name: "rke2-lab-cluster"  
flannel-iface: "ens3"
```



설치시작(컴퓨터 1)

위와 같이 구성이 완료가 되면, 이전과 동일하게 작업을 수행한다.

```
curl -sfL https://get.rke2.io > install_rke2.sh
chmod ug+x install_rke2.sh
export INSTALL_RKE2_VERSION="v1.30.0+rke2r1"
export INSTALL_RKE2_CHANNEL="stable"
export INSTALL_RKE2_TYPE="agent"
```



설치(컴퓨터 1)

컨트롤러 서비스는 동작이 안되도록 마스킹 및 중지 처리한다.

```
sh install_rke2.sh
mkdir -p /var/lib/rancher/rke2/agent/images
mkdir -p /etc/rancher/rke2
curl http://10.0.0.250/rke2-images-core.linux-amd64.tar.gz -o
/var/lib/rancher/rke2/agent/images/rke2-images-core.linux-amd64.tar.gz
cp ~/compute1.yaml /etc/rancher/rke2/config.yaml
hostnamectl set-hostname compute1.example.com
sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config
systemctl disable --now firewalld
systemctl enable --now rke2-agent.service
```



최종 설치 확인(컨트롤러 1)

문제없이 구성이 되면, 다음과 같이 결과가 출력이 된다.

```
# kubectl get nodes
```

bootstrap.example.com	Ready	control-plane,etcd,master	38m
control1.example.com	Ready	control-plane,etcd,master	5m
control2.example.com	Ready	control-plane,etcd,master	5m
control3.example.com	Ready	control-plane,etcd,master	5m
compute1.example.com	Ready	control-plane,etcd,master	5m



초기화

클러스터 초기화가 필요한 경우 다음과 같이 실행한다. rke2명령어는 /opt/rke2/bin에 존재한다.

```
systemctl stop rke2-server  
rke2 server --cluster-reset
```



BOOTSTRAP제거 및 CLIENT 구성(옵션)

기본적인 구성이 완료가 되면, BOOTSTRAP노드는 더 이상 필요하지 않는다. BOOTSTRAP제거를 하기 위해서 다음과 같은 조건이 구성이 되어야 한다.

1. 최소 3개의 컨트롤러 구성
2. 최소 1개의 컴퓨트 노드 구성

이 조건이 성립이 되면, 클라이언트 노드에 kubectl명령어를 설치한다. 보안상 이유로 절대로 control 1/2/3에는 설치하면 안된다. 아래와 같이 client 인스턴스에서 실행한다.

```
curl -L "https://dl.k8s.io/release/v1.30.0/bin/linux/amd64/kubectl" -o  
~/bin  
chmod u+rx ~/bin/kubectl
```



BOOTSTRAP제거 및 CLIENT 구성(옵션)

설치가 완료가 되면 bootstrap, control에서 rke2.yaml파일을 복사한다.

```
client# mkdir -p /etc/rancher/rke2
client# scp root@bootstrap:/etc/rancher/rke2/rke2.yaml
/etc/rancher/rke2/rke2.yaml
client# export KUBECONFIG=/etc/rancher/rke2/rke2.yaml
client# export PATH=$PATH:/var/lib/rancher/rke2/bin
client# kubectl get nodes
client# dnf install bash-completion
client# kubectl completion bash > /etc/profile.d/kubectl.sh
client# source /etc/profile
client# complete -rp
```



BOOTSTRAP제거 및 CLIENT 구성(옵션)

bootstrap 노드를 아래 명령어로 제거한다. rancher cli는 아래 슬라이드에서 설치를 진행한다. client구성이 불편하면 bootstrap노드에서 계속 진행하여도 상관 없다.

```
client# kubectl delete node/bootstrap.example.com  
client# kubectl get nodes
```



기능 확장

GNOME DESKTOP CLIENT

Rancher dashboard/CLI TLS

Longhorn Storage

NFS Storage

SelfSign(비권장)

수동 생성

TLS FOR LAB

TLS키가 필요한 경우 뒤에 나오는 명령어로 생성 및 적용.

Makefile형식으로 랩에서 바로 사용이 가능하도록 구성. 이 방법이 복잡하면, 직접 명령어 생성하여도 무관함.



TLS FOR LAB(SelfSigned)

TLS키를 자주 업데이트 및 생성해야 되기 때문에 Makefile로 아래와 같이 구성한다.

```
certs/  
├─ Makefile  
├─ ca.conf          # 루트 CA용 설정  
├─ san.conf         # leaf cert용 SAN 설정  
└─ output/          # 결과물 저장
```



TLS FOR LAB(ca.conf)

빌드 파일은 다음과 같이 구성 및 생성한다.

```
mkdir certs && cd certs
cat <<EOF> ca.conf
[ req ]
default_bits      = 4096
prompt            = no
default_md         = sha256
distinguished_name = dn
x509_extensions   = v3_ca
```



TLS FOR LAB(ca.conf)

위의 내용 계속 이어서...

```
[ dn ]  
C   = KR  
ST  = Seoul  
L   = Seoul  
O   = Dustbox Inc.  
CN  = Dustbox Root CA
```



TLS FOR LAB(ca.conf)

앞에 내용 이어서 계속...

```
[ v3_ca ]
subjectAltName = @alt_names
basicConstraints = critical, CA:TRUE
keyUsage = critical, digitalSignature, cRLSign, keyCertSign

[ alt_names ]
DNS.1 = ca.example.com
DNS.2 = *.example.com
EOF
```



TLS FOR LAB(san.conf)

빌드 파일은 다음과 같이 구성 및 생성한다.

```
cat <<EOF> san.conf
[ req ]
default_bits      = 2048
prompt            = no
default_md         = sha256
distinguished_name = dn
req_extensions     = req_ext
```



TLS FOR LAB(san.conf)

앞에 내용 이어서 계속...

```
[ dn ]  
C   = KR  
ST  = Seoul  
L   = Seoul  
O   = Dustbox Inc.  
OU  = Platform  
CN  = rancher.example.com
```



TLS FOR LAB(san.conf)

앞에 내용 이어서 계속...

```
[ req_ext ]
subjectAltName = @alt_names
[ alt_names ]
DNS.1 = rancher.example.com
DNS.2 = dashboard.example.com
DNS.3 = node1.example.com
DNS.4 = node2.example.com
DNS.5 = node3.example.com
DNS.9 = *.example.com
DNS.10 = cli.example.com
EOF
```



TLS FOR LAB(SelfSigned)

빌드 파일은 다음과 같이 구성 및 생성한다. [링크](#)

```
cat <<EOF> Makefile
```

```
CA_KEY=output/ca.key
```

```
CA_CERT=output/ca.crt
```

```
CA_CONF=ca.conf
```

```
LEAF_KEY=output/rancher.key
```

```
LEAF_CSR=output/rancher.csr
```

```
LEAF_CERT=output/rancher.crt
```

```
LEAF_CONF=san.conf
```

```
.PHONY: all clean ca leaf
```

```
all: ca leaf
```



TLS FOR LAB(SelfSigned)

빌드 파일은 다음과 같이 구성 및 생성한다.

루트 CA 생성

ca:

```
mkdir -p output
openssl genrsa -out $(CA_KEY) 4096
openssl req -x509 -new -nodes -key $(CA_KEY) -sha256 -days 3650 \
-out $(CA_CERT) -config $(CA_CONF) -extensions v3_ca
@echo "✅ [CA 생성 완료] $(CA_CERT)"
```



TLS FOR LAB(SelfSigned)

빌드 파일은 다음과 같이 구성 및 생성한다.

리프(leaf) 인증서 생성

leaf:

```
    mkdir -p output
    openssl req -new -nodes -out $(LEAF_CSR) -newkey rsa:2048 -keyout
$(LEAF_KEY) -config $(LEAF_CONF)
    openssl x509 -req -in $(LEAF_CSR) -CA $(CA_CERT) -CAkey $(CA_KEY) -
CAcreateserial \
    -out $(LEAF_CERT) -days 825 -sha256 -extfile $(LEAF_CONF) -
extensions req_ext
    @echo "✅ [Leaf 인증서 생성 완료] $(LEAF_CERT)"
```



TLS FOR LAB(SelfSigned)

빌드 파일은 다음과 같이 구성 및 생성한다.

```
## 정리
clean:
    rm -rf output
    @echo " 🧹 모든 생성 파일 삭제됨."
EOF
```



TLS FOR LAB(SelfSigned)

실행은 다음과 같이 한다. 생성이 문제없이 되면, 적절한 위치에 인증서를 저장한다. CA 경우에는 각 노드에 ca-update를 수행해야 한다.

```
dnf install -y make
zypper install -y make
make
ls -l output/
```



Cert-Manager(권장)

TLS 자동생성

cert-manager 설치

클러스터 인증서는 cert-manager로 구성 및 관리가 가능하다. 아래와 같이 helm에 등록 및 구성한다.

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true
mkdir ~/tls
```



인증서 발급

Rancher 클러스터에서 사용하는 인증서를 발급 및 배포한다. 인증서에 사용할 root-ca, SelfSign 선언한다.

```
cat <<EOF>> tls/rancher-root-ca.yaml
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned-root
  namespace: cattle-system
spec:
  selfSigned: {}
```



인증서 발급

위의 내용 계속 이어서...

```
apiVersion: cert-manager.io/v1
```

```
kind: Certificate
```

```
metadata:
```

```
  name: rancher-root-ca
```

```
  namespace: cattle-system
```



인증서 발급

위의 내용 계속 이어서...

```
spec:  
  isCA: true  
  commonName: "rancher-root-ca"  
  secretName: rancher-root-ca  
  duration: 87600h  
  issuerRef:  
    name: selfsigned-root  
    kind: Issuer
```



대시보드 인증서

대시보드에서 사용할 인증서를 생성한다.

```
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: rancher-ca-issuer
spec:
  ca:
    secretName: rancher-root-ca
```



대시보드 인증서

앞에 내용 계속 이어서...

```
apiVersion: cert-manager.io/v1
```

```
kind: Certificate
```

```
metadata:
```

```
  name: rancher-cert
```

```
  namespace: cattle-system
```

```
spec:
```

```
  secretName: tls-rancher
```

```
  commonName: rancher.example.com
```



대시보드 인증서

앞에 내용 계속 이어서...

```
dnsNames:
```

- rancher.example.com
- dashboard.example.com
- demo-rancher.dustbox.kr

```
issuerRef:
```

```
  name: rancher-ca-issuer  
  kind: ClusterIssuerEOF
```

```
EOF
```

```
kubectl create namespace cattle-system
```

```
kubectl apply -f tls/rancher-root-ca.yaml
```



Rancher 관리자(cert-manager)

cert-manager를 통해서 구성한 경우, 아래와 같이 설정을 변경한다.

```
kubectl get secret -n cattle-system  
rancher-root-ca
```

```
kubectl describe secret -n cattle-system rancher-root-ca-<SHA>  
tls.pem or ca.crt
```

```
kubectl get secret rancher-root-ca -n cattle-system \  
-o jsonpath="{.data['ca\.crt']}" | base64 -d > /root/tls/ca.crt  
kubectl create secret generic tls-ca --from-  
file=cacerts.pem=/root/tls/ca.crt -n cattle-system  
kubectl create secret generic tls-ca-additional \  
--from-file=ca-additional.pem=/root/tls/ca.crt -n cattle-system
```



인증서 도메인 확인

아래와 같이 도메인 확인이 가능하다.

```
kubectl -n cattle-system get secret tls-rancher -o  
jsonpath="{.data['tls\.crt']}" | base64 -d | openssl x509 -text -noout |  
grep -A1 "Subject Alternative Name"
```

X509v3 Subject Alternative Name:

DNS:rancher.example.com, DNS:dashboard.example.com,
DNS:demo-rancher.dustbox.kr



Rancher 관리자(cert-manager)

다음과 같은 옵션으로 인하여 아래와 같은 secret를 요구한다.

Secret 이름	설명	필수 조건
tls-ca	주 CA 인증서 (Rancher 인증용)	privateCA=true
tls-ca-additional	추가적으로 신뢰할 CA들 (옵션)	additionalTrustedCAs=true



Rancher 대시보드

대시보드 설치

클라이언트 컴퓨터 구성

가상머신을 클라이언트 용도로 생성한다. 이 부분은 강사 안내를 받아서 같이 진행한다.

필요한 패키지는 다음과 같다.

```
dnf install gnome-desktop3 gnome-session firefox gnome-terminal gdm -y
systemctl set-default graphical
systemctl isolate graphical
systemctl get-default
```



Rancher 저장소 등록

앞에서 Rancher를 사용하기 위해서 쿠버네티스를 구성하였다. 클러스터 내부에 Rancher를 사용할 수 있도록 helm를 설치 및 구성한다. 이 작업은 **bootstrap/client node**에서 진행한다. 만약, bootstrap를 제거한 경우 client에서 진행한다.

```
helm repo add rancher-stable https://releases.rancher.com/server-  
charts/stable  
helm repo update
```



Rancher 관리자(SelfCert)

앞에 내용 이어서...

```
helm install rancher rancher-stable/rancher \
--namespace cattle-system --create-namespace \
--set hostname=rancher.example.com \
--set bootstrapPassword=rancher \
--set ingress.tls.source=none \
--set privateCA=true \
--set caCertsSecret=cacerts \
--set global.cattle.psp.enabled=false
```

Ingress TLS키가 없으면 none으로 설정

```
kubectl rollout restart deployment rancher -n cattle-system
```



Rancher 관리자(cert-manager)

cert-manager를 통해서 구성한 경우, 아래와 같이 설정을 변경한다.

```
helm install rancher rancher-stable/rancher \
--namespace cattle-system --create-namespace \
--set hostname=rancher.example.com \
--set bootstrapPassword=rancher \
--set ingress.tls.source=secret \
--set privateCA=true \
--set additionalTrustedCAs=true \
--set ingress.tls.secretName=tls-rancher

kubectl rollout restart deployment rancher -n cattle-system
```



Rancher 관리자(cert-manager+airgapped)

cert-manager를 통해서 구성한 경우, 아래와 같이 설정을 변경한다.

```
helm install rancher rancher-stable/rancher \
--namespace cattle-system --create-namespace \
--set hostname=rancher.example.com \
--set bootstrapPassword=rancher \
--set ingress.tls.source=secret \
--set ingress.tls.secretName=tls-rancher-ingress \
--set privateCA=true \
--set rancherImage=10.10.10.250:5000/rancher/rancher \
--set systemDefaultRegistry=10.10.10.250:5000

kubectl rollout restart deployment rancher -n cattle-system
```



인증서 검사(Rancher)

cacerts

CA Certificates needed to verify the server's certificate.

Hide cacerts

-----BEGIN CERTIFICATE-----

```
MIIDATCCAemgAwIBAgIRAjVknHhMfmM6DCK6sX3LP7swDQYJKoZIhvcNAQELBQAw
GjEYMBYGA1UEAxMPcmFuY2hlcilcyb290LWNhMB4XDTI1MDcyMDA0MTYzNFoXDTM1
MDcxODAwMTYzNFowGjEYMBYGA1UEAxMPcmFuY2hlcilcyb290LWNhMIIBIjANBgkq
hkiG9w0BAQEFAAOCQAQ8AMIIBCgKCAQEA6dNY3B/txLleqZrGWCJhh4men0EoiFZd
Ibk0UB/k/qm+dLBJuol8oNyMQxBSYySDH24kKmnjqCwZ10zd0kpVy4pxJGf4+dyN
y+H3LXibSQ+X7vfc6Kac6Trsq5s5A4QyU5NKcIXNTdR0i7w5HiIW6FidlVUUSaxB
GXCuFpBonQm/lVjGEzpLuLBvm/6AMC0mnfZB0ug0TpiD8zMnjQcV87amrldDrasM
g/Q2owbrCaGxV/6ALEDL17iX0hqpKbcvQtf3igEy/5mY3yQGwRqlE8IwHhSRCYfD
DWZs/vMtpA5R2y7u0z0c1diHQKmV0qg1NZ48SLSZNB0AliLSuJfmzQIDAQABo0Iw
QDA0BgNVHQ8BAf8EBAMCAqQwDwYDVR0TAQH/BAUwAwEB/zAdBgNVHQ4EFgQUWRpi
3SRLQl8J0qenD+/zFWQCBCkwdQYJKoZIhvcNAQELBQADggEBAADK9Eq4m2kXp7xMs
f3JrcI91V4n0w0B5K0Hu69A7froLTionXuXQ9dWKZB5gISS6cv1RLyGCrLrz09rx
NdWZfev+BUerHKV2jF3BZ6S5tZFauHyEys51gcJRz00W2Ntrt5LsTHDSTsGkMkwq
EmCN4Pc8Dn5H7BFH8AcAAlbP++/uQ3bz/A86Ww/KFqM3TtYAkBWP/ONqYsV4Itew
60RnGIJZrEUhcSocI9dYsgTLGIB6spCEkFgPUIVknDdj6nZUKVFf17AQiZtwQu24
wimdWL7a0p1YtasYtAP7LXaSQIWjqzeYJllcX5gQTti9QjU+4m/33k9AF2c0zyuH
ePcg63M=
```

-----END CERTIFICATE-----



인증서 검사(RKE2)

```
curl --cacert <ca.crt 경로> https://rancher.example.com
```



기능 확장(선택)

MetalLB

rke2-ingress

Longhorn Storage

NFS Storage

METALLB FOR LAB

HAProxy 기반으로 접근하는 경우가 아니면, MetalLB로 VIP 별도로 구성 후, 도메인 접근이 가능하도록 Ingress를 구성한다.

하지만, 굳이 이러한 기능이 필요가 없으면 이 기능은 구성하지 않아도 상관이 없다.



METALLB FOR LAB 설치 준비

MetalLB를 사용하기 위해서, 아래와 같이 클러스터 설정을 수정해야 한다. MetalLB는 기본적으로 kube-proxy를 사용한다.

```
cat <<EOF> config.yaml
```

```
kube-proxy-arg:
```

- proxy-mode=ipvs
- ipvs-strict-arp=true

```
disable:
```

- servicelb
- traefik

```
EOF
```

```
cp config.yaml /etc/rancher/rke2/config.yaml
```

```
systemctl restart rke2-server
```

```
systemctl restart rke2-agent
```

VLAN 기반 네트워크가 아니면 L2 구성이 잘 안됨.

계속



METALLB FOR LAB

strictARP: 옵션을 컨트롤 노드에만 적용한다. **config.yaml**에 적용한 경우, 이 방식은 사용하지 않아도 된다.

```
mkdir -p /var/lib/rancher/rke2/server/manifests
cat <<EOF> kube-proxy-config.yaml
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-kube-proxy
  namespace: kube-system
spec:
  valuesContent: |
    proxyMode: "ipvs"
    ipvs:
      strictARP: true
```



METALLB FOR LAB

커널 모듈이 영구적으로 구성이 되어야 한다. 모든 노드에 적용한다.

```
echo -e "ip_vs\nip_vs_rr\nip_vs_wrr\nip_vs_sh\nnf_conntrack" | sudo tee  
/etc/modules-load.d/ipvs.conf
```

```
for i in ip_vs ip_vs_rr ip_vs_wrr ip_vs_sh nf_conntrack ; do  
modprobe $i  
done
```

```
lsmod | grep ip_vs  
systemctl daemon-reload
```



METALLB FOR LAB

Helm에 저장소를 등록 및 설치한다.

```
helm repo add metallb https://metallb.github.io/metallb
helm install --create-namespace --namespace metallb-system metallb
metallb/metallb --version 0.14.9
```



METALLB FOR LAB

ip-pool를 아래와 같이 등록한다.

```
cat <<EOF> vi ippool.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  avoidBuggyIPs: true
  autoAssign: true
  addresses:
    - 192.10.10.251-192.10.10.254
```

EOF

```
kubectl apply -f ippool.yaml && kubectl get -f ippool.yaml
```



METALLB FOR LAB

l2 advertisement를 아래와 같이 등록한다.

```
cat <<EOF> l2-advertisement.yaml
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
  interfaces:
  - ens4
EOF
kubectl apply -f l2-advertisement.yaml
```



METALLB 설치 확인 로드 밸런서 1

올바르게 동작하는지 확인하기 위해서 80/TCP를 Loadbalancer로 생성한다. MetalLB가 잘 동작하면 올바르게 아이피가 출력이 된다.

```
kubectl expose deployment rancher \  
  --name=rancher-lb-http \  
  --namespace=cattle-system \  
  --type=LoadBalancer \  
  --port=80 \  
  --target-port=80 \  
  --protocol=TCP  
kubectl get svc -n cattle-system
```



METALLB 설치 확인 로드 밸런서 2

올바르게 동작하는지 확인하기 위해서 443/TCP를 Loadbalancer로 생성한다. MetalLB가 잘 동작하면 올바르게 아이피가 출력이 된다.

```
kubectl expose deployment rancher \  
  --name=rancher-lb-https \  
  --namespace=cattle-system \  
  --type=LoadBalancer \  
  --port=443 \  
  --target-port=443 \  
  --protocol=TCP  
kubectl get svc -n cattle-system
```



METALLB 설치 확인 노드 포트 1

올바르게 동작하는지 확인하기 위해서 80/TCP를 NodePort로 생성한다.

```
kubectl expose deployment rancher \  
  --name=rancher-nodeport \  
  --namespace=cattle-system \  
  --type=NodePort \  
  --port=80 \  
  --target-port=80 \  
  --protocol=TCP
```



METALLB 설치 확인 노드 포트 2

올바르게 동작하는지 확인하기 위해서 NodePort 형식으로 생성한다.

```
kubectl expose deployment rancher \  
  --name=rancher-nodeport-https \  
  --namespace=cattle-system \  
  --type=NodePort \  
  --port=443 \  
  --target-port=443 \  
  --protocol=TCP  
kubectl get svc -n cattle-system
```



METALLB 서비스 재구성

문제가 없으면 앞에서 생성한 SVC를 다시 제거한다.

```
kubectl -n cattle-system delete svc/rancher-lb-https  
kubectl -n cattle-system delete svc/rancher-lb-http
```



오픈스택 IPVS

오픈스택에서 LoadBalancer 아이피를 미리 할당해야 한다. 호라이즌에서 아이피를 생성 및 구성한다. 먼저, 다음 아이피를 생성한다.

1. 192.10.10.251
2. 192.10.10.252
3. 192.10.10.253
4. 192.10.10.254



INGRESS+EXTERNAL IP

`rke2-ingress`기반으로 `LoadBalancer`를 사용하기 위해서 부트스트랩 노드에서 아래와 같이 파일을 작성한다.

만약, 높은 호환성을 원하는 경우, `rke2-ingress`를 `disable: {}`를 통해서 삭제 후 보통 `ingress-nginx`를 설치한다. `rke2-ingress`는 기본적으로 `NodePort`기반으로 동작한다.

이러한 이유로 `ingress`에서 다음과 같이 화면에 표시한다. `rke2-ingress`는 아래와 같이 표시가 되기 때문에, 올바르게 동작이 되지 않는다고 판단을 할 수 있다. `rke2-ingress`에서는 지극히 정상이다.

```
kubectl get ingress -n cattle-system
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
rancher	nginx	dashboard.example.com	192.10.10.10	80, 443	12h
rancher-cli	nginx	cli.example.com	192.10.10.10	80, 443	11h



INGRESS+EXTERNAL IP

rke2-ingress 또한, ingress-controller에 LoadBalancer 할당 및 생성하지 않는다. 아래와 같이 bootstrap 혹은 control1/2/3에서 구성 및 생성 후 서비스를 재 시작한다.

```
vi /var/lib/rancher/rke2/server/manifests/rke2-ingress-nginx-config.yaml
---
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-ingress-nginx
  namespace: kube-system
```



INGRESS+EXTERNAL IP

앞에 내용 계속 이어서...

```
spec:
  valuesContent: |-
    controller:
      config:
        use-forwarded-headers: "true"
        enable-real-ip: "true"
```



INGRESS+EXTERNAL IP

앞에 내용 계속 이어서...

```
service:
  enabled: true
  type: LoadBalancer
  external:
    enabled: true
  externalTrafficPolicy: Local
  annotations:
    metallb.universe.tf/loadBalancerIPs: 192.10.10.251
```

```
systemctl restart rke2-server
kubectl get svc -n kube-system
kubectl get ingress -n cattle-system
```



참고

https://docs.rke2.io/networking/networking_services

<https://github.com/kubernetes/ingress-nginx/tree/main/charts/ingress-nginx#configuration>

Longhorn Storage

POSIX OBJECT STORAGE

설명

Rancher에서 기본으로 제공하는 Object Storage이다. CephFS처럼 블록 형태로 제공하지는 않지만, 쿠버네티스 클러스터에서 생성된 POD에 PVC 기반으로 저장소를 제공한다.

사용 조건은 다음과 같다.

1. Node에 디스크가 구성이 되어 있어야 한다
2. 기본 replication 개수는 3개로 구성이 되어 있다
3. 디스크 개수가 많을수록 복제 성능은 더 개선이 된다



설치(CLI)

명령어로 아래와 같이 설치가 가능하다.

```
dnf install -y iscsi-initiator-utils
zypper install -y open-iscsi
helm repo add longhorn https://charts.longhorn.io
helm repo update
helm install longhorn longhorn/longhorn --namespace longhorn-system \
--create-namespace
kubectl -n longhorn-system get pod
```



랜처 대시보드 설치 1

Cluster Tools

All charts have at least one version that is installable on clusters with Linux and Windows nodes unless otherwise indicated.



Alerting Drivers

v106.0.0

The manager for third-party webhook receivers used in Prometheus Alertmanager

Install



CIS Benchmark

v106.0.0+up8.0.0

The cis-operator enables running CIS benchmark security scans on a kubernetes cluster

Install



Elemental

v106.0.1+up1.6.9

Elemental provides Cloud Native OS Management for Cluster Nodes.

Install



Istio

v106.3.1+up1.25.0

A basic Istio setup that installs with the istioctl. Refer to <https://istio.io/latest/> for details.

Install



Logging

v106.0.2+up4.10.0-rancher.6

Logging operator for Kubernetes based on Fluentd and Fluentbit.

Deploys on Windows

Install



Longhorn

v106.2.1+up1.8.2

Longhorn is a distributed block storage system for Kubernetes.

Install



Monitoring

v106.1.2+up69.8.2-rancher.7

kube-prometheus-stack collects Kubernetes manifests, Grafana dashboards, and Prometheus rules combined with documentation and scripts to provide easy to operate end-to-end Kubernetes...

Deploys on Windows

Install



NeuVector

v106.0.1+up2.8.6

Helm feature chart for NeuVector container security platform.

Install



NeuVector Monitor

v106.0.1+up2.8.6

Helm feature chart (optional) add-on to NeuVector for monitoring with Prometheus/Grafana.

Install



랜처 대시보드 설치 2

All charts have at least one version that is installable on clusters with Linux and Windows nodes unless otherwise indicated.

**Longhorn**
106.2.1+up1.8.2

Install: Step 1
Set App metadata

Metadata

Values

This process will help create the chart. Start by setting some basic information used by Rancher to manage the App.

Version
106.2.1+up1.8.2

Install into Project
(None)

☐ Customize Helm options before install

☐ Container Registry for Rancher System Container Images



랜처 대시보드 설치 3

 **Longhorn**
106.2.1+up1.8.2

Install: Step 2
Change how the App works

 Metadata

 Values

Configure Values used by Helm that help define the App.

Edit Options Edit YAML Compare Changes View Chart Info

Longhorn Images

Private Registry Settings

Longhorn Default Settings

Longhorn Storage Class Settings

Services and Load Balancing

Longhorn Default Resources

Other Settings

Longhorn Images

☒ Use Default Images

Use default Longhorn images



랜처 대시보드 설치 4

[Edit Options](#) [Edit YAML](#) [Compare Changes](#) [View Chart Info](#)

[Longhorn Images](#)
[Private Registry Settings](#)
[Longhorn Default Settings](#)
[Longhorn Storage Class Settings](#)
[Services and Load Balancing](#)
[Longhorn Default Resources](#)
[Other Settings](#)

Longhorn Storage Class Settings

☒ Default Storage Class

Storage Class Retain Policy *

Delete

▼

Storage Class Filesystem Type

ext4

▼

Default Storage Class Disable Revision Counter *

true

▼

Default Storage Class Replica Count

3

▼

Default Storage Class Data Locality

disabled

▼

Setting that allows you to specify the default Longhorn StorageClass.

Reclaim policy that provides instructions for handling of a volume after its claim is released. (Options: "Retain", "Delete")

Filesystem type of the default Longhorn StorageClass

Setting that disables the revision counter and thereby prevents Longhorn from tracking all write operations to a volume. When salvaging a volume, Longhorn uses properties of the volume-head-xxx.img file (the last file size and the last time the file was modified) to select the replica to be used for volume recovery. (Options: "true", "false")

Replica count of the default Longhorn StorageClass.

Data locality of the default Longhorn StorageClass. (Options: "disabled", "best-effort")



NFS Storage

GENERAL NFS STORAGE

NFS 서버 구성

NFS서버는 HAPROXY서버에 같이 설치한다. 각 배포판마다 조금씩 다르니 적절하게 명령어 선택해서 설치한다.

```
dnf install -y nfs-utils
mkdir -p /export/data
chown nobody:nobody /export/data
chmod 755 /export/data
echo "/export/data 10.0.0.0/8(rw,sync,no_root_squash)" | sudo tee -a
/etc/exports
systemctl enable --now nfs-server
exportfs -rav
showmount -e 10.10.10.250
zypper install -y nfs-kernel-server
systemctl enable --now nfsserver
exportfs -rav
```



CSI-NFS-DRIVER

CSI NFS DRIVER를 CHART를 통해서 설치 및 구성한다.

```
helm repo add csi-driver-nfs https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/charts
```

```
helm install csi-driver-nfs csi-driver-nfs/csi-driver-nfs --namespace kube-system --version 4.11.0
```



STORAGE CLASS

스토리지 클래스는 다음처럼 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-csi
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.10.10.250
  share: /export/data
```



STORAGE CLASS

위의 내용 계속 이어서...

```
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - hard
  - nfsvers=4.1
EOF
kubectl apply -f storageclass-configure.yaml
```



PVC 생성 테스트 1

스토리지 클래스는 다음처럼 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  storageClassName:
  accessModes:
    - ReadWriteMany
```



PVC 생성 테스트 2

위의 내용 계속 이어서...

```
resources:  
  requests:  
    storage: 1Gi
```

EOF

```
kubectl apply -f pvc.yaml
```



참고자료

VANILLA INGRESS-NGINX

VIP

VANILLA INGRESS-NGINX

사용자가 `hostNetwork(HostPort)` 형식이 불편하다면, `Vanilla Ingress-nginx` 설치도 가능하다. 아래와 같이 `config.yaml` 파일에서 수정 후 재시작 한다.

그리고, HelmChart를 통해서 서비스를 재설치 한다.

```
vi config.yaml
```

```
disable:
```

```
  - rke2-ingress-nginx
```

```
systemctl restart rke2-server
```



VANILLA INGRESS-NGINX

저장소 등록 후 바닐라 버전 ingress-nginx 재설치.

```
helm repo add nginx https://helm.nginx.com/stable
```

```
helm install nginx-ingress nginx/nginx-ingress \
--namespace ingress-nginx --create-namespace \
--set controller.ingressClassResource.name=nginx \
--set controller.ingressClass=nginx \
--set controller.service.type=LoadBalancer \
--set controller.service.loadBalancerIP=192.10.10.251 \
--set controller.hostNetwork=false
```



오픈스택 IPVS

네트워크에서 다음과 같이 아이피를 생성한다.

☐	infra-net	infra-net 10.0.0.0/8	Yes	No	Active	True	nova
☐	rancher-internal	rancher-internal-subnet 192.168.0.0/16	No	No	Active	True	nova
☐	external-net	external-net 121.254.163.0/24	No	Yes	Active	True	nova

[Overview](#) [Subnets](#) [Ports](#)

Ports

Filter

[+ Create Port](#)



오픈스택 IPVS

251~254번까지 아래처럼 등록한다.

InfoSecurity Groups

Name

192.168.10.251

☐ Enable Admin State ?

Device ID ?

Device Owner ?

Specify IP address or subnet ?

Fixed IP Address

Fixed IP Address * ?

192.168.10.251

MAC Address ?

☒ Port Security ?

VNIC Type ?

Normal

Create Port

InfoSecurity Groups

Add or remove security groups to the port from the list of available security groups. The "default" security group is associated by default and you can remove "default" security group from the port.

All Security Groups

Filter

Q

rancher-lab

+

default

+

Port Security Groups

Filter

Q

rancher-sg

-

Cancel

Create



오픈스택 IPVS

이제 10~40번까지 IP Address Pair를 구성한다.

☐ rancher-lab-node3_internal_port-hmbojlevvnb

• 192.168.10.10

+ Add Allowed Address Pair

Delete

Add Allowed Address Pair

IP Address or CIDR * ⓘ
192.168.10.251

MAC Address ⓘ

Description:
Add an allowed address pair for this port. This will allow multiple MAC/IP address (range) pairs to pass through this port.

Cancel

Submit

Overview			Allowed Address Pairs				
					+ Add Allowed Address Pair		Delete
Displaying 2 Items							
			☐ IP Address or CIDR	MAC Address	Actions		
			☐ 192.168.10.251	fa:16:3e:47:b9:6b	Delete		
			☐ 192.168.10.252	fa:16:3e:47:b9:6b	Delete		

오픈소스 Rancher

1/2024

239

폐쇄망 고려

disconnected/air gapped/offline

RKE2

Rancher는 기본적으로 RKE2 core image, Rancher image가 필요하다. 이미지를 구성 및 사용하기 위해서 아래 사이트 내용을 참고한다.

<https://docs.rke2.io/install/airgap>

이미지는 여러가지 형태로 제공하는데, 모든 이미지를 가져와서 사용하려면 `rke2-images.linux-amd64.tar.zst`를 사용한다.

압축 이미지는 CNI형식별로 분리가 되어 있지만, 가급적이면 모든 이미지를 가지고 있는 압축 파일 사용을 권장한다. 또한, 한 개 이상의 컨테이너 이미지를 가지고 있는 bundle image file이기 때문에 docker-registry와 같은 서버에 직접 업로드가 불가능하다.



HelmChart

Rancher에서 제공하는 HelmChart의 이미지도 위의 이미지에 같이 포함이 되어있다. 만약, 사용자가 추가로 Chart를 내부망에 구성을 원하는 경우 Helm Chart Museum를 통해서 구성이 가능하다.

필요 없는 경우 HelmChart Museum를 굳이 설치할 필요는 없다.

역시 registry서버도 굳이 Harbor를 설치가 필요 없다. 가볍게 사용하는 경우, docker-registry 기반으로 운영하여도 문제가 없다. 다만, 웹 관리자가 없기 때문에 관리에 불편한 부분이 많다.



HelmChart 폐쇄망

폐쇄망에서 Chart를 배포하기 위해서 다음과 같은 단계로 진행한다.

```
# cert-manager chart 다운로드 (예: v1.14.4)
```

```
helm repo add jetstack https://charts.jetstack.io
```

```
helm pull jetstack/cert-manager --version v1.14.4 --untar
```

```
helm pull jetstack/cert-manager --version v1.14.4
```



Chart Museum 구성

Chart Museum를 구성하여 내부에서 Chart Repository를 구성한다.

```
podman run -d \  
  --name chartmuseum \  
  -p 8080:8080 \  
  -v /opt/chartmuseum/charts:/charts \  
  -e DEBUG=true \  
  -e STORAGE=local \  
  -e STORAGE_LOCAL_ROOTDIR=/charts \  
  ghcr.io/helm/chartmuseum:latest
```



Chart Museum에 Chart 업로드

업로드는 다음과 같이 진행 한다.

```
curl --data-binary "@cert-manager-1.14.4.tgz" http://<chartmuseum-  
ip>:8080/api/charts
```



Chart Museum에 Chart 업로드

폐쇄망에서 아래와 같이 Chart Museum 저장소 등록 및 설치가 가능하다.

```
helm repo add local-charts http://<chartmuseum-ip>:8080  
helm repo update
```

```
helm install cert-manager local-charts/cert-manager \\  
  --namespace cert-manager --create-namespace \\  
  --set installCRDs=true
```



Chart Museum에 TLS 적용 (Cert-manager 설치 전)

TLS 적용을 원하는 경우 아래와 같이 진행한다.

```
podman run -d \  
  --name chartmuseum \  
  -p 8443:8080 \  
  -v /opt/chartmuseum/charts:/charts \  
  -v /opt/chartmuseum/certs:/certs \  
  -e DEBUG=true \  
  -e STORAGE=local \  
  -e STORAGE_LOCAL_ROOTDIR=/charts \  
  -e TLS_CERT=/certs/tls.crt \  
  -e TLS_KEY=/certs/tls.key \  
  ghcr.io/helm/chartmuseum:latest
```



폐쇄망 HelmChart 정리

마지막으로 정리하면 다음과 같은 단계로 진행한다.

단계	설명
1단계	외부에서 cert-manager Chart 다운로드 (.tgz)
2단계	폐쇄망에 ChartMuseum 설치 (podman, /charts 볼륨)
3단계	.tgz Chart 파일을 ChartMuseum에 업로드 (curl)
4단계	내부 Helm 클라이언트로 ChartMuseum repo 등록 및 설치
5단계	(선택) ChartMuseum TLS 설정



오프라인 준비, RKE2 INSTALL

RKE2에서 사용하는 이미지를 미리 복사한다. 이 랩에서는 미리 RKE2 이미지를 앞서 미리 복사 하였다.

```
mkdir /root/rke2-artifacts && cd /root/rke2-artifacts/
```

```
curl -OLs
```

```
https://github.com/rancher/rke2/releases/download/v1.33.1%2Brke2r1/rke2-images.linux-amd64.tar.zst
```

```
curl -OLs
```

```
https://github.com/rancher/rke2/releases/download/v1.33.1%2Brke2r1/rke2.linux-amd64.tar.gz
```

```
curl -OLs
```

```
https://github.com/rancher/rke2/releases/download/v1.33.1%2Brke2r1/sha256sum-amd64.txt
```



오프라인 준비, RKE2 INSTALL

스크립트 파일도 미리 오프라인에서 사용이 가능하도록 준비한다.

```
curl -sfL https://get.rke2.io --output install.sh
```



오프라인 준비, RKE2 RPM(레드햇 계열)

RPM 미러링도 아래와 같이 준비한다.

```
dnf install dnf-plugins-core -y
cat <<EOF | sudo tee /etc/yum.repos.d/rke2.repo
[rke2-common]
name=RKE2 Common
baseurl=https://rpm.rancher.io/rke2/latest/common/
enabled=1
gpgcheck=0
EOF
reposync --repoid=rke2-common --download-path=/srv/rke2-rl-rpm --download-
metadata
```



오프라인 준비, RKE2 RPM(레드햇 계열)

저장소 등록한다. 아파치 서버는 적절하게 설정한다.

```
dnf install httpd httpd-core -y
tee /etc/yum.repos.d/rke2.repo <<EOF
[rke2-sle]
name=RKE2 RL Local Repo
enabled=1
autorefresh=1
baseurl=http://<서버IP>/rke2-rl-rpm/
type=rpm-md
gpgcheck=0
EOF
# reposync --repoid=rke2-common --download-path=/srv/rke2-rl-rpm --
download-metadata
```



오프라인 준비, RKE2 RPM(수세)

RPM 미러링도 아래와 같이 준비한다. 여기서 버전은 v1.33.2, stable로 명시.

```
mkdir -p /srv/rke2-sle && cd /srv/rke2-sle-rpm
TAG="v1.33.2%2Brke2r1.stable.0"
BASE_URL="https://github.com/rancher/rke2-
packaging/releases/download/${TAG}"

curl -s https://github.com/rancher/rke2-
packaging/releases/expanded_assets/${TAG} \
  | grep -oP 'href="[^"]+\.sle\.x86_64\.rpm"' \
  | cut -d'"' -f2 \
  | sed 's|^|https://github.com|' \
  | xargs -n1 wget
createrepo_c /srv/rke2-sle-rpm
```



오프라인 준비, RKE2 RPM(수세)

아래와 같이 확인이 가능하다. 아파치 서버는 적절하게 설정한다.

```
zypper in createrepo_c apache2
createrepo_c /srv/rke2-sle-rpm
systemctl enable --now apache2
tee /etc/zypp/repos.d/rke2.repo <<EOF
[rke2-sle]
name=RKE2 SLE Local Repo
enabled=1
autorefresh=1
baseurl=http://<서버IP>/rke2-sle/
type=rpm-md
gpgcheck=0
EOF
```



RPMS

RPMS은 각 배포판별 미리 준비해야 한다. 수세 및 레드햇 계열은 보통 다음과 같이 필요하다.

운영체제	필요한 저장소	설명
RHEL	extras, epel-release	container-selinux, policycoreutils 등
Rocky/Alma	AppStream, BaseOS, extras	기본 활성화됨
SLES 15+	SUSE PackageHub, Basesystem Module, Containers Module	runc, containerd, iptables, iptables-nft 등 포함
openSUSE	Main Repository, Containers	대부분 포함되어 있음



RKE2 INSTALL

오프라인 설치를 진행하기 위해서 오프라인 설치 시 필요한 Artifacts 위치 명시 후, 아래와 같이 실행한다.

```
INSTALL_RKE2_ARTIFACT_PATH=/root/rke2-artifacts sh install.sh
```



레지스트리 서버(디렉터리/인증서)

디렉터리 생성 후, docker-registry서버를 구성한다. 인증서 기간은 기본 1년으로 되어 있다. 더 기간 사용을 원하는 경우, 아래와 같이 10년으로 생성한다.

```
mkdir -p /opt/registry/{data,certs,auth} && cd /opt/registry
```

```
openssl req -newkey rsa:4096 -nodes -sha256 -keyout certs/domain.key -x509  
-days 3650 -out certs/domain.crt -subj "/CN=registry.tang.local"
```

```
# vi /etc/docker/daemon.json  
"insecure-registries": [  
    "registry.tang.local:5000"  
]
```

```
systemctl daemon-reload docker
```



레지스트리 서버(TLS)

레지스트리 서버가 필요한 경우 아래와 같이 구성한다. Podman/Docker 둘 중 아무거나 사용한다. TLS가 없는 경우 해당 옵션을 제외 후, **insecure**로 구성 및 사용한다.

```
podman run -d \  
  --restart=always \  
  --name registry \  
  -v /opt/registry/certs:/certs:Z \  
  -e REGISTRY_HTTP_ADDR=0.0.0.0:5000 \  
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/opt/registry/certs/domain.crt \  
  -e REGISTRY_HTTP_TLS_KEY=/opt/registry/certs/domain.key \  
  -p 5000:5000 \  
  registry:3
```



레지스트리 서버(NON TLS)

TLS 없이, docker-registry를 구성 및 생성한다.

```
podman run -d \  
  --name registry \  
  -p 5000:5000 \  
  -v /srv/docker-registry:/var/lib/registry \  
  registry:2
```



RKE2 이미지

전체 이미지를 다운로드 받는 경우 아래와 같이 작업을 수행한다. 모든 노드에 각각 해당 작업 수행이 필요하다.

```
wget http://10.0.0.250/rke2-v1.33.2+rke2r1-all.tar.gz -O  
/var/lib/rancher/rke2/agent/images/rke2-v1.33.2+rke2r1-all.tar.gz
```

```
ls -l /var/lib/rancher/rke2/agent/images/  
ctr -a /run/k3s/containerd/containerd.sock image import --compress-blobs  
rke2-images.linux-amd64.tar.gz
```

```
ctr -a /run/k3s/containerd/containerd.sock image list
```



Rancher 이미지

Rancher v2.11.3 기준으로 다음과 같이 실행한다.

```
wget https://10.10.10.250/rancher-save-images.sh  
wget https://10.10.10.250/rancher-load-images.sh  
wget http://10.10.10.250/rancher-images.txt
```



Cert-Manager

Cert-Manager 이미지 및 chart도 폐쇄망 설치를 위해서 준비해야 한다.

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
helm fetch jetstack/cert-manager --version v1.18.2
helm template ./cert-manager-v1.18.2.tgz | awk '$1 ~ /image:/ {print $2}' |
sed s/\\"//g >> ./rancher-images.txt

sort -u rancher-images.txt -o rancher-images.txt
```



Rancher Image pull/push

파일로 생성 후, 레지스트리 서버에 업로드 한다.

```
chmod +x rancher-save-images.sh
./rancher-save-images.sh --image-list ./rancher-images.txt

ls -l rancher-images.tar.gz
./rancher-load-images.sh --image-list ./rancher-images.txt --registry
10.0.0.0.250:5000
```



Rancher Image Bundle

번들 이미지를 다음처럼 생성 및 런타임에 적용이 가능하다. 각 노드에서 아래 명령어를 실행한다. 이 방법은 공간을 많이 차지하기 때문에 권장하지는 않는다.

```
docker save -o all-images.tar \  
$(docker images --format '{{.Repository}}:{{.Tag}}')
```

```
docker load -i all-images.tar  
ctr -n=k8s.io images import all-images.tar  
ctr -n=k8s.io images ls
```



이미지 레지스트리 업로드

편하게 업로드 하기 위해서 아래와 같이 **skopeo**를 통해서 업로드 한다. 상황에 맞게 수정해서 사용한다.

```
#!/bin/bash
REGISTRY="localhost:5000"
for image in $(cat rke2-v1.33.2+rke2r1-all.txt); do
    skopeo copy docker-daemon:$image docker://${REGISTRY}/${image}
done
```



레지스트리 서버

/etc/rancher/rke2/registries.yaml에 아래와 같이 작성한다.

```
mirrors:
  "myregistry.local:5000":
    endpoint:
      - "https://myregistry.local:5000"
configs:
  "myregistry.local:5000":
    auth:
      username: myuser
      password: mypass
    tls:
      ca_file: /etc/rancher/rke2/myregistry-ca.crt
# systemctl restart rke2-server
```



참고자료

https://docs.rke2.io/install/private_registry

https://docs.rke2.io/install/registry_mirror

<https://docs.rke2.io/install/airgap>

<https://docs.rke2.io/install/ha>

<https://ranchermanager.docs.rancher.com/how-to-guides/new-user-guides/infrastructure-setup/ha-rke2-kubernetes-cluster>

<https://docs.rke2.io/install/ha>

Rancher 대시보드

설명

최종 확인

rancher cli

rancher dashboard

대시보드

대시보드에 접근하기 위해서 아래와 같이 명령어를 실행한다. 기본 사용자 아이디는 **admin**이다.

```
kubectl get secret --namespace cattle-system bootstrap-secret -o go-  
template='{{.data.bootstrapPassword|base64decode}}{{"\n"}}'  
bfgf2nncnc9fc652kdt75vvgm7rhd96lqltgscjq6q85jt22wv9rx
```

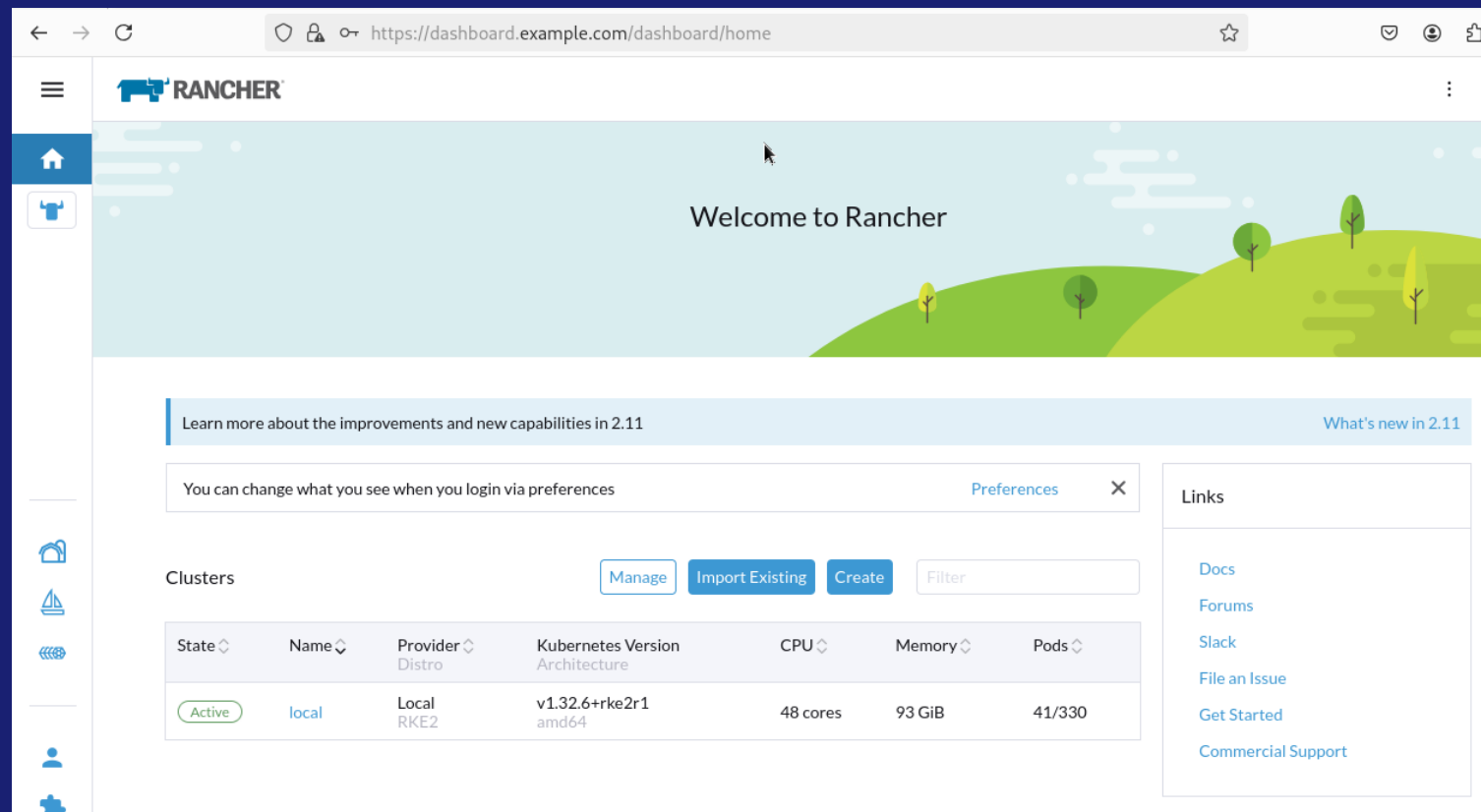
비밀번호 변경은 다음과 같이 가능하다. 현재 rancher의 비밀번호는 **rancher**로 구성이 되어 있다.

```
kubectl -n cattle-system exec $(kubectl -n cattle-system get pods -l  
app=rancher | grep '1/1' | head -1 | awk '{ print $1 }') -- reset-password  
client_config.go:659] Neither --kubeconfig nor --master was specified.  
Using the inClusterConfig. This might not work.  
New password for default admin user (user-qf9kr):  
9rBgwn8z5_tCdiv0l9Y0
```

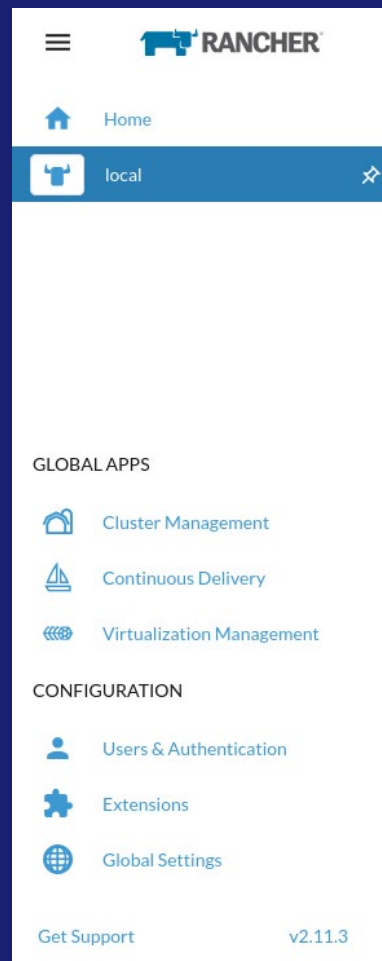


대시보드 접근

접근 시, 올바르게 dnsmasq가 구성이 되어 있으면, <https://dashboard.example.com>으로 접근이 가능하다.

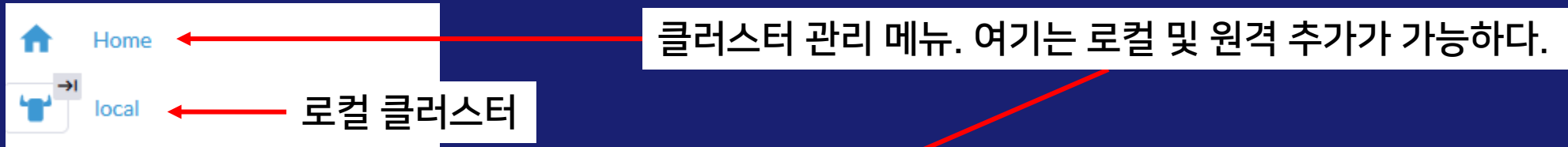


대시보드 기능



대시보드 기능

대시보드 로그인 후, 나오는 화면은 다음과 같다. 첫 화면은 추후에 변경이 가능하다.



Clusters 1

Manage Import Existing Create Filter

State	Name	Provider Distro	Kubernetes Version Architecture	CPU	Memory	Pods
Active	local	Local RKE2	v1.31.5+rke2r1 amd64	24 cores	39 GiB	58/550



RANCHER CLI 설치

Rancher는 CLI도구를 지원한다. `kubectl`명령어 사용 및 관리가 어렵다면, `rancher` CLI도구를 사용한다. 내려받기 및 설치하는 다음과 같이 실행한다. 토큰은 `dashboard`에서 생성 후 사용한다.

```
mkdir ~/bin/  
wget http://10.0.0.250/rancher-linux-amd64-v2.11.3.tar.gz  
tar xf rancher-linux-amd64-v2.11.3.tar.gz  
mv rancher-v2.11.3/rancher ~/bin  
rancher login --token \  
token-h6j2l:jxsqk5pn4nnqdnvvv6t8bslf4bxds275z4pg42kfdql8gq8t7ms5pv \  
https://rancher.example.com
```

```
rancher context ls  
rancher context switch 1  
rancher clusters
```



RANCHER CLI 설정

kubectl 인증 및 사용을 위해서 **kubeconfig** 파일을 다음과 같이 구성이 가능하다. 기존 **KUBECONFIG** 셸 변수도 활용이 가능하다.

다만, **kubectl**은 Rancher에서 제공하는 버전이나 혹은 kubernetes사이트에서 **kubectl** 저장소 구성 후, 설치 및 사용이 가능하다. 이 문서에서는 이 부분에 대해서는 다루지 않는다.

```
rancher kubeconfig > ~/.kube/config  
mv ~/.kube/config ~/.kube/config.backup  
export KUBECONFIG=~/.cluster1.conf
```



설명

Rancher는 K3S혹은 K8S기반으로 대시보드를 제공한다. 쿠버네티스의 대시보드와 비교 하였을 때 다음과 같은 차이가 있다.

항목	Kubernetes Dashboard	Rancher Dashboard
관리 대상	단일 클러스터에 집중	여러 클러스터 및 환경(클라우드, 온프레미스 등) 통합 관리
제공 주체	Kubernetes 공식 프로젝트	Rancher Labs (SUSE)
기본 기능	파드, 서비스, 디플로이먼트 등 기본 리소스 관리 및 모니터링	클러스터 프로비저닝, 업그레이드, 멀티 테넌시, RBAC, 통합 모니터링 등 확장 기능
사용자/역할 관리	RBAC를 통한 기본 사용자 관리 (추가 구성 필요)	내장된 멀티 테넌시 및 상세한 역할 기반 접근 제어 제공



설명

Rancher는 K3S혹은 K8S기반으로 대시보드를 제공한다. 쿠버네티스의 대시보드와 비교 하였을 때 다음과 같은 차이가 있다.

항목	Kubernetes Dashboard	Rancher Dashboard
설치 및 구성	간단하고 가볍게 설치 가능	Rancher 플랫폼의 일부로 설치되며, 초기 구성 및 학습 필요
모니터링 통합	외부 도구와 연동 필요	기본적으로 Prometheus, Grafana 등과 연동되어 통합 모니터링 지원
확장성	단일 클러스터 내 리소스 관리에 적합	다양한 클러스터 및 복잡한 인프라 환경 관리에 적합



설명

Rancher 대시보드에서 다음과 같은 기능을 지원한다.

기능	설명
멀티 클러스터 관리	여러 클러스터를 중앙에서 등록, 모니터링 및 관리하며, 클러스터 추가/삭제 및 상태 확인 가능
클러스터 프로비저닝	클라우드(AWS, GCP, Azure) 및 온프레미스 환경에서 클러스터 생성 및 설정을 쉽게 진행
통합 사용자 및 RBAC 관리	사용자, 팀, 프로젝트 단위의 세분화된 접근 제어를 제공하여 멀티 테넌시 환경에서 권한 관리 용이
애플리케이션 카탈로그 및 배포	Helm 차트 기반의 애플리케이션 카탈로그를 통해 앱 검색, 설치 및 업그레이드를 지원
모니터링 및 로깅 통합	Prometheus, Grafana 등과 연동하여 클러스터 및 애플리케이션의 성능 모니터링, 로그 분석 기능 제공



설명

앞에 내용 이어서...

기능	설명
업그레이드 및 유지보수	클러스터와 관련 구성 요소(예: Nginx Ingress Controller, CoreDNS 등)의 버전 관리 및 업그레이드 중앙 관리
알림 및 이벤트 관리	클러스터 이벤트, 경고, 알림을 한눈에 확인하고 필요한 조치를 취할 수 있도록 지원
보안 및 정책 관리	네트워크 정책, Pod Security Policies 등 보안 관련 설정 관리 및 감사 로그 기능 제공

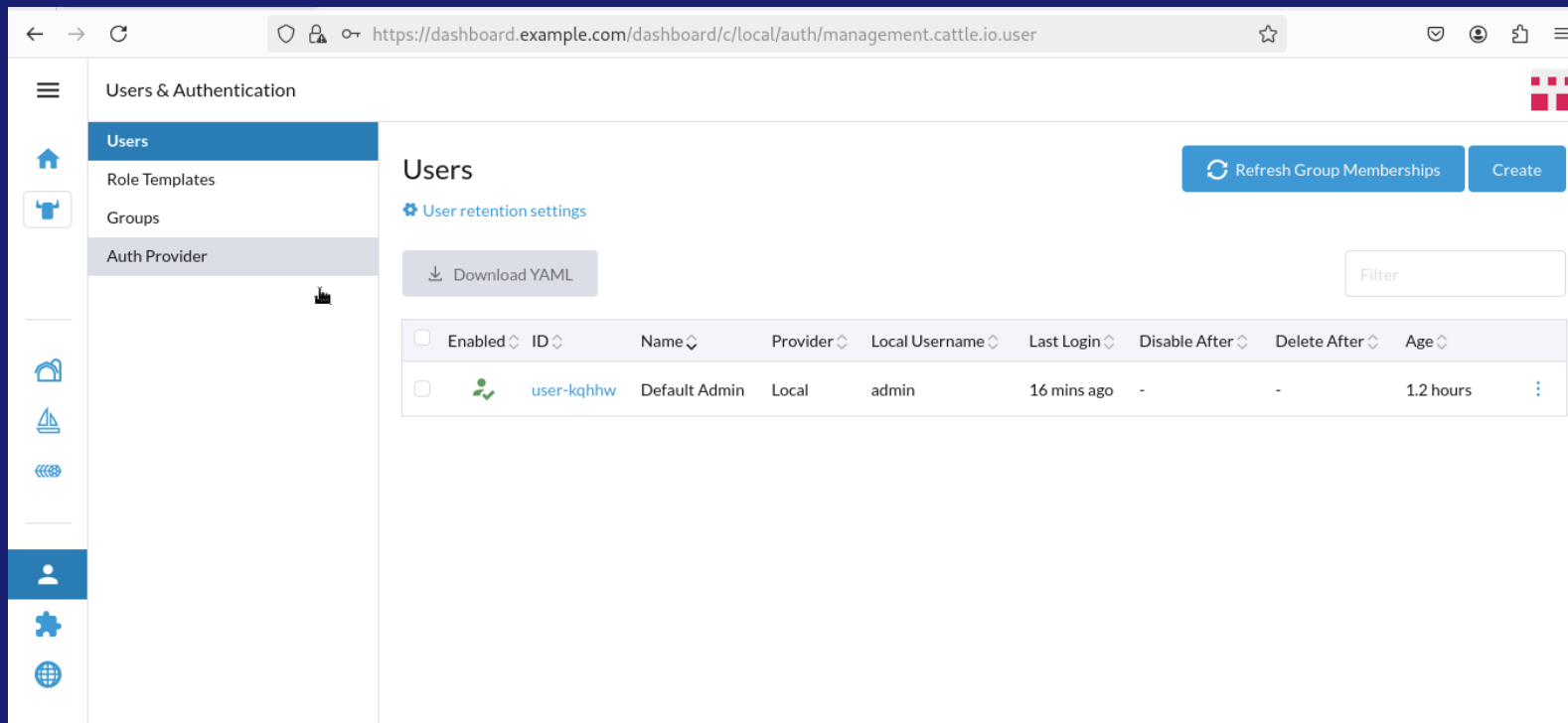


사용자 관리

대시보드

사용자

Rancher는 손쉽게 사용자를 대시보드에서 관리가 가능하다.



The screenshot shows the Rancher web interface for managing users. The left sidebar contains navigation links for Users, Role Templates, Groups, and Auth Provider. The main content area is titled 'Users' and includes a 'Download YAML' button, a 'Filter' input, and a table of users. The table has columns for Enabled, ID, Name, Provider, Local Username, Last Login, Disable After, Delete After, and Age. A single user is listed: 'user-kqhwh' with the role 'Default Admin' and local username 'admin'.

Enabled	ID	Name	Provider	Local Username	Last Login	Disable After	Delete After	Age
☐	user-kqhwh	Default Admin	Local	admin	16 mins ago	-	-	1.2 hours



사용자 생성

사용자 생성은 아래와 같이 가능하다.

User: Create

Credentials

Username *

user1

Display Name

user1

Description

testuser

☐ Ask user to change their password on next login

☐ Generate a random password

New Password *

••••••••

Show

Confirm Password *

••••••••

Show



사용자 권한

Global Permissions

Controls what access the user has to administer the overall Explorer installation.

☐ Administrator

Administrators have full control over the entire installation and all resources in all clusters.

☒ Standard User

Standard Users can create new clusters and manage clusters and projects they have been granted access to.

☐ User-Base

User-Base users have login-access only.



사용자 권한

사용자 권한 범위는 다음과 같다.

권한 등급	설명	클러스터/프로젝트	리소스 관리
Administrator	전체 설치와 모든 클러스터의 모든 리소스를 완전하게 제어 가능	가능	전체 가능
Standard User	자신에게 할당된 클러스터와 프로젝트를 생성하고 관리할 수 있음	가능	지정된 범위만
User-Base	로그인만 가능하며 클러스터/프로젝트 생성 및 리소스 접근은 불가	불가	불가



빌트인 권한

아래 권한은 RKE2에서 추가적으로 제공하는 권한.

Built-in

Additional roles to define more fine-grain permissions model.

☐ Configure Authentication

Allows the user to enable, configure, and disable all Authentication provider settings.

☐ Create new Cluster Drivers

Allows the user to create new cluster drivers and become the owner of them.

☐ Create RKE Template Revisions

No description provided

☐ Manage Users

Allows the user to create, remove, and set passwords for all Users.

☐ Configure Feature Flags

Allows the user to enable and disable custom features via feature flag settings.

☐ Create new Clusters

Allows the user to create new clusters and become the owner of them. Standard Users have this permission by default.

☐ Manage Roles

Allows the user to define, edit, and remove Role definitions.

☐ View Rancher Metrics

Allows the user to view Metrics through the API.

☐ Configure Node Drivers

Allows the user to enable, configure, and remove all Node Driver settings.

☐ Create new RKE Cluster Templates

Allows the user to create new RKE cluster templates and become the owner of them.

☐ Manage Settings

Allows the user to manage Rancher Settings.



빌트인 권한 설명

권한 항목	설명
Configure Authentication	인증 공급자 설정을 활성화/비활성 및 구성할 수 있음
Configure Feature Flags	기능 플래그(실험적 기능 등)를 설정하고 비활성화할 수 있음
Configure Node Drivers	노드 드라이버를 활성화, 구성, 삭제할 수 있음
Create new Cluster Drivers	클러스터 드라이버를 생성하고 그 소유자가 될 수 있음
Create new Clusters	새로운 클러스터를 생성할 수 있음 (Standard User는 기본적으로 가짐)
Create new RKE Cluster Templates	새로운 RKE 클러스터 템플릿을 생성하고 소유자가 될 수 있음



빌트인 권한 설명

권한 항목	설명
Create RKE Template Revisions	RKE 템플릿의 리비전을 생성할 수 있음 (설명은 UI에 없음)
Manage Users	사용자를 생성, 삭제하고 비밀번호를 설정할 수 있음
Manage Roles	사용자 역할(Role)을 정의, 수정, 삭제할 수 있음
Manage Settings	Rancher의 전역 설정을 관리할 수 있음
View Rancher Metrics	Rancher API를 통해 메트릭 정보를 조회할 수 있음



계정 설명

사용자가 생성이 되면 다음과 같이 화면에 출력 및 표시가 된다.

☐	Enabled	ID	Name	Provider	Local Username	Last Login	Disable After	Delete After	Age	
☐		user-kqhww	Default Admin	Local	admin	26 mins ago	-	-	1.4 hours	
☐		u-n84p2	user1	Local	user1	-	-	-	5 secs	



역할 템플릿

Role Templates

Create Global Role

Global Cluster Project/Namespaces

Download YAML

Delete

Filter

☐ State	Display Name	Name	Built-In	New User Default	Age	
☐ Active	Administrator	admin	✓	—	1.4 hours	⋮
☐ Active	Configure Authentication	authn-manage	✓	—	1.4 hours	⋮
☐ Active	Create new Clusters	clusters-create	✓	—	1.4 hours	⋮
☐ Active	Create RKE Template Revisions	clustertemplaterevisions-create	✓	—	1.4 hours	⋮
☐ Active	Create new RKE Cluster Templates	clustertemplates-create	✓	—	1.4 hours	⋮
☐ Active	Configure Feature Flags	features-manage	✓	—	1.4 hours	⋮
☐ Active	Create new Cluster Drivers	containerdrivers-manage	✓	—	1.4 hours	⋮



Authentication Providers

Rancher is configured to allow access to accounts in its local database. [Manage Accounts](#)
Local Authentication is always enabled, but you may select another additional authentication provider from those shown below.

 ActiveDirectory LDAP	 FreeIPA LDAP	 OpenLDAP LDAP	 AzureAD OAuth
 GitHub OAuth	 Google OAuth	 Generic OIDC OIDC	 Keycloak OIDC
 ADFS SAML	 Keycloak SAML	 Okta SAML	 Ping Identity SAML



CLI

CLI로 사용자 생성 및 구성은 가능하다. 하지만, REK2는 K8S와 구조적으로 조금 다른 부분이 있기 때문에, 가급적이면 Rancher를 통해서 생성 및 관리를 강력하게 권장한다.



사용자 생성 및 구성

위의 설명 기반으로 사용자를 생성한다.

```
kubectl create namespace user1-namespace
namespace/user1-namespace created
openssl genrsa -out user1.key 2048
openssl req -new -key user1.key -out user1.csr -subj "/CN=user1/O=group1"
openssl x509 -req -in user1.csr -CA
/var/lib/rancher/rke2/server/tls/client-ca.crt -CAkey
/var/lib/rancher/rke2/server/tls/client-ca.key -CAcreateserial -out
user1.crt -days 365
```



사용자 생성 및 구성

등록된 사용자를 인증서에 등록한다.

```
kubectl config set-credentials user1 --client-certificate=user1.crt \  
--client-key=user1.key
```

```
kubectl config set-context user1-context \  
--cluster=default --namespace=user1-namespace --user=user1
```

```
User "user1" set.Context "user1-context" created.
```

```
kubectl get csr
```

```
user1, pending
```

```
kubectl certificate approve user1
```

```
kubectl get csr
```

```
user1, Approved, Issued
```



CSR 등록

등록된 사용자를 인증서에 등록한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: user1
spec:
  request: $(cat user1.csr | base64 | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  usages:
    - client auth
EOF
```



ROLE구성

다시 서명 요청 후, 노란색 명령어 다시 실행.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: user1-namespace
  name: user1-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "create", "delete"]
EOF
```



ROLE구성

사용자와 그리고 생성된 네임스페이스를 연결한다. 이때, roleBinding를 사용해서 구성한다.

```
# cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: user1-namespace
  name: user1-rolebinding
subjects:
- kind: User
  name: user1
  apiGroup: rbac.authorization.k8s.io
```



ROLE구성

앞에 내용 계속 이어서...

```
roleRef:  
  kind: Role  
  name: user1-role  
  apiGroup: rbac.authorization.k8s.io  
EOF
```



확인하기

생성된 사용자 및 네임스페이스 자원이 올바르게 바인딩이 되었는지 확인한다. 앞에서 만든 RBAC는 POD만 허용하였다.

```
kubectl config get-contexts
```

```
kubectl --context=user1-context get pods
```

```
No resources found in user1-namespace namespace.
```

```
kubectl --context=user1-context get deployments
```

```
Error from server (Forbidden): deployments.apps is forbidden: User "user1" cannot list resource "deployments" in API group "apps" in the namespace "user1-namespace"
```



확장

기존 내용은 POD만 허용했기 때문에, user1-namespace의 모든 자원에 접근이 가능하도록 수정한다. 아래와 같이 내용을 수정한다.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: user1-namespace
  name: user1-role
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
EOF
```



확장

자원 접근이 올바르게 되는지 적용 및 확인한다.

```
kubectl --context=user1-context get all -n user1-namespace  
No resources found in user1-namespace namespace.
```

```
kubectl --context=user1-context create deployment nginx --image=nginx -n  
user1-namespace  
deployment.apps/nginx created
```

```
kubectl --context=user1-context expose deployment nginx --port=80 --target-  
port=80 --type=ClusterIP -n user1-namespace  
service/nginx exposed
```



결과

결과는 다음과 같이 출력이 된다.

```
kubectl get pod,svc,deployment,rs
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-6799fc88d8-xxxxx	1/1	Running	0	10s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/nginx	ClusterIP	10.96.12.34	<none>	80/TCP	5s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx	1/1	1	1	10s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nginx-6799fc88d8	1	1	1	10s



결론

가급적이면 RKE2에서는 Rancher를 통해서 사용자 구성 및 관리를 적극적으로 권장. kubectl에서는 다중 클러스터 관리가 어려운 부분이 많음.

다중 클러스터 관리를 CLI로 하는 경우 rancher 명령어로 권장.



클러스터 관리 기능

프로젝트/네임스페이스

프로젝트/네임스페이스

Cluster

Projects/Namespaces

Nodes 3

Cluster and Project Members

Events 5

Tools

Workloads >

Apps >

Service Discovery >

Storage >

Policy >

More Resources >

local

Only User Namespaces

Move

Download YAML

Delete

Create Project

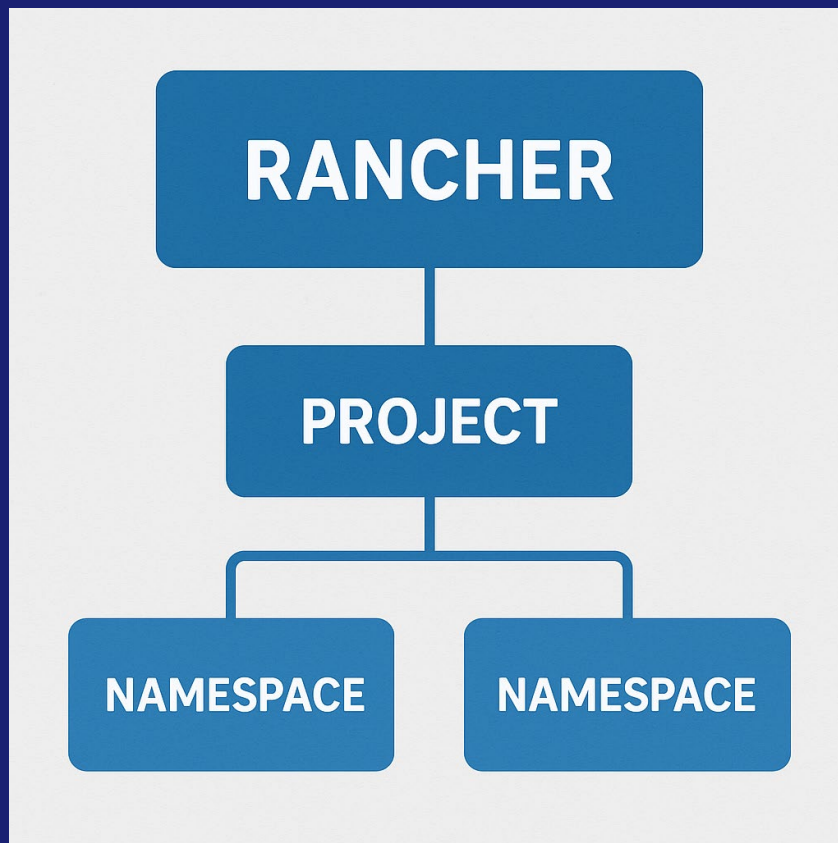
Filter

State	Name	Age
Project: Default Default project created for the cluster		
☐ Active	default	13 hours
Not in a Project		
☐ Active	local	1.5 hours
☐ Active	user-kqhhw	37 mins
Project: System System project created for the cluster		

There are no namespaces defined.



프로젝트/네임스페이스



프로젝트

Project: Create

Name *

A unique name

Description

Any text you want that better describes this resource

Members

Resource Quotas

Container Default Resource Limit

Labels & Annotations

Members

Add

사용자 추가는 여기서 가능하다.



프로젝트 생성 및 권한

Project: Create

Name
qa-team

Members

Resource Quotas

Container Default
Resource Limit

Labels & Annotations

Add Project Member

Select Member

user1
Local

Project Permissions

Controls what access users have to the Project

☐ Owner
Owners have full control over the Project and all resources inside it.

☒ Member
Members can manage the resources inside the Project but not change the Project itself.

☐ Read Only
Members can only view the resources inside the Project but not change the resources.

☐ Custom
Choose individual roles for this user.

Cancel Add

별도로 RBAC 구성이 가능
기본 정책은 아래와 같이 제공



프로젝트 생성

Project: Create

Name *
qa-team

Description
Any text you want that better describes this resource

Members

Resource Quotas

Container Default Resource Limit

Labels & Annotations

Resource Quotas

Resource Type

Project Limit

Namespace Default Limit

Remove

Config Maps

Config Maps

CPU Limit

Memory Limit

Persistent Volume Claims

Pods

Replication Controllers

CPU Reservation

Memory Reservation

Storage Reservation

Secrets

Services

Services Load Balancers

Service Node Ports

e.g. 50

e.g. 50

프로젝트에서는 주요 자원에 대한 개수 제한이 가능



프로젝트 옵션

Members

Resource Quotas

Container Default Resource Limit

Labels & Annotations

Container Default Resource Limit

CPU Reservation
e.g. 1000

^
v

mCPUs

Memory Reservation
e.g. 128

^
v

MiB

CPU Limit
e.g. 1000

^
v

mCPUs

Memory Limit
e.g. 128

^
v

MiB

NVIDIA GPU Limit/Reservation
e.g. 1

^
v

GPUs

애플리케이션 컨테이너 CPU/MEM 제한



프로젝트 옵션

Members

Resource Quotas

Container Default Resource Limit

Labels & Annotations

Labels & Annotations

Labels

Key/value pairs that are attached to objects which specify identifying attributes.

Add Label

Annotations

Add Annotation

검색 조건 혹은 특정 기능 활성화



프로젝트 확인

Project: qa-team

프로젝트는 네임스페이스의 상위 개념

Create Namespace ⋮

There are no namespaces defined.



네임스페이스 생성

Namespace: Create

Name *
api-test

Description
Any text you want that better describes this r

네임 스페이스는 프로젝트에 상속

Container Resource Limit

Pod Security Admission

Labels & Annotations

Container Resource Limit

Configure how much of the resources the container can consume by default.

CPU Reservation
e.g. 1000 mCPUs

Memory Reservation
e.g. 128 MiB

CPU Limit
e.g. 1000 mCPUs

Memory Limit
e.g. 128 MiB

NVIDIA GPU Limit/Reservation
e.g. 1 GPUs

POD는 CPU/MEMORY만 제한이 가능하다.
GPU는 아직 AMD는 지원안함.



네임스페이스 보안

Namespace: Create

Name *

api-test

Description

Any text you want that better describes this namespace

Container Resource Limit

Pod Security Admission

Labels & Annotations

Pod Security Admission

Define the admission control mode you want to use for the pod security

☐ Enforce

restricted



Version (default: latest)

☐ Audit

privileged



Version (default: latest)

☐ Warn

privileged



Version (default: latest)

POD 보안 적용



네임스페이스 확인

Project: qa-team

Create Namespace

☐

Active

api-test

9 secs



CLI

`kubectl` 명령어로 조회는 가능하다. 다만, 생성은 `kubectl` 명령어로 불가능 하다. 추가 및 관리하기 위해서 `rancher` 명령어 사용을 권장한다.

```
kubectl get project -A
```

NAMESPACE	NAME	BACKINGNAMESPACE	AGE
local	p-2zlgk	p-2zlgk	19h
local	p-mrspr	p-mrspr	19h

```
rancher projects create myproject
```

```
rancher projects
```

```
rancher namespaces move user1-namespace --project p-abcde --cluster c-xxxxx
```



클러스터 관리 기능

노드 관리

노드관리

Nodes ☆

Download YAML

Delete

Filter

☐	State ▾	Name ▾	Roles ▾	Version ▾	External/Internal IP ▾	OS ▾	CPU ▾	RAM ▾	Pods ▾	Age ▾	
☐	Active	rancher-node1.example.com	Control Plane, Etcd	v1.32.6+rke2r1	- / 10.0.1.130	Linux	6.8%	7.3%	13%	19 hours	⋮
☐	Active	rancher-node2.example.com	Control Plane, Etcd	v1.32.6+rke2r1	- / 10.0.3.251	Linux	5.9%	7%	13%	18 hours	⋮
☐	Active	rancher-node3.example.com	Control Plane, Etcd	v1.32.6+rke2r1	- / 10.0.2.11	Linux	2.9%	7.2%	12%	18 hours	⋮



TAINT

Node: rancher-node1.example.com Active

Age: 19 hours

Name *

rancher-node1.example.com

Description

Any text you want that better describes this resource

Taints

Labels & Annotations

Taints

Key 

Value

Effect

e.g. foo

e.g. bar

NoSchedule



Remove

Add Taint



ANNOTATIONS

Node: rancher-node1.example.com Active

Detail Config YAML ⋮

Age: 19 hours

Internal IP: [10.0.1.130](#) Version: v1.32.6+rke2r1 OS: Rocky Linux 9.6 (Blue Onyx) Container Runtime: containerd://2.0.5-k3s1

Labels: beta.kubernetes.io/arch: amd64 beta.kubernetes.io/instance-type: rke2 beta.kubernetes.io/os: linux kubernetes.io/arch: amd64 kubernetes.io/hostname: rancher-node1.example.com kubernetes.io/os: linux
node-role.kubernetes.io/control-plane: true node-role.kubernetes.io/etcd: true node-role.kubernetes.io/master: true node.kubernetes.io/instance-type: rke2

Annotations: [Show 13 annotations](#)

✓ PID Pressure ✓ Disk Pressure ✓ Memory Pressure ✓ kubelet

CPU
Used 0.97 of 16 / 6.1%

MEMORY
Used 2.34 of 31 GiB / 7.5%

PODS
Used 14 of 110 / 13%



CLI

노드 관리는 `kubectl`, `rancher` 두 개 명령어 사용이 가능하다. 하지만, 안전하게 관리하기 위해서는 Rancher Dashboard에서 관리를 강력하게 권장한다.

```
rancher nodes --cluster c-xxxxx
```

ID	Hostname	State	Roles
m-xxxxx	node1.local	active	etcd,controlplane
m-yyyyy	node2.local	active	worker

```
rancher nodes update m-xxxxx --label foo=bar
```

```
rancher nodes delete m-xxxxx
```



CLI

kubectl로 하는 경우 이전과 동일하다.

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
node1.local	Ready	control-plane	5d	v1.30.0
node2.local	Ready	worker	5d	v1.30.0

```
kubectl label nodes node2.local environment=dev zone=us-west
```

```
kubectl get nodes --show-labels
```

```
kubectl taint nodes node2.local dedicated=ml:NoSchedule
```

```
kubectl label nodes node2.local environment- zone-
```

```
kubectl taint nodes node2.local dedicated=ml:NoSchedule-
```



클러스터 관리 기능

노드 관리

클러스터/프로젝트 사용자 관리

Cluster and Project Members ☆

Caution: This is the cluster that Rancher is using as a data store. Only administrators should be given write access to this cluster. Users with write access to this cluster can use it to grant themselves access to any part of this installation.

Cluster Membership Project Membership

- 특정 클러스터의 관리자 구성이 필요 시 사용
- 특별한 경우 아니면, 가급적이면 기본값 그대로 사용
- 이 정보는 Rancher에서만 지원함

Add



Filter

☐ State

Name

Role

Age

There are no rows to show.



클러스터 이벤트

☐	State ▾	Last Seen ▾	Type ▾	Reason ▾	Object ▾	Subobject ▾	Source ▾	Message ▾	First Seen ▾	Count ▾	Name ▾	
Namespace: default												
☐	Active	2m5s	Warning	ErrGetKeyPair	ClusterIssuer rancher-ca-issuer		cert-manager-clusterissuers	Error getting keypair for CA issuer: secrets "rancher-root-ca-secret" not found	6h57m	89	rancher-ca-issuer.1851fb9ace0ddacc	⋮



클러스터 기능 추가

- 앞서 Longhorn 스토리지 추가했음
- helm명령어나 혹은 대시보드에서 처리 가능
- 가급적이면 대시보드 사용 권장

The screenshot displays the Rancher Helm charts interface. An orange callout box highlights the text: "앞서 Longhorn 스토리지 추가했음", "helm명령어나 혹은 대시보드에서 처리 가능", and "가급적이면 대시보드 사용 권장". The interface shows a grid of charts with icons, names, versions, descriptions, and "Install" buttons. The charts include: Istio (v106.3.1+up1.25.0), Logging (v106.0.2+up4.10.0-rancher.6), Longhorn (v106.2.1+up1.8.2), Monitoring (v106.1.2+up69.8.2-rancher.7), NeuVector (v106.0.1+up2.8.6), and Elemental (v106.0.1+up1.6.9). Some charts have a "Deploys on Windows" tag.

Chart Name	Version	Description	Install Button
Istio	v106.3.1+up1.25.0	A basic Istio setup that installs with the istioctl. Refer to https://istio.io/latest/ for details.	Install
Logging	v106.0.2+up4.10.0-rancher.6	Logging operator for Kubernetes based on Fluentd and Fluentbit.	Install
Longhorn	v106.2.1+up1.8.2	Longhorn is a distributed block storage system for Kubernetes.	Install
Monitoring	v106.1.2+up69.8.2-rancher.7	kube-prometheus-stack collects Kubernetes manifests, Grafana dashboards, and Prometheus rules combined with documentation and scripts to provide easy to operate end-to...	Install
NeuVector	v106.0.1+up2.8.6	Helm feature chart for NeuVector container security platform.	Install
NeuVector Monitor	v106.0.1+up2.8.6	Helm feature chart (optional) add-on to NeuVector for monitoring with Prometheus/Grafana.	Install
Elemental	v106.0.1+up1.6.9	Elemental provides Cloud Native OS Management for Cluster Nodes.	Install



클러스터 추가

Rancher는 다중 클러스터 관리에 특화된 미들웨어 솔루션이다. Rancher로 추가가 되는 클러스터 유형은 다음과 같다.

1. RKE1/2
2. Kubernetes
3. K3S

하지만, 이 이외 쿠버네티스 기반으로 구성된 모든 유형에서 확장 및 관리가 가능하다.



클러스터 추가

클러스터 추가를 위해서 기존에 설치 된 RKE2 혹은 K8S를 미리 Rancher에서 등록한다.

Learn more about the improvements and new capabilities in 2.11

What's new in 2.11

You can change what you see when you login via preferences

Preferences

Clusters

Manage

Import Existing

Create

Filter

State	Name	Provider Distro	Kubernetes Version Architecture	CPU	Memory	Pods
Active	aaaaaa	Imported RKE2	v1.32.6+rke2r1 amd64	4 cores	15 GiB	16/110
Active	bbbbbb	Imported RKE2	v1.32.6+rke2r1 amd64	4 cores	15 GiB	16/110
Active	local	Local RKE2	v1.32.6+rke2r1 amd64	12 cores	46 GiB	105/330

Links

Docs

Forums

Slack

File an Issue

Get Started

Commercial Support



클러스터 추가

Cluster: Import

Register an existing cluster in a hosted Kubernetes provider



Amazon EKS



Azure AKS



Google GKE

Import any Kubernetes cluster



Generic



클러스터 추가

Cluster: Import Generic

Import Harvester Clusters via Virtualization Management

Cluster Name *

A unique name for the cluster

Cluster Description

Any text you want that better describes this cluster

Basics

K8s version management

Future changes to the system default setting will result in redeployment of the cluster agent.

☒ Global default (enabled)

☐ Enabled

☐ Disabled

Version management for imported clusters is globally enabled.

Member Roles

User	Role
Default Admin (admin) Local	Cluster Owner

Add

Labels and Annotations

Advanced



클러스터 추가

Cluster: test Provisioning

Explore **Detail** Config YAML ⋮

Namespace: fleet-default Age: 3 secs

This resource is currently in a transitioning state, but there isn't a detailed message available.

Provisioner: Imported
Annotations: Show 1 annotation

Provisioning Log Registration Conditions

You should not import a cluster which has already been connected to another instance of Rancher as it will lead to data corruption.

Run the `kubectl` command below on an existing Kubernetes cluster running a supported Kubernetes version to import it into Rancher:

```
kubectl apply -f https://rancher.example.com/v3/import/sdf8tzvrjqfzw4ck2c6gd1kxrgwb8cksh4kpt9vgt6kq2zqrk6ps2h_c-gw2cr.yaml
```

If you get a "certificate signed by unknown authority" error, your Rancher installation has a self-signed or untrusted SSL certificate. Run the command below instead to bypass the certificate verification:

```
curl --insecure -sL https://rancher.example.com/v3/import/sdf8tzvrjqfzw4ck2c6gd1kxrgwb8cksh4kpt9vgt6kq2zqrk6ps2h_c-gw2cr.yaml | kubectl apply -f -
```

If you get permission errors creating some of the resources, your user may not have the `cluster-admin` role. Use this command to apply it:

```
kubectl create clusterrolebinding cluster-admin-binding --clusterrole cluster-admin --user <your username from your kubeconfig>
```

CA가 OS에 등록이 안되어 있으면 TLS 문제로 접근이 안된다.
두 번째 명령어를 통해서 YAML 내려받기 후 적용한다.



클러스터 추가

올바르게 등록이 되면, 추가된 클러스터에 다음과 같이 POD 확인이 가능하다.

```
kubectl get pod -n cattle-system
```

```
cattle-fleet-system    fleet-agent-6c7d5cf7ff-gjn29
cattle-system          cattle-cluster-agent-8f694fb87-4r79w
cattle-system          dashboard-shell-4ncmm
cattle-system          rancher-webhook-6d67cf7c8c-r98jc
cattle-system          system-upgrade-controller-766f9c8f9d-k8k9j
```



클러스터 추가가 안되는 경우

coredns에서 외부 도메인 질의가 되지 않는 경우가 대다수 이다.

위의 슬라이드 coredns부분에 명시가 된 부분을 추가하려는 클러스터에 적용한다. 바닐라 버전 쿠버네티스를 사용하는 경우 configmap에서 수정하여도 된다. 바닐라 쿠버네티스에서는 다음과 같이 수정한다.

```
kubectl get pod -n cattle-system
```

```
cattle-fleet-system    fleet-agent-6c7d5cf7ff-gjn29
cattle-system          cattle-cluster-agent-8f694fb87-4r79w
cattle-system          dashboard-shell-4ncmm
cattle-system          rancher-webhook-6d67cf7c8c-r98jc
cattle-system          system-upgrade-controller-766f9c8f9d-k8k9j
```



클러스터 추가가 안되는 경우

아래와 같이 수정 및 추가한다.

```
kubectl -n kube-system edit configmap coredns
```

```
.:53 {  
  hosts {  
    192.168.10.250 rancher.example.com  
    fallthrough  
  }  
}  
  
prometheus :9153  
forward . 192.168.10.250 8.8.4.4  
}
```

```
kubectl -n kube-system rollout restart deployment coredns
```



워크로드

dashboard

RANCHER WORKLOADS

Cluster >

Workloads >

CronJobs (0)

DaemonSets (0)

Deployments (0)

Jobs (0)

StatefulSets (0)

Pods (0)

Apps >

Service Discovery >

Storage >

Policy >

More Resources >

Workloads ☆

Create

☰

📁

Filter

State	Name	Type	Image	Restarts	Age	Health
There are no rows to show.						



POD

애플리케이션 묶음

POD(CONTAINER)

Cluster >

Workloads ▾

CronJobs (→) 0

DaemonSets (→) 0

Deployments (→) 0

Jobs (→) 0

StatefulSets (→) 0

Pods (→) 0

Apps >

Service Discovery >

Storage >

Policy >

More Resources >

Pod: Create

Namespace *
default ▾

Name *

A unique name

Description
Any text you want that better describes this resource

Pod

container-0

+ Add Container

General

Health Check

Resources

Security Context

Storage

General

Container Name
container-0

☐ Init Container

☒ Standard Container

Image

Container Image
e.g. nginx:latest

Pull Policy
Always ▾

Pull Secrets

i

▾

■ 기본적으로 실행은 Standard Container

오픈소스 Rancher

4/1/2024

337

POD(NETWORK)

Networking

Define a Service to expose the container, or define a non-functional, named port so that humans will know where the app within the container is expected to run.

If ClusterIP, LoadBalancer, or NodePort is selected, a Service is automatically created that will select the Pods in this workload using labels.

Service Type Do not create a service	Name	Private Container Port e.g. 8080	Protocol TCP	Add Host	Remove
---	------	-------------------------------------	-----------------	--------------------------	------------------------

[Add Port or Service](#)

쿠버네티스 SVC 영역
kubectl get svc

Command

Command e.g. /bin/sh	Arguments e.g. /usr/sbin/httpd -f httpd.conf
WorkingDir e.g. /myapp	Stdin No



POD(SA,CMD)

Command

Command
e.g. /bin/sh

Arguments

e.g. /usr/sbin/httpd -f httpd.conf

WorkingDir

e.g. /myapp

Stdin

No

Environment Variables

Add Variable

Service Account Name

Service Account Name

Clear

Lifecycle Hooks

PostStart

None

Add command to execute

Create HTTP request

PreStop

None

Add command to execute

Create HTTP request

SA 계정으로 실행이 필요한 경우



POD(CGROUP)

neral ⓘ

Resources

Security Context

Storage

Resources

CPU Reservation
e.g. 1000

⌵ ⌴

mCPUs

Memory Reservation
e.g. 128

⌵ ⌴

MiB

CPU Limit
e.g. 1000

⌵ ⌴

mCPUs

Memory Limit
e.g. 128

⌵ ⌴

MiB

NVIDIA GPU Limit/Reservation
e.g. 1

⌵ ⌴

GPUs



POD(SECCOMP)

General ⓘ

Resources

Security Context

Storage

Security Context

Privileged

☒ No

☐ Yes: container has full access to the host

Run as Non-Root

☒ No

☐ Yes: container must run as a non-root user

Run as User ID

Add Capabilities

Privilege Escalation

☒ No

☐ Yes: container can gain more privileges than its parent process

Read-Only Root Filesystem

☒ No

☐ Yes: container has a read-only root filesystem

Drop Capabilities

컨테이너 프로세서 실행 시 Pod에서 제공할 보안 범주



POD(STORAGE)

General ⓘ

Resources

Security Context

Storage

Storage

Select Volume ▼

사용할 StorageClass 선택



POD비교

Rancher(Vanila)/OCP POD와 비교하였을 때 다음과 같은 차이가 있다.

항목	설명	주요 역할/용도	특징
Pod	Kubernetes의 기본 실행 단위로, 하나 이상의 컨테이너가 그룹화되어 실행됨	컨테이너 그룹이 동일 네임스페이스와 스토리지를 공유하며 배포됨	애플리케이션의 최소 배포 단위로, 네트워크와 볼륨을 공유
Kubernetes Pause	Pod 생성 시 먼저 실행되어 네트워크 네임스페이스를 유지하는 최소 기능 컨테이너	Pod 내 다른 컨테이너가 공유할 네트워크(및 PID) 네임스페이스 제공	일반적으로 k8s.gcr.io/pause 이미지를 사용하며, 단순한 인프라 컨테이너 역할 수행
Rancher Pause	Rancher에서 클러스터 관리 시 사용하는 pause 컨테이너 이미지의 변형	Rancher 환경에서 Pod의 네트워크 격리를 유지하는 인프라 컨테이너 역할	Kubernetes Pause와 유사하나, Rancher 클러스터 관리에 맞춰 커스터마이징된 버전



POD비교

이어서...

항목	설명	주요 역할/용도	특징
Podman Pause	Podman CLI 명령어를 통해 실행 중인 컨테이너의 프로세스를 일시 중지하는 기능	컨테이너의 실행 상태를 "freeze"하여 리소스 사용량 감소 또는 디버깅 목적으로 사용	컨테이너의 내부 프로세스를 중지시켰다가 재개할 수 있음 네트워크 네임스페이스 유지 목적이 아님
OpenShift Pause	OpenShift가 Pod의 네트워크 구성을 위해 사용하는 pause 컨테이너 이미지	Pod 생성 시 네트워크 네임스페이스를 유지하여, 동일 네트워크 환경 제공	Kubernetes Pause와 유사하며, OpenShift 환경에 맞게 별도 이미지로 관리
catatonit	경량화된 OCI init 프로세스로, 컨테이너의 첫 프로세스로 사용됨	컨테이너의 init 역할(자식 프로세스 재aping, 신호 전달)과 네임스페이스 유지(일부 기능 포함)	매우 작은 용량과 빠른 시작, 기본 init 기능을 수행하며, 필요에 따라 pause 역할로도 활용 가능



POD비교

엔터프라이즈급 POD 프로그램은 서로 다른 방식으로 `/pause` 프로그램을 지원하고 있다. Rancher 표준을 최대한 준수 하고 있다.

항목	OpenShift Pause	Rancher Pause
프로그램 이름	/catatonic 인프라 컨테이너 - OpenShift 배포 시 Red Hat 인증 이미지를 사용 (예: ose-pause)	/pause 인프라 컨테이너 - 기본적으로 Kubernetes upstream pause를 기반 - 필요에 따라 Rancher용으로 재태깅(re-tag)된 버전을 사용
이미지 레포지토리	OpenShift 4 이상은 내부 이미지 스트림으로 관리	기본적으로 k8s.gcr.io/pause (또는 그 변형) - 이미지를 사용하거나 Rancher에서 관리 - 혹은 커스텀 저장소의 pause 이미지를 사용



POD비교

앞에 내용 이어서...

항목	OpenShift Pause	Rancher Pause
사용 환경	OpenShift 클러스터 내 Pod 생성 시 네트워크 네임스페이스를 유지하기 위한 인프라 컨테이너로 사용	Rancher가 관리하는 클러스터 내에서 Pod 네트워크 구성을 위해 사용하며, Rancher UI 및 운영 정책에 맞게 제공
차이점	- OpenShift 환경에 최적화 - Red Hat의 보안 패치 및 인증 과정을 거친 이미지 제공	- Kubernetes의 기본 pause 기능을 그대로 활용 - Rancher 플랫폼 통합 및 클러스터 관리와 운영 정책에 맞게 커스터마이징 가능 - 높은 호환성 제공
	- OpenShift 고유의 이미지 스트림	- 멀티 클러스터 관리 시 통일된 이미지 정책 적용 용이 - 클러스터의 /pause버전이 달라도 표준 준수하기에 서로 호환 가능



POD비교

정리하면 다음과 같다.

1. Rancher는 표준 POD기반으로 사용하기 때문에, 바닐라 버전과 높은 호환성 및 빠른 수행보장.
2. OCP는 표준 POD기반이 아닌, /catatonit기반으로 동작한다.



CLI

CLI는 `kubectl`, `rancher` 둘 다 사용이 가능하다. 하지만, Dashboard 기반으로 사용을 권장한다.

```
vi nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-nginx
  namespace: user1-namespace
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```



CLI

CLI는 `kubectl`, `rancher` 둘 다 사용이 가능하다. 다만, Dashboard에서 제공하는 기능을 CLI로 100%지원하지 않기 때문에 가급적이면 Dashboard에서 작업을 권장한다.

```
kubectl apply -f nginx-pod.yaml  
kubectl run my-nginx --image=nginx --port=80 \  
--restart=Never --namespace=user1-namespace
```



CLI

rancher 명령어를 사용하는 경우 다음과 같다.

```
rancher login https://<RANCHER_URL> --token token-xxx:yyy --context c-xxxxx  
rancher context switch user1-context  
rancher kubectl apply -f nginx-pod.yaml
```



STATEFULSET

상태자원

StatefulSet 설명 1

StatefulSet은 일반적으로 3-Tier 구조의 레거시 애플리케이션이나 고정된 네트워크 식별자와 저장소를 요구하는 **Client/Server** 형태의 서비스에 적용된다.

이 리소스는 각 Pod에 고정된 이름(DNS)과 안정적인 스토리지를 제공함으로써 기존 애플리케이션이 Kubernetes 환경에서도 변경 없이 동작할 수 있도록 돕는다.

StatefulSet은 Headless 서비스와 함께 사용하는 것이 일반적이며, Headless 서비스란 **ClusterIP**를 사용하지 않고, 대신 각 Pod의 IP를 그대로 DNS를 통해 노출하는 방식이다.



StatefulSet 설명 2

이를 통해 **로드밸런싱이 아닌 직접적인 DNS 기반 접근이 가능**해지며, 이는 기존의 Stateful 애플리케이션에서 요구하는 고정 호스트네임 기반 통신 구조를 충족시킨다.

Headless 서비스는 쿠버네티스 외에도 여러 아키텍처에서 유사 개념으로 사용되지만, 쿠버네티스에서는 DNS 서브도메인을 통해 각 Pod에 접근 가능하게 구성하는 것이 특징이다.



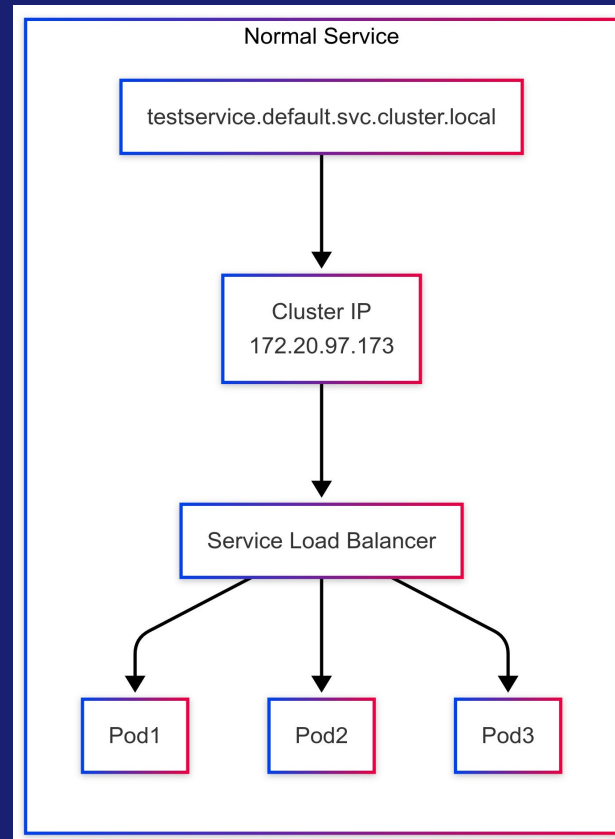
StatefulSet 정리

STS에 대해서 정리하면 다음과 같다.

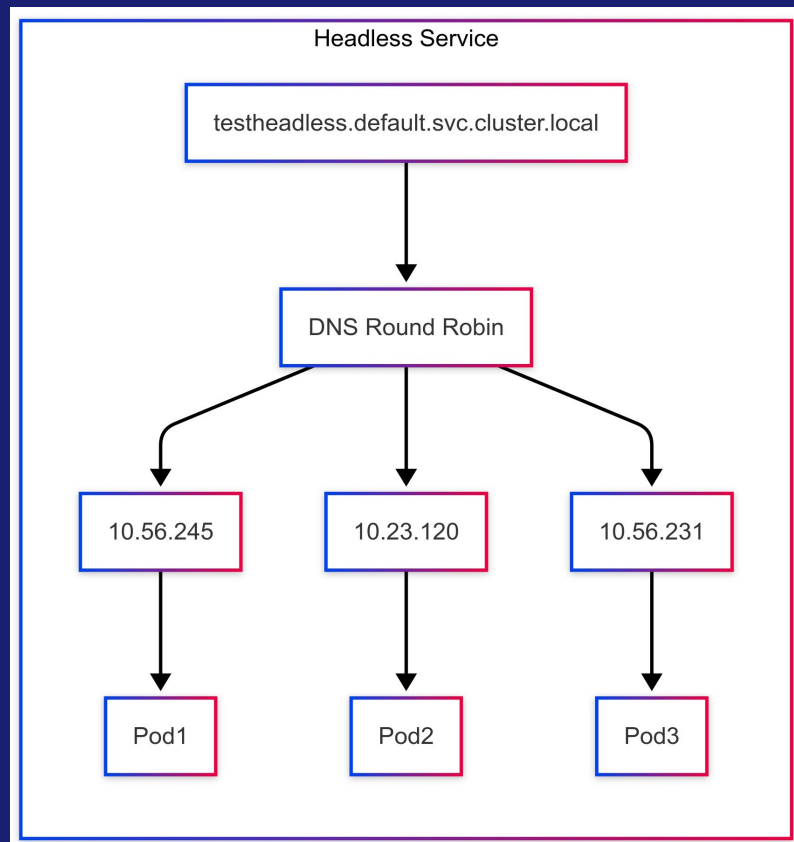
항목	설명
StatefulSet	상태 유지, 고정 네트워크 ID 및 PVC 제공
Headless Service	clusterIP: None 설정, DNS만 제공, 로드밸런싱 없음
기존 앱 호환성	기존 C/S 구조 또는 DB 등의 Stateful 앱과 궁합이 좋음
DNS 기반 접근	podname.servicename.namespace.svc.cluster.local 형식



StatefulSet



StatefulSet



StatefulSet(CONTAINER)

StatefulSet: Create

Namespace *

default

Name *

A unique name

Description

Any text you want that better describes this resource

Replicas *

1

Service Name *

StatefulSet

Pod

container-0

+ Add Container

General

Health Check

Resources

Security Context

Storage

General

Container Name

container-0

Init Container

Standard Container

Image

Container Image

e.g. nginx:latest

Pull Policy

Always

Pull Secrets



StatefulSet(NETWORK)

Networking

Define a Service to expose the container, or define a non-functional, named port so that humans will know where the app within the container is expected to run.

[Add Port or Service](#)

Command

Command

e.g. /bin/sh

Arguments

e.g. /usr/sbin/httpd -f httpd.conf

WorkingDir

e.g. /myapp

Stdin

No



Environment Variables

[Add Variable](#)

Service Account Name

Service Account Name



[Clear](#)



StatefulSet(SA)

Service Account Name

Service Account Name



Clear

Lifecycle Hooks

PostStart

- ☒ None
- ☐ Add command to execute
- ☐ Create HTTP request

PreStop

- ☒ None
- ☐ Add command to execute
- ☐ Create HTTP request



StatefulSet(PROBE)

General ⓘ

Health Check

Resources

Security Context

Storage

Health Check

Readiness Checks ⓘ

Type
None

Liveness Check ⓘ

Type
None

Startup Check ⓘ

Type
None



StatefulSet(CPU/MEMORY)

General ⓘ

Health Check

Resources

Security Context

Storage

Resources

CPU Reservation
e.g. 1000

↕

mCPUs

Memory Reservation
e.g. 128

↕

MiB

CPU Limit
e.g. 1000

↕

mCPUs

Memory Limit
e.g. 128

↕

MiB

NVIDIA GPU Limit/Reservation
e.g. 1

↕

GPUs



StatefulSet(SECCOMP)

General ⓘ

Health Check

Resources

Security Context

Storage

Security Context

Privileged

☒ No
☐ Yes: container has full access to the host

Run as Non-Root

☒ No
☐ Yes: container must run as a non-root user

Run as User ID

Add Capabilities

Drop Capabilities

Privilege Escalation

☒ No
☐ Yes: container can gain more privileges than its parent process

Read-Only Root Filesystem

☒ No
☐ Yes: container has a read-only root filesystem



StatefulSet(STORAGE)

General ⓘ

Health Check

Resources

Security Context

Storage

Storage

Select Volume ▾



StatefulSet

StatefulSet 서비스를 생성 및 구성한다. 구성 시 클러스터 아이피가 없어야 한다.

```
# kubectl apply -f sts-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: sts-nginx
  labels:
    app: sts-nginx
spec:
  ports:
    - port: 80
      name: web
```



StatefulSet

앞에 내용 계속 이어서...

```
clusterIP: None
selector:
  app: sts-nginx
# kubectl apply -f sts-svc.yaml
# kubectl get -f sts-svc.yaml
```



StatefulSet

StatefulSet 자원을 생성한다.

```
# vi sts.yaml
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-app
  labels:
    app: sts-app
spec:
  serviceName: "nginx"
  replicas: 2
```



StatefulSet

앞에 내용 계속 이어서...

```
selector:
  matchLabels:
    app: sts-app
template:
  metadata:
    labels:
      app: sts-app
  spec:
    containers:
      - name: nginx
        image: registry.k8s.io/nginx-slim:0.21
        ports:
          - containerPort: 80
        name: web
```



StatefulSet

앞에 내용 계속 이어서...

```
# kubectl apply -f sts.yaml  
# kubectl get -f sts.yaml
```



CronJob

작업 예약

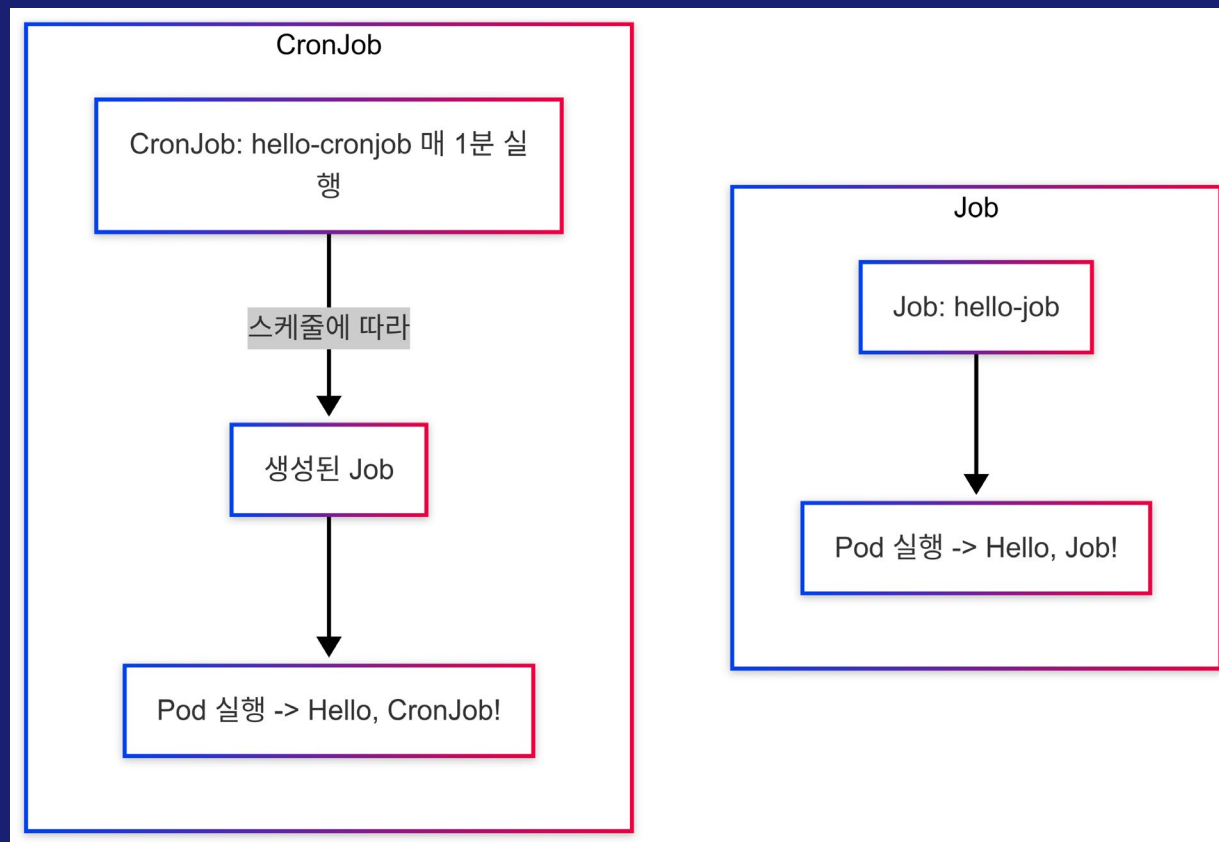
설명

Job, CronJob의 기능 차이는 다음과 같다.

항목	Job	CronJob
목적	일회성 또는 단발성 작업을 실행하여 완료 될 때까지 관리	정해진 일정에 따라 주기적으로 작업을 실행 (크론 스케줄러 사용)
실행 방식	Job 리소스가 생성되면 단 한 번 실행되고 완료될 때까지 실행	CronJob 스케줄에 따라 Job 리소스를 주기적으로 생성하여 실행
용도	데이터 마이그레이션, 일회성 배치 작업, 단 발 테스트 등	정기적인 백업, 리포트 생성, 스케줄링된 데이터 처리 등
관리	Job이 완료되면 기본적으로 리소스가 남을 수 있으며, ttlSecondsAfterFinished 옵션을 통해 정리 가능	CronJob은 Job을 주기적으로 생성하므로, 완료된 Job 관리 및 정리 필요



설명(CRONJOB/JOB)



JOBS

Jobs ☆

Create



Filter



State ↕

Name ↕

Image ↕

Completions ↕

Duration ↕

Restarts

Age ↕

Health

There are no rows to show.



JOBS

Job

Pod

container-0 

+ Add Container

Labels & Annotations

Scaling and Upgrade Policy

Labels & Annotations

Labels

Key/value pairs that are attached to objects which specify identifying attributes.

Add Label

Annotations

Add Annotation



JOBS

Job

Pod

container-0 

+ Add Container

Labels & Annotations

Scaling and Upgrade Policy

Scaling and Upgrade Policy

Completions



Times 

Back Off Limit



Times 

Pod Active Deadline



Seconds

Parallelism



Times 

Active Deadline

Seconds 

CRONJOBS

CronJobs ☆ Create

☰ ■

☐ State ⚡	Name ⚡	Image ⚡	Schedule ⚡	Last Schedule ⚡	Restarts	Age ⚡	Health
There are no rows to show.							



CRONJOBS

CronJob: Create

Namespace *

default



Name *

A unique name



Description

Any text you want that better describes this resource

Schedule *

0 * * * *



JOB YAML

다음과 같이 작성 및 생성이 가능하다.

```
vi hello-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      containers:
      - name: hello
        image: busybox
        command: ["echo", "Hello, Job!"]
      restartPolicy: Never
    backoffLimit: 4
```



JOB

다음과 같이 작성 및 생성이 가능하다.

```
kubectl apply -f hello-job.yaml  
job.batch/hello-job created
```

```
kubectl get pods --selector=job-name=hello-job
```

NAME	READY	STATUS	RESTARTS	AGE
hello-job-xxxxx	0/1	Completed	0	30s

```
kubectl logs <pod-name>  
Hello, Job!
```



CRONJOB

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
kubectl apply -f hello-cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cronjob
spec:
  schedule: "*/1 * * * *" # 매 1분마다 실행
```



CRONJOB

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
jobTemplate:
  spec:
    template:
      spec:
        containers:
        - name: hello
          image: busybox
          command: ["echo", "Hello, CronJob!"]
          restartPolicy: Never
        successfulJobsHistoryLimit: 3
        failedJobsHistoryLimit: 1
```



CRONJOB

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
kubectl apply -f cronjob-example.yaml  
cronjob.batch/hello-cronjob created
```

```
kubectl get jobs --watch
```

NAME	COMPLETIONS	DURATION	AGE
hello-cronjob-1615939200	1/1	2s	1m
hello-cronjob-1615939260	1/1	2s	30s ...

```
kubectl get pods --selector=job-name=hello-cronjob-1615939200
```

```
kubectl logs <pod-name>
```

```
Hello, CronJob!
```



명령어

명령어로 생성 및 구성이 가능하다.

```
kubectl create job hello-job --image=busybox -- /bin/sh -c "echo Hello, Job!"
```

```
job.batch/hello-job created
```

```
kubectl get pods --selector=job-name=hello-job
```

```
kubectl logs <pod-name>
```

```
Hello, Job!
```



명령어

명령어로 생성 및 구성이 가능하다.

```
kubectl create cronjob hello-cronjob --image=busybox --schedule="*/1 * * *  
*" -- /bin/sh -c "echo Hello, CronJob!"  
cronjob.batch/hello-cronjob created
```

```
kubectl get cronjob hello-cronjob  
kubectl get jobs --watch  
kubectl get pods --selector=job-name=<cronjob-generated-job-name>  
  
kubectl logs <pod-name>  
Hello, CronJob!
```

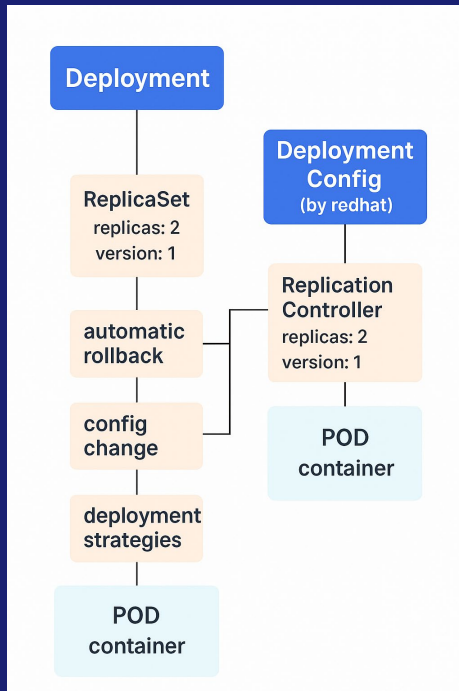


DEPLOYMENT

설정배포

Deployment 설명

Deployment는 ReplicaSet과 함께 도입이 되었다. 생각보다 버전 안정화는 늦었다. GA기준 1.9에서 안정화가 되었다. Deployment의 주요 역할은 애플리케이션 설정 내용을 가지고 ReplicaSet으로 전달 및 배포가 주요 목적이다.



Deployment 구성 1

Deployment 구성은 다음과 같이 구성이 되어있다.

```
vi deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
```

이 부분은 기존과 동일하다.

"replicas: 3"를 생성한다.

Deployment가 관리할 RS를 찾습니다. RS에 해당 레이블이 존재해야 한다.



Deployment 구성 2

계속...

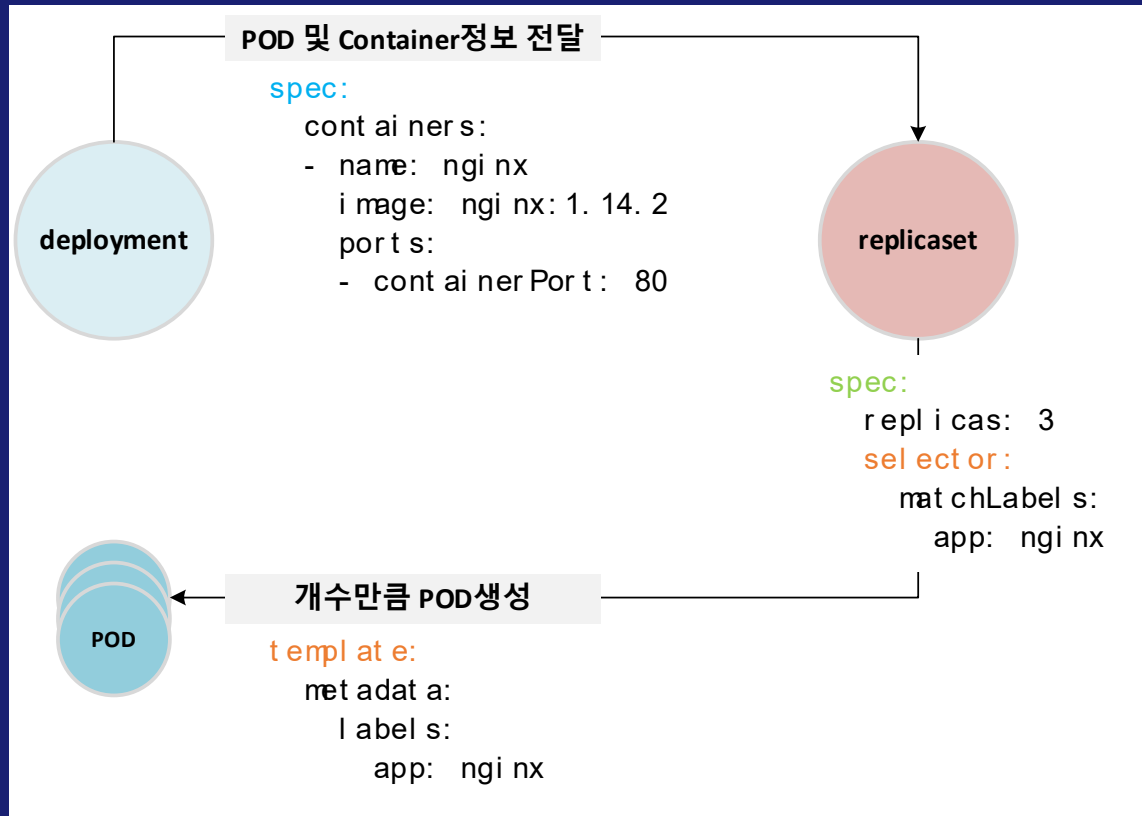
```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
          - containerPort: 80
```

"template",은 "matchLabels:"과 동일해야 한다.



Deployment 동작방식

도식으로 표현하면 다음과 같이 동작한다.



Deployment 확인

Deployment에 구성된 자원은 하나만 존재하며, Deployment가 갱신이 되면, 해당 내용에 대해서 ReplicaSet에서 기록을 가지고 있다. 이미지나 혹은 설정 내용에 변경이 발생하면, 다음처럼 ReplicaSet에서 기록이 남는다.

```
kubectl get deploy
```

NAME	READY	AVAILABLE
AGEdeployment.apps/nginx-deployment	3/3	3

```
kubectl get replicaset
```

NAME	DESIRED	CURRENT
nginx-deployment-665fd86dbb	1	1
nginx-deployment-77d8468669	3	3



DEPLOYMENT 대시보드 1

Deployments ☆ Create

☰

📄

Filter

☐ State	Name	Image	Ready	Up To Date	Available	Restarts	Age	Health
There are no rows to show.								



DEPLOYMENT 대시보드 2

Deployment: Create

Namespace *

default

Name *

A unique name

Description

Any text you want that better describes this resource

Replicas *

1

Deployment

Pod

container-0

+ Add Container

General

Health Check

Resources

Security Context

Storage

General

Container Name

container-0

Init Container

Standard Container

Image

Container Image

e.g. nginx:latest

Pull Policy

Always

Pull Secrets

Networking

Define a Service to expose the container, or define a non-functional, named port so that humans will know where the app within the container is expected to run.

Add Port or Service



DAEMONSET

노드 자원

DaemonSet 설명

DaemonSet은 Deployment와 기능상 동일하다. DaemonSet도 Deployment와 동일하게 2015년도에 정식으로 GA되었다. Deployment와 제일 큰 다른 부분은 Deployment는 Scheduler를 통해서 클러스터에 구성된 노드에 배포가 주요 목적이다.

하지만, DaemonSet은 Deployment와 동일하게 Scheduler를 통하지만, 각 노드당 하나씩 POD생성이 주요 목적이다. 다시 말하지만, 컨테이너가 아닌 POD를 각 노드에 구성이다. 쿠버네티스에서 대표적인 DaemonSet POD는 kube-proxy이다. kube-proxy는 모든 노드에 최소 한 개씩 구성이 된다.

제한적으로 DaemonSet이 구성이 되는 서비스는 kube-apiserver, kube-scheduler, kube-controller-manager와 같은 POD가 있다. 이 서비스들은 Compute노드를 제외한, Controller노드에 구성이 된다.



DaemonSet 대시보드

DaemonSets ☆ Create

Filter

☐ State ◇	Name ◇	Image ◇	Ready ◇	Current ◇	Desired ◇	Restarts	Age ◇	Health
There are no rows to show.								



DaemonSet 구성

대몬셋은 다음과 같이 구성 및 작성한다. 올바른 학습을 위해서 최소 2개 이상의 컴퓨트 노드를 권장한다. 그렇지 않는 경우, 컴퓨트 노드에만 생성이 된다.

컨트롤러에도 생성하기 위해서는 **톨러레이션(toleration)**를 같이 구성해야 한다. 여기서는 **톨러레이션을 구성하지 않는다**. 순수하게 컴퓨트 노드에서만 생성한다.

```
spec:
```

```
  spec:
```

```
    tolerations:
```

- key: node-role.kubernetes.io/control-plane
operator: Exists
effect: NoSchedule
- key: node-role.kubernetes.io/master
operator: Exists
effect: NoSchedule



DaemonSet 코드

예를 들어서 모든 노드에 로그 분석 에이전트가 필요한 경우 아래와 같이 생성 및 구성.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
```



DaemonSet

위의 내용 이어서 계속...

```
template:
  metadata:
    labels:
      name: fluentd-elasticsearch
  spec:
    containers:
      - name: fluentd-elasticsearch
        image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2
```



DaemonSet

위의 내용 이어서 계속...

```
resources:
  limits:
    memory: 200Mi
  requests:
    cpu: 100m
    memory: 200Mi
volumeMounts:
- name: varlog
  mountPath: /var/log
```



DaemonSet

자원 생성 후, 올바르게 반영이 되었는지 확인한다.

```
terminationGracePeriodSeconds: 30
volumes:
- name: varlog
  hostPath:
    path: /var/log
```

EOF



HelmChart

패키지 관리

랜처 애플리케이션

Cluster >

Workloads >

Apps ▾

Charts

Installed Apps (0)

Repositories 4

Recent Operations (0)

Service Discovery >

Storage >

Policy >

More Resources >

Charts

Browse Featured

Featured Charts



rancher-partner-charts

JFrog Artifactory HA

Universal Repository Manager supporting all major packaging formats, build tools and CI servers.



All charts have at least one version that is installable on clusters with Linux and Windows nodes unless otherwise indicated.

All ▾

All Categories ▾

Filter charts results

⌂

☐ Show deprecated apps



Alerting Drivers

The manager for third-party webhook receivers used in Prometheus Alertmanager



CIS Benchmark

The cis-operator enables running CIS benchmark security scans on a kubernetes cluster



Elemental

Elemental provides Cloud Native OS Management for Cluster Nodes.

Linux only



Harvester Cloud Provider

A Helm chart for Harvester Cloud Provider

Linux only



Harvester CSI Driver

A Helm chart for Harvester CSI driver

Linux only



Istio

A basic Istio setup that installs with the istioctl. Refer to https://istio.io/latest/ for details.



Logging

Logging operator for Kubernetes based on Fluentd and Fluentbit.

Deploys on Windows



Longhorn

Longhorn is a distributed block storage system for Kubernetes.

오픈소스 Rancher

402

4/1/2024

차트 프로그램 설치 목록

Installed Apps ☆

☰

▶

Filter

State	Name	Chart	Upgradable	Resources	Age
There are no rows to show.					



차트 저장소

Repositories ☆ Create

Refresh Disable Download YAML Delete Filter

☐ State ◇	Name ◇	Type ◇	URL ◇	Branch ◇	Age ◇	
☐ Active	partner-extensions	git	https://github.com/rancher/partner-extensions	main	7 hours	⋮
☐ Active	Partners	git	https://git.rancher.io/partner-charts	main	8 hours	⋮
☐ Active	Rancher	git	https://git.rancher.io/charts	release-v2.11	8 hours	⋮
☐ Active	RKE2	git	https://git.rancher.io/rke2-charts	main	8 hours	⋮



설치 기록

Recent Operations ☆



Filter

☐ State ↕	Name ↕	Action ↕	Release Namespace ↕	Release Name ↕	Age ↕
There are no rows to show.					



Service Discovery

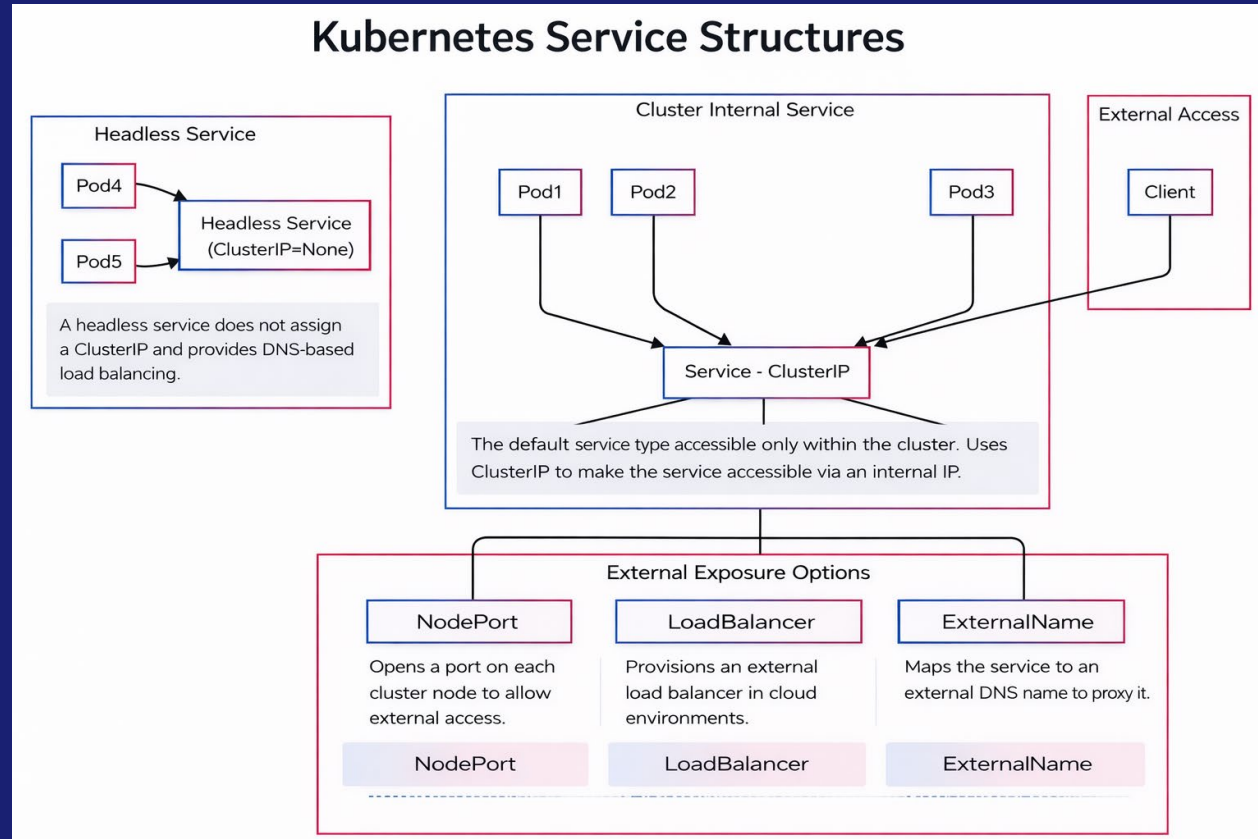
서비스 관리

설명

항목	설명	사용 예시
ClusterIP	클러스터 내부에서만 접근 가능한 가상 IP를 할당해 내부 통신을 지원합니다.	내부 마이크로서비스 간 통신, 클러스터 내 API 서버 접근
NodePort	각 노드의 고정 포트를 통해 외부에서 접근할 수 있도록 노출합니다.	외부 접근이 필요한 테스트 환경이나 개발 단계의 서비스
LoadBalancer	클라우드 제공업체의 로드 밸런서를 이용하여 외부 접근을 위한 단일 IP를 할당합니다.	프로덕션 환경에서 안정적인 외부 접근 제공
ExternalName	DNS 이름을 통해 클러스터 외부의 서비스를 참조할 수 있도록 합니다.	외부 데이터베이스나 API를 내부 서비스처럼 사용하고자 할 때
Headless	클러스터 IP가 할당되지 않아 DNS SRV 레코드를 통해 각 Pod의 IP를 직접 반환, Pod의 개별 주소를 노출합니다.	StatefulSet, 분산 시스템 등에서 Pod 간 직접 통신이나 서비스 디스커버리 목적으로 사용



서비스 아키텍처



기본 서비스 노출

Services ☆

Download YAML

Delete

☰

📁

Filter

☐ State ⚡	Name ⚡	Target	Selector ⚡	Type ⚡	Age ⚡
Namespace: default					
☐ Active	kubernetes	https 📡 6443/TCP		Cluster IP	21 hours ⋮



CLUSTERIP

클러스터 아이피는 다음과 같이 구성이 가능하다. expose는 API에 다음과 같은 정보를 보낸다.

```
kubectl run --image=nginx my-nginx
```

```
kubectl expose pod/my-nginx
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
name: my-nginx
```

```
labels:
```

```
    run: my-nginx
```

```
spec:
```

아래는 참고 코드.
자세한 내용은 강사 가이드



NODEPORT

클러스터 아이피는 외부에서 접근이 불가능 하기 때문에, 외부에서 접근하기 위해서 NodePort로 자원을 노출한다.
아래 명령어는 NodePort구성 내용이다. (파일 생성 하지 않음)

```
kubectl create service nodeport --tcp=8080:80 --node-port=30013 -o yaml --dry-run=client test-httpd
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  labels:
```

```
    app: test-httpd
```

```
  name: test-httpd
```



NODEPORT

spec:

ports:

- name: 8080-80

nodePort: 30013

port: 8080

protocol: TCP

targetPort: 80

selector:

app: test-httpd

type: NodePort



MULTIPORT

한 개 이상의 애플리케이션 노출이 필요한 경우, 아래와 같이 실행이 가능하다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: v1  
kind: Service  
metadata:  
  name: my-nginx-multi-port  
  labels:  
    run: my-nginx
```



MULTIPOINT

위의 내용 계속...

spec:

ports:

- name: normal
port: 80
protocol: TCP
targetPort: 8080
- name: secure
port: 82
protocol: TCP
targetPort: 8082



MULTIPORT

위의 내용 계속...

```
- name: monitoring
  port: 83
  protocol: TCP
  targetPort: 8083
  selector:
    run: my-nginx
EOF
```



명령어(EXPOSE)

명령어로 처리가 가능하다.

```
kubectl create deployment my-deployment --image=nginx --port=8080
```

```
deployment.apps/my-deployment created
```

```
kubectl expose deployment my-deployment --port=80 --target-port=8080 --  
name=my-service
```

```
service/my-service exposed
```

```
kubectl get svc my-service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	ClusterIP	10.96.123.45	<none>	80/TCP	2m



서비스 외부 노출(Ingress)

Ingresses ☆

Create

☰

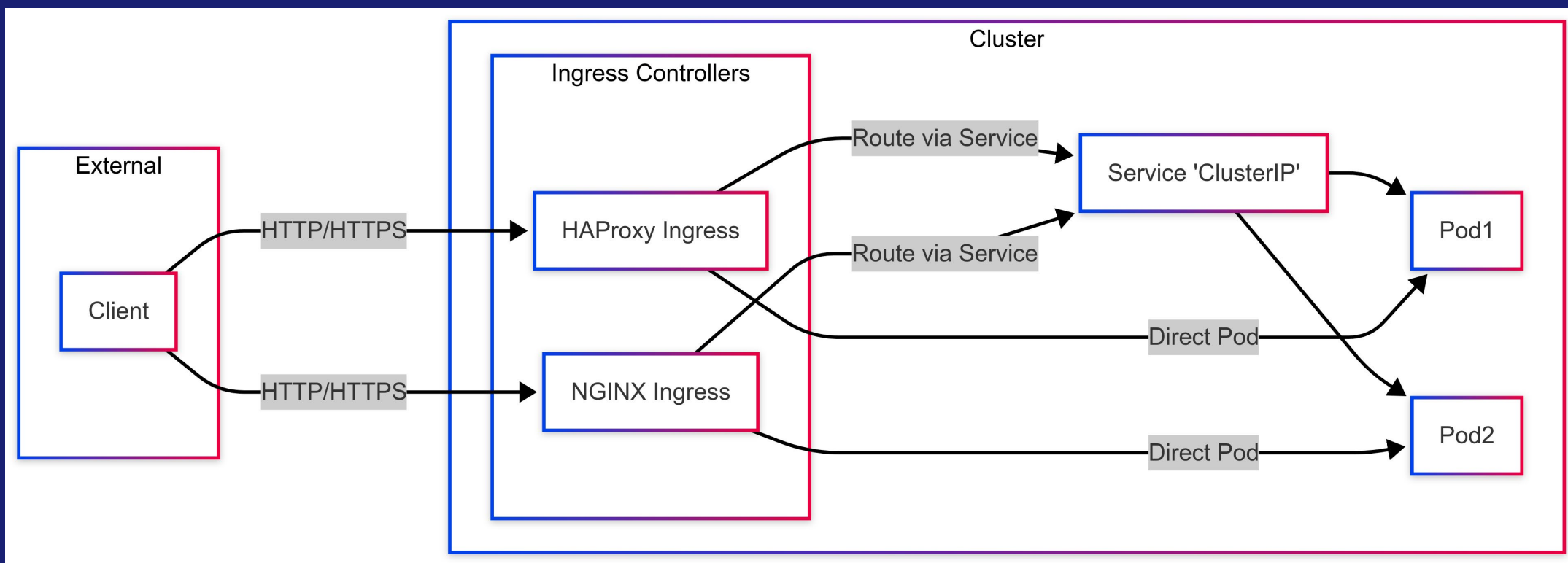
📁

Filter

State	Name	Target	Default	Ingress Class	Age
There are no rows to show.					



설명 (Ingress)



구성(Ingress)

인프라가 올바르게 구성이 되어 있다면, 다음과 같이 인그레스 서비스를 구성한다. 매우 기본적인 기능만 사용한다.

```
vi web-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
```



구성

임시 도메인은 `example.com`으로 선언, 문제가 없으면 접근이 가능하다.

```
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```



구성

올바르게 구성이 되었는지 확인한다.

```
kubectl apply -f web-ingress.yaml
```

```
kubectl get -f web-ingress.yaml
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
web-ingress	<none>	example.com	203.0.113.10	80	1m

```
kubectl get svc -n ingress-nginx
```

ingress-nginx	nginx-ingress-nginx-controller				
LoadBalancer	11.88.128.115	192.168.0.242			

```
kubectl get ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
web-ingress	<none>	example.com		80	4m14s



SCALE

HorizontalPodAutoscalers ☆

Create

☰

■

Filter

State ⚡	Name ⚡	Workload ⚡	Minimum Replicas ⚡	Maximum Replicas ⚡	Current Replicas ⚡	Age ⚡
There are no rows to show.						



SCALE 테스트 애플리케이션 배포 1

테스트를 위해서 아래와 같이 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scale-nginx
  labels:
    app: scale-nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: scale-nginx
```



SCALE 테스트 애플리케이션 배포 2

위의 내용 계속...

```
template:
  metadata:
    labels:
      app: scale-nginx
  spec:
    containers:
      - name: sample-container
        image: busybox
        command: ["/bin/sh"]
        args:
          - -c
          - "while true; do yes > /dev/null; sleep 1; done"
```

계
속



SCALE 테스트 애플리케이션 배포 3

최종적으로 자원 크기를 제한한다. 앞서 이야기 하였지만 쿠버네티스는 CPU, MEM지원한다.

```
resources:  
  requests:  
    cpu: "100m"  
    memory: "128Mi"  
  limits:  
    cpu: "200m"  
    memory: "256Mi"
```

EOF



SCALE 코드 1

애플리케이션 실행 시, 워크로드에 따라서 POD의 개수를 늘리고 줄이기 위해서 HPA기능을 구성해야 한다. HPA는 Horizontal Pod Autoscaler의 약자다. 코드는 다음과 같이 작성이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: scale-nginx
  minReplicas: 2
```

계
속



SCALE 코드 2

CPU사용율이 50%가 넘어가면 POD 확장을 시작한다.

```
metrics:  
- type: Resource  
  resource:  
    name: cpu  
    target:  
      type: Utilization  
      averageUtilization: 50
```

EOF



SCALE

HPA와 같은 스케일링 기반으로 서비스가 구성되는 경우, 스케줄러가 모든 노드 상태를 실시간으로 알 수 없기 때문에 종종, 예상된 숫자보다 더 작게 스케일 아웃(Scale Out)이 되는 경우가 있다. 이러한 이유로, POD개수를 유지할 수 있도록 명시해야 한다. 앞에 내용에 추가한다.

```
maxReplicas: 10
```

```
metrics:
```

```
- type: Resource
```

```
  resource:
```

```
    name: cpu
```

```
  target:
```

```
    type: Utilization
```

```
    averageUtilization: 50
```



Storage

PV/PVC/STORAGE CLASS

스토리지 대시보드

Cluster >

Workloads >

Apps >

Service Discovery >

Storage ▾

PersistentVolumes 0

StorageClasses 0

ConfigMaps (=) 4

PersistentVolumeClaims (=) 0

Secrets (=) 0

Policy >

More Resources >

PersistentVolumes ☆

Create

Filter

State	Name	Reclaim Policy	Persistent Volume Claim	Source	Reason	Age
There are no rows to show.						



스토리지 대시보드

PersistentVolume: Create

Name *

A unique name

Description

Any text you want that better describes this resource

Volume Plugin *

Amazon EBS Disk

▼

Capacity *

10

↑ ↓

GiB

Plugin configuration

Customize

Plugin configuration

Volume ID

e.g. volume1

Partition

0

↑ ↓

Filesystem Type

e.g. ext4

Read Only

☐ Yes

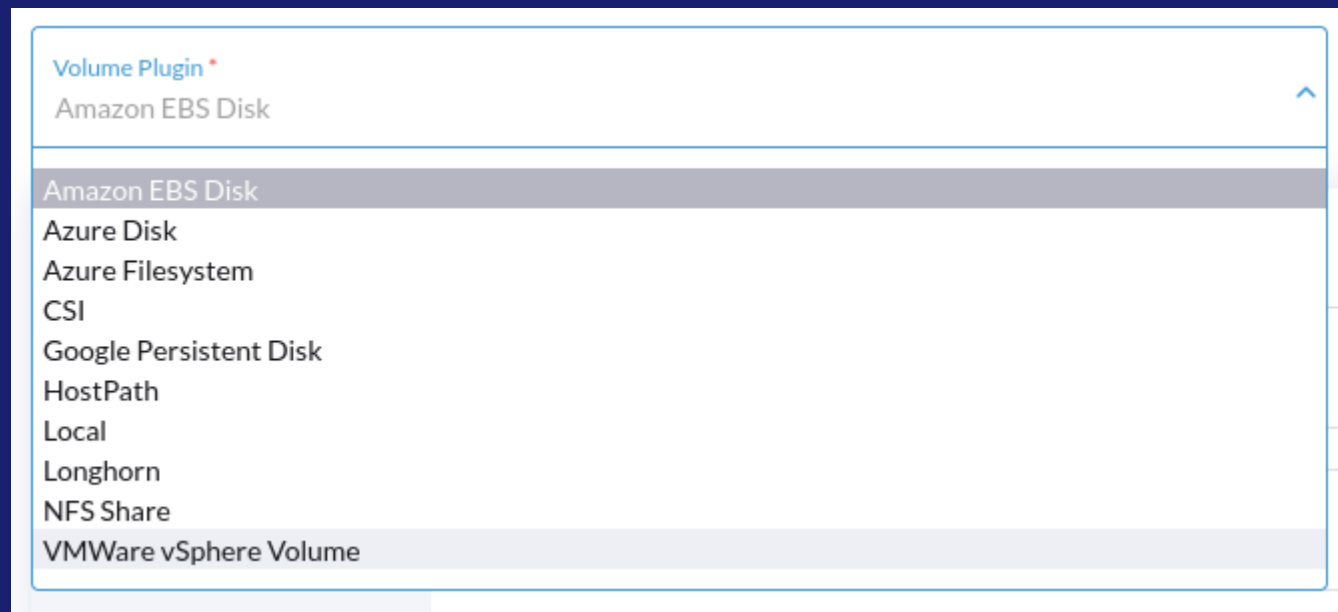
☒ No

오픈소스 Rancher

436

4/1/2024

스토리지 플러그인



스토리지 클래스 드라이버

 Harvester CSI Driver A Helm chart for Harvester CSI driver Linux only	 vSphere CSI vSphere Cloud Storage Interface (CSI)	 Dell CSI PowerFlex VxFlex OS CSI (Container Storage Interface) driver Kubernetes integration. This... Linux only	 Dell CSI PowerMax PowerMax CSI (Container Storage Interface) driver Kubernetes integration. This... Linux only
 Dell CSI PowerScale PowerScale CSI (Container Storage Interface) driver Kubernetes integration. This... Linux only	 Dell CSI PowerStore PowerStore CSI (Container Storage Interface) driver Kubernetes integration. This... Linux only	 Dell CSI Unity Unity XT CSI (Container Storage Interface) driver Kubernetes integration. This... Linux only	 HPE CSI Driver for Kubern... A Helm chart for installing the HPE CSI Driver for Kubernetes Linux only
 HPE CSI Info Metrics Provi... A Helm chart for installing the HPE CSI Info Metrics Provider for Prometheus Linux only	 Nutanix CSI Snapshot Snapshot components required for CSI snapshotting and not specific to any CSI driver Linux only	 Nutanix CSI Storage Nutanix Container Storage Interface (CSI) Driver Linux only	 WekaFS CSI Driver Helm chart for Deployment of WekaIO Container Storage Interface (CSI) plugin for... Linux only
 harvester-csi-driver A Helm chart for Harvester CSI driver Linux only			



스토리지 클래스 대시보드 1

StorageClasses ☆ Create

Filter

State ◇	Name ◇	Provisioner ◇	Default ◇	Age ◇
There are no rows to show.				



스토리지 클래스 대시보드 2

StorageClass: Create

Name *

A unique name

Description

Any text you want that better describes this resource

Provisioner

Longhorn (CSI)

Parameters

Customize

Parameters

Key	Value	
numberOfReplicas	3	Remove
staleReplicaTimeout	2880	Remove
fromBackup	e.g. bar	Remove
diskSelector	e.g. bar	Remove
nodeSelector	e.g. bar	Remove
recurringJobs	e.g. bar	Remove

Add Parameter

Cancel

Edit as YAML

Create



설정파일 대시보드

ConfigMaps ☆ Create

Download YAML Delete Filter

☐ Name ↕	Data	Age ↕
Namespace: api-test		
☐ kube-root-ca.crt	ca.crt	12 hours ⋮
Namespace: default		
☐ kube-root-ca.crt	ca.crt	1 day ⋮
Namespace: local		
☐ kube-root-ca.crt	ca.crt	14 hours ⋮
Namespace: user-kqhhw		
☐ kube-root-ca.crt	ca.crt	13 hours ⋮



스토리지 클래스 클레임 대시보드

PersistentVolumeClaims ☆

Create

☰

📁

Filter

State ⚡	Name ⚡	Status ⚡	Volume ⚡	Capacity ⚡	Access Modes ⚡	Storage Class ⚡	VolumeAttributesClass ⚡	Volume Mode ⚡	Age ⚡
There are no rows to show.									



YAML STORAGE CLASS

StorageClass는 YAML로 구성 시, 다음처럼 가능하다. 다만, CSI 드라이버마다 다르기 때문에 매뉴얼 확인이 필요하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: default
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: nfs.csi.k8s.io
parameters:
  server: storage.example.com
  share: /nfs
```



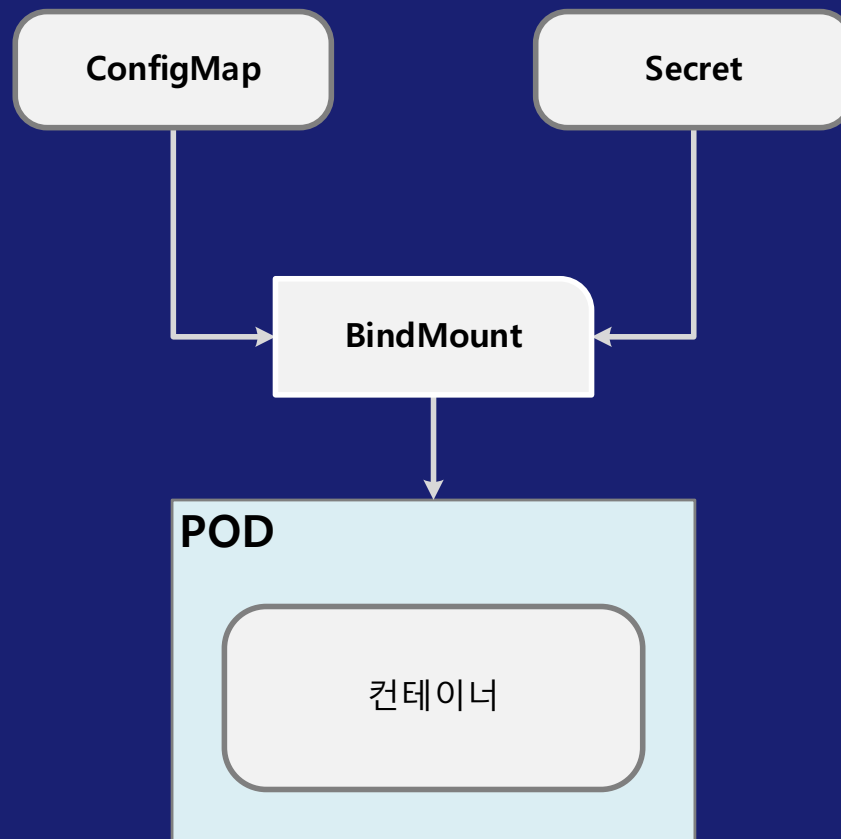
YAML STORAGE CLASS

위의 내용 계속 이어서...

```
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - hard
  - nfsvers=4.1
EOF
```



설명(CONFIGMAP/SECRET)



설명(CONFIGMAP/SECRET)

위의 기능들을 정리하면 다음과 같다.

기능	설명	적용 명령어
ConfigMap 생성	환경 변수 관리	<code>kubectl apply -f my-config.yaml</code>
Secret 생성	비밀번호, API 키 관리	<code>kubectl apply -f my-secret.yaml</code>
Deployment에서 사용	환경 변수로 전달	<code>kubectl apply -f nginx-deployment.yaml</code>
데이터 확인	ConfigMap & Secret 조회	<code>kubectl get configmap my-config -o yaml</code>
		<code>kubectl get secret my-secret -o yaml</code>
Secret 복호화	base64 인코딩된 데이터 확인	<code>kubectl get secret my-secret -o jsonpath="{.data.DB_PASSWORD}"</code>



설명(CONFIGMAP/SECRET)

항목	ConfigMap	Secret
파일로 마운트 가능 여부	가능 (configMap 볼륨 사용)	가능 (secret 볼륨 사용)
Pod 내부에서 사용 가능	가능 (env, volume)	가능 (env, volume)
보안 수준	낮음 (평문 저장)	높음 (Base64 인코딩)
RBAC 적용 가능 여부	불가능 (일반 리소스)	가능 (kubectl get secret 권한 필요)
사용 예제	<code>kubectl create configmap my-config --from-literal=DB_HOST=mydatabase.local</code>	<code>kubectl create secret generic my-secret --from-literal=DB_PASSWORD=SuperSecret123</code>



RANCHER CONFIGMAP 대시보드 1

ConfigMap: Create

Namespace *
default

Name *
A unique name

Description
Any text you want that better describes this resource

Data

Binary Data

Labels & Annotations

Key 
e.g. foo

Value

Remove

Add Read from File



RANCHER CONFIGMAP 대시보드 2

Data

Binary Data

Labels & Annotations

Binary Data

Key 

Value

e.g. foo

e.g. bar

Remove

Add

Read from File



ConfigMap 코드

명령어로 생성하는 방법은 다음과 같다.

```
kubectl create configmap my-config --from-literal=DB_HOST=mydatabase.local  
kubectl get cm
```

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: my-config  
data:  
  DB_HOST: "mydatabase.local"  
EOF
```



RANCHER SECRET 대시보드 1

Secrets ☆ Create

Filter

State ◇	Name ◇	Kind ◇	Data	Age ◇
There are no rows to show.				



RANCHER SECRET 대시보드 2

Secret: Create



Custom Type

Create a Secret with a custom type



Opaque

Default type of Secret using key-value pairs

[More Info](#)



SSH Key

Public key and private key for SSH authentication

[More Info](#)



HTTP Basic Auth

Authentication with a username and password

[More Info](#)



Registry

Authenticated registry for pulling container images

[More Info](#)



TLS Certificate

Store a certificate and key for TLS

[More Info](#)



Secret

명령어로 생성하는 방법은 다음과 같다. 파일로 다음과 같이 작성 및 생성이 가능하다.

```
kubectl create secret generic my-secret \  
--from-literal=DB_PASSWORD=SuperSecret123
```

```
kubectl get secret  
echo -n "SuperSecret123" | base64  
U3VwZXJTZWNYZXQxMjM=
```



Secret

다음과 같이 파일로 작성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  DB_PASSWORD: "U3VwZXJTZWNyZXQxMjM=" # base64 인코딩된 값
EOF
kubectl get secret my-secret -o jsonpath="{.data.DB_PASSWORD}" | base64 --
decode
```



정책

클러스터 Ingress/Egress 정책

자원제한 정책

자원 범위 제한

LimitRange는 쿠버네티스의 네임스페이스 단위 리소스 제한 도구로, 포드(Pod) 또는 컨테이너(Container)에 기본 리소스 요청(requests)과 제한(limits)을 자동으로 적용하거나, 특정 범위 내로 제한할 수 있도록 설정하는 역할을 한다.

이 기능은 다음과 같은 목적을 위해 사용된다:

1. **기본값 설정:** 개발자가 resources를 명시하지 않아도 기본값이 자동 적용되도록 함
2. **상한선 제한:** 컨테이너나 파드가 과도한 리소스를 요청하거나 사용하는 것을 방지
3. **하한선 제한:** 너무 적은 리소스를 요청해 노드에 부담을 주는 작은 작업들을 제한

LimitRange는 네임스페이스에 단 하나 이상 존재할 수 있으며, 해당 네임스페이스에 생성되는 파드/컨테이너에 대해 자동 적용된다.



자원제한 대시보드 1

Cluster >

Workloads >

Apps >

Service Discovery >

Storage >

Policy ▾

Limit Ranges (0)

Network Policies (0)

Pod Disruption Budgets (0)

Resource Quotas (0)

More Resources >

Limit Ranges ☆

Create from YAML

Filter

State	Name	Created At
There are no rows to show.		



자원제한 대시보드 2

Limit Range: Create

```
1  apiVersion: v1
2  kind: LimitRange
3  metadata:
4    name: #string
5    namespace: default
6  # annotations: key: string
7  # labels: key: string
8  spec:
9  # limits:↔
```



자원제한 적용 1

YAML로 작성 시, 다음과 같이 구성이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: LimitRange
metadata:
  name: resource-limits
  namespace: default
spec:
  limits:
  - type: Container
    max:
      cpu: "2"
      memory: "1Gi"
```



자원제한 적용 2

YAML로 작성 시, 다음과 같이 구성이 가능하다.

```
min:
  cpu: "200m"
  memory: "128Mi"
default:
  cpu: "500m"
  memory: "256Mi"
defaultRequest:
  cpu: "250m"
  memory: "256Mi"
```

EOF



자원 총량 제한

네임스페이스에서 사용하는 총 자원에 대해서 제한을 거는 객체. 이를 통해서 네임스페이스에서 사용하는 대다수 자원에 대해서 개수 제한이 가능하다.

기능	설명
네임스페이스 단위 리소스 제한	CPU, Memory, Storage, Object 개수 제한 가능
팀별 리소스 분배	팀 단위 네임스페이스마다 자원 할당량을 나눌 수 있음
LimitRange와 연동	LimitRange 없이 Pod 생성 시 Quota를 충족 못 하면 에러 발생
오브젝트 개수 제한	예: pods, services, persistentvolumeclaims 등 개수 제한



자원 총량 제한 대시보드 1

Resource Quotas ☆ Create from YAML

☰

■

Filter

State ◇	Name ◇	Request ◇	Limit ◇	Age ◇
There are no rows to show.				



자원 총량 제한 대시보드 2

Resource Quota: Create

```
1  apiVersion: v1
2  kind: ResourceQuota
3  metadata:
4    name: #string
5    namespace: default
6    # annotations: key: string
7    # labels: key: string
8  spec:
9    # hard: key: string
10  # scopeSelector: ↗
16  # scopes: ↗
```



Resource Quotas

아래와 같이 YAML으로 구성이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev-ns
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: "4Gi"
    limits.cpu: "4"
    limits.memory: "8Gi"
    persistentvolumeclaims: "5"
```

EOF



NetworkPolicy 설명

네임스페이스 간의 접근을 위한 기능으로, L2/L3/L4 레이어를 거치지 않고 라우팅을 설정함으로써 오버헤드 없이 신속하게 접근할 수 있다. 동작 방식은 아래 컴포넌트 기반으로 구성이 된다.

- Container Network Interface(CNI)
- Netfilter(iptables, nftables)
- Flannel에서는 해당 기능은 지원하지 않음

위의 기능 기반으로 백-엔드에서 다음과 같이 구성한다. 다음과 같은 조건으로 구성이 되었다.

1. Namespace: test-np
2. Pod: nginx-server
3. 허용: allowed-pod만 80 포트로 접근 가능
4. 차단: denied-pod는 접근 차단



NetworkPolicy 대시보드 1

Network Policy: Create

Namespace *

default

▼

Name *

A unique name

Description

Any text you want that better describes this resource

Ingress Rules

Egress Rules

Selectors

Labels & Annotations

Ingress Rules

☐ Configure ingress rules to restrict incoming traffic



NetworkPolicy 대시보드 2

☐ State	Name	Pod-Selector	Age	
Namespace: cattle-fleet-local-system				
☐ Active	default-allow-all	<none>	1.2 days	⋮
Namespace: cattle-provisioning-capi-system				
☐ Active	default-allow-all	<none>	1.2 days	⋮
Namespace: cattle-resources-system				
☐ Active	rancher-backup-default-allow-all	<none>	21 hours	⋮



NetworkPolicy 대시보드 3

Ingress Rules

☒ Configure ingress rules to restrict incoming traffic

Rule 1

Sources 

Rule type
Namespace Selector

Matches 0 of 27 namespaces

Key	Operator	Value	
database	in list	mongodb	Remove

Add Rule

Add allowed traffic source

Allowed ports 

Port
27020

Protocol
TCP

Add allowed port

+ -



특정 POD만 허용 1

특정 POD만 허용하는 경우 YAML코드는 다음과 같다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-specific-pod
  namespace: test-np
spec:
  podSelector:
    matchLabels:
      app: nginx
  policyTypes:
  - Ingress
```



특정 POD만 허용

특정 POD만 허용하는 경우 YAML코드는 다음과 같다.

```
ingress:
- from:
  - podSelector:
      matchLabels:
        role: allowed
  ports:
  - protocol: TCP
    port: 80
```

"role=allowed" 라벨이 있는 Pod만 허용

EOF



네임스페이스+POD 차단

특정 네임스페이스에 속한 모든 POD를 차단하는 경우, 다음과 같이 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-namespace
  namespace: test-np
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```



네임스페이스+POD 차단

반드시, 네임 스페이스 생성 시 레이블 구성이 되어 있어야 한다.

```
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        project: restricted
  ports:
  - protocol: TCP
    port: 80
```

EOF

"project=restricted" 네임스페이스에
서 오는 트래픽 차단



네임스페이스+POD

특정 네임스페이스와 POD를 선택하는 경우, 다음과 같이 구성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-specific-namespace
  namespace: test-np
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
  - Ingress
```



네임스페이스+POD

특정 네임스페이스와 POD를 선택하는 경우, 다음과 같이 구성한다.

```
ingress:
```

```
- from:
```

```
  - namespaceSelector:
```

```
    matchLabels:
```

```
      env: dev
```

```
- podSelector:
```

```
  matchLabels:
```

```
    team: frontend
```

```
ports:
```

```
- protocol: TCP
```

```
  port: 443
```

"env=dev" 네임스페이스의 모든 Pod에서 접근 가능

"team=frontend" 라벨이 있는 Pod에서만 접근 가능



EOF

Pod Disruption Budget

PodDisruptionBudget(PDB)는 관리자가 계획된 작업(예: 노드 유지보수, 롤링 업데이트)을 할 때, 너무 많은 파드가 동시에 중단되지 않도록 보장하는 역할을 한다.

즉, 파드의 최소 가용성(minimum availability)을 정의하여 의도적인 중단(disruption)에 대한 허용 한계를 설정할 수 있다.

기능	설명
고가용성 보장	파드가 한꺼번에 너무 많이 내려가지 않도록 제어
롤링 업데이트 제어	파드 교체 시에도 일정 수 이상 항상 살아있도록 강제
계획된 노드 유지보수 대응	drain 명령 등으로 노드를 비울 때 제한이 걸림
voluntary disruption만 제어	관리자가 유발하는 의도적 중단만 제한, 장애나 crash는 제어 불가



Pod Disruption Budget

Pod Disruption Budget: Create

Namespace *

default

▼

Name *

A unique name

Description

Any text you want that better describes this resource

Budget

Selector

Labels & Annotations

Budget

Min. available Pods

Max. unavailable Pods



Pod Disruption Budget

아래와 같이 YAML으로 구성이 가능하다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: web
EOF
```



Longhorn

간단하게

설명

Longhorn은 클라우드 네이티브 환경에서 사용할 수 있는 분산 블록 스토리지 시스템으로, Kubernetes 전용으로 설계된 오픈소스 스토리지 솔루션이다.

기본적으로 Rancher Labs(현재 SUSE 소속)에서 개발되었고, CNCF에 기여된 프로젝트.

Longhorn은 각 노드의 로컬 디스크를 사용해서 안정적인 블록 스토리지 구성, 이를 기반으로 PVC 형태로 POD에 볼륨(디스크)를 제공한다.

내부적으로는 복제, 스냅샷, 백업, 자동 복구 등을 제공하며, 스토리지 고가용성과 운영 자동화를 쿠버네티스 환경에서 간단하게 구현할 수 있게 돕는다. 기본 복제자 개수는 3개로 구성이 되어 있다.



설명

여기서는 간단하게 Longhorn에 대해서 학습한다. REK2 및 Rancher에서 기본으로 제공하는 오브젝트 스토리지는 Longhorn이다. 블록 장치 형태로 제공하지 않으며, PV/PVC 기반으로 POD에 저장소를 제공한다.

Longhorn은 아직 ReadWriteMany는 지원하지 않지만, 추후에 ReadWriteMany도 지원할 예정이다. ReadWriteMany가 필요한 경우 아래와 같은 도구 사용을 권장한다.

RWX 지원되는 CSI

1. NFS (기본 RWX 가능)
2. CephFS (Ceph의 파일시스템, RWX 가능)
3. OpenEBS Mayastor (실험적 RWX 지원)
4. Portworx, GlusterFS 등



권장환경

다음과 같은 환경에서 Longhorn는 최고의 성능을 제공한다.

1. Rancher, RKE2, K3S 등 경량 쿠버네티스 환경
2. 단일 클러스터 혹은 엣지 클러스터
3. 스토리지 인프라가 복잡하지 않고, 로컬 디스크 기반의 저장소가 필요한 경우



Longhorn 대시보드 1

Cluster >

Workloads >

Apps >

Service Discovery >

Storage >

Policy >

Longhorn

Rancher Backups >

More Resources >

Overview

Powered By: [Longhorn](#)

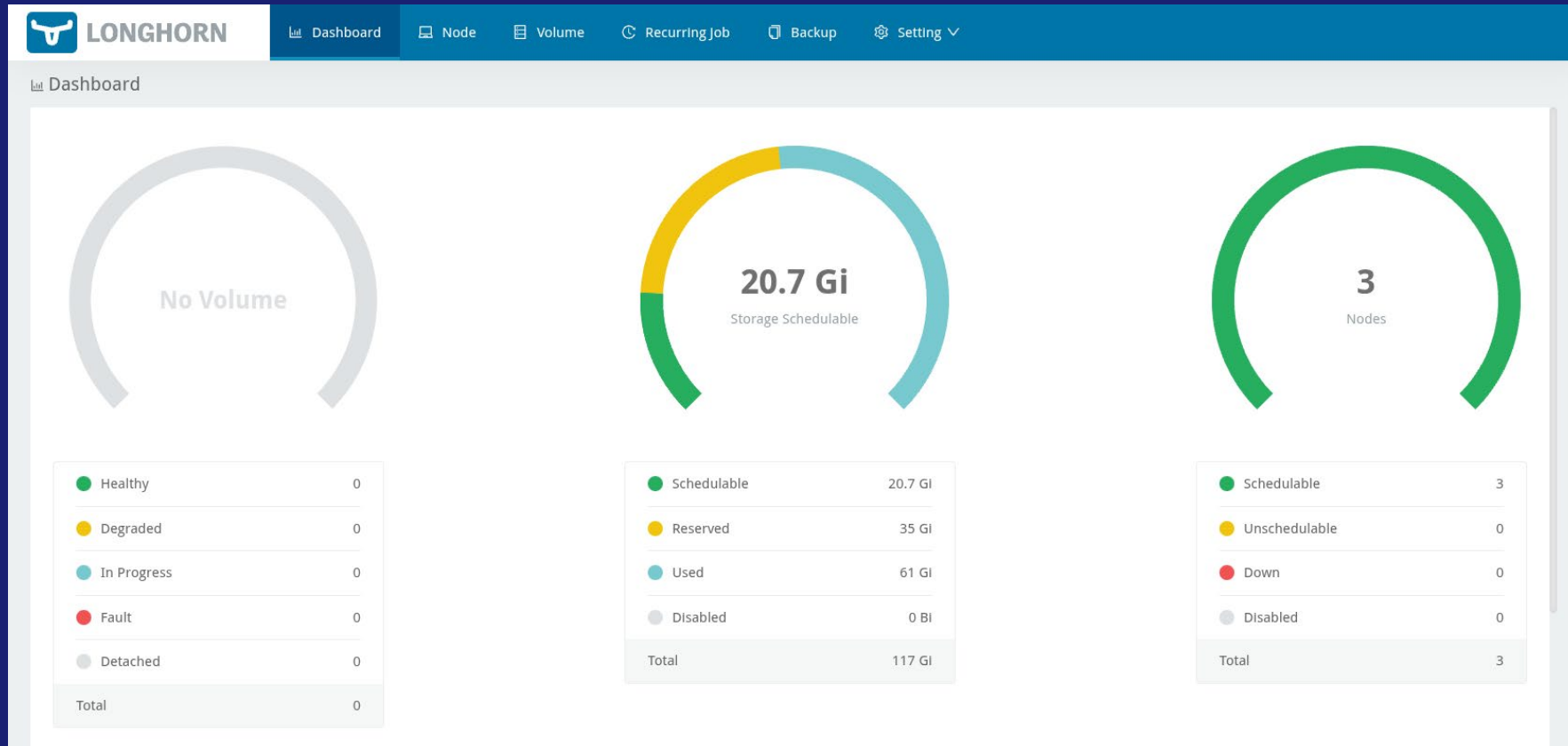


Longhorn

Manage storage system via UI



Longhorn 대시보드 2



Longhorn 기능

Longhorn에서 제공하는 기능은 다음과 같다.

메뉴 이름	기능 요약
Dashboard	클러스터 전체 상태를 요약하여 보여줌 (볼륨 상태, 노드 상태, 알림 등)
Node	Longhorn이 설치된 노드 목록과 각 노드의 디스크/상태/리소스 사용량 관리
Volume	생성된 Longhorn 볼륨 목록과 상태 관리, 볼륨 생성/삭제/확장 가능
Recurring Job	볼륨의 자동 스냅샷 및 백업 작업 스케줄을 설정 및 관리
Backup	스냅샷 백업 목록 확인, 복원 및 외부 백업 저장소(S3 등) 연결
Setting	Longhorn 설정(고급 설정, 백업 설정, UI 설정 등) 변경 가능



디스크 추가

Rancher에서 사용하기 위해서 디스크를 추가한다. 추가 위치는 사용자마다 다르기는 하겠지만, 일반적으로 `/var/lib/longhorn`에 구성한다.

구성 단계는 다음과 같다.

1. 디스크 파티션 생성 및 구성
2. ext4, xfs으로 포맷
3. 적절한 위치에 마운트 및 `/etc/fstab` 등록
4. dashboard에서 디스크 등록 및 확인



랜처 디스크 박스 구성 1

	Status	Readiness	Name	Replicas	Allocated	Used	Size	Tags	Operation
☐	☒	Schedulable	Ready	rancher-node1.example.com 11.44.0.40	0	0 / 27.19 Gi	25.73 / 48.87 Gi	27.2 Gi +21.7 Gi Reserved	
Disks									
☒	Schedulable		default-disk- fd0200000000	filesystem	Path: /var/lib/longhorn/	0	0 / 27.26 Gi	25.56 / 38.94 Gi +11.7 Gi Reserved	
☐	Disabled		disk-2	filesystem	Path: /mnt/vdb1	0	0 / -0.06 Gi	0.17 / 9.94 Gi -68177920 B +10 Gi Reserved	



랜처 디스크 박스 구성 2

Unschedulable	disk-2	filesystem	Path: /mnt/vdb1
☒	Schedulable	Ready	rancher-node2.example.com 11.44.2.26
Disks			
Schedulable	default-disk-fd0200000000	filesystem	Path: /var/lib/longhorn/
Schedulable	disk-5	filesystem	Path: /mnt/vdb1



디스크 박스 구성 1

Harvester에서 디스크 추가는 Rancher와 동일하다. 다만, 사용하는 O/S가 Immutable이라서 사용이 가능한 위치는 제한적이다. 권장하는 위치는 `/var/lib/longhorn`에 연결 및 구성한다.

디스크 파티션 생성 및 구성

1. ext4, xfs으로 포맷
2. 적절한 위치에 마운트 및 `/etc/fstab` 등록
3. dashboard에서 디스크 등록 및 확인



디스크 복제 개수

기본 값은 3으로 되어있다. 자원이 작은 시스템에서는 아래 부분은 조정해서 사용한다.

*** Default Minimum Number of BackingImage Copies**

② Required.

The default minimum number of backing image copies Longhorn maintains

*** Default Replica Count**

② Required.

The default number of replicas when a volume is created from the Longhorn UI. For Kubernetes configuration, update the `numberOfReplicas` in the StorageClass



참고

<https://longhorn.io/docs/1.9.0/best-practices/#node-and-disk-setup>

GPU

Nvidia Driver Test

GPU

Rancher에서 GPU드라이버는 제공한다. 현재 사용중인 클러스터는 아쉽게도 A100/200와 같은 장치가 없기 때문에, 가상으로 GPU를 구성하여 어떻게 Rancher에서 사용이 가능한지 확인한다.



GPU 드라이버 설치

설치 및 구성은 다음과 같이 한다.

```
kubectl delete runtimeclasses.node.k8s.io nvidia  
runtimeclass.node.k8s.io "nvidia" deleted
```

```
helm upgrade -i gpu-operator oci://ghcr.io/run-ai/fake-gpu-operator/fake-  
gpu-operator --namespace gpu-operator --create-namespace --version 0.0.63
```

```
kubectl get pods -n gpu-operator
```



테스트 자원 구성

런타임 클래스를 임의로 하나 생성 및 구성한다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: node.k8s.io/v1  
kind: RuntimeClass  
metadata:  
  name: fake-nvidia  
handler: runc  
EOF
```



테스트 자원 구성

테스트 용도 POD를 아래처럼 하나 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fake-gpu-device-plugin
  namespace: kube-system
spec:
  selector:
    matchLabels:
      name: fake-gpu-device-plugin
```



테스트 자원 구성

위의 내용 계속 이어서...

```
template:
  metadata:
    labels:
      name: fake-gpu-device-plugin
  spec:
    runtimeClassName: fake-nvidia
    containers:
      - name: fake-gpu
        image: busybox
        command: ["sh", "-c", "sleep 3600"]
```



테스트 자원 구성

위의 내용 계속 이어서...

```
resources:
  limits:
    nvidia.com/gpu: 1
  volumeMounts:
  - name: dev
    mountPath: /dev
tolerations:
- key: "CriticalAddonsOnly"
  operator: "Exists"
```



테스트 자원 구성

POD를 생성 및 적용한다.

```
volumes:  
  - name: dev  
    hostPath:  
      path: /dev
```

EOF

```
# kubectl get pods -o widekubectl describe pod fake-gpu-test
```

